

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DA PARAÍBA
CAMPUS CAMPINA GRANDE
COORDENAÇÃO DO CURSO SUPERIOR BACHARELADO EM ENGENHARIA DE COMPUTAÇÃO

LUIZ OLIVEIRA DE SOUZA NETO

JIGA DE TESTES *HARDWARE-IN-THE-LOOP* DE BAIXO CUSTO PARA VALIDAÇÃO *BLACK-BOX* DE *FIRMWARE* EMBARCADO

CAMPINA GRANDE – PB
2026

LUIZ OLIVEIRA DE SOUZA NETO

JIGA DE TESTES *HARDWARE-IN-THE-LOOP* DE BAIXO CUSTO PARA VALIDAÇÃO *BLACK-BOX* DE *FIRMWARE* EMBARCADO

Monografia apresentada ao Curso de Bacharelado em Engenharia de Computação, do Instituto Federal de Educação, Ciência e Tecnologia da Paraíba – Campus Campina Grande, em cumprimento às exigências parciais para a obtenção do título de Bacharel em Engenharia de Computação.

Orientador: Prof. Alexandre Sales Vasconcelos

CAMPINA GRANDE – PB
2026

Souza Neto, Luiz Oliveira de.

Jiga de testes *Hardware-in-the-Loop* de baixo custo para validação *black-box* de *firmware* embarcado / Luiz Oliveira de Souza Neto. – Campina Grande, 2026.

96 f. : il. color.

Trabalho de Conclusão de Curso (Bacharelado em Engenharia de Computação) – Instituto Federal de Educação, Ciência e Tecnologia da Paraíba, Campus Campina Grande, 2026.

Orientador: Prof. Alexandre Sales Vasconcelos.

1. Sistemas embarcados. 2. *Hardware-in-the-loop*. 3. Jiga de testes. 4. Teste *black-box*. 5. Validação de *firmware*. I. Vasconcelos, Alexandre Sales. II. Título.

LUIZ OLIVEIRA DE SOUZA NETO

JIGA DE TESTES *HARDWARE-IN-THE-LOOP* DE BAIXO CUSTO PARA VALIDAÇÃO *BLACK-BOX* DE *FIRMWARE* EMBARCADO

Monografia apresentada ao Curso de Bacharelado em Engenharia de Computação, do Instituto Federal de Educação, Ciência e Tecnologia da Paraíba – Campus Campina Grande, em cumprimento às exigências parciais para a obtenção do título de Bacharel em Engenharia de Computação.

Aprovada em 19 de novembro de 2026.

BANCA EXAMINADORA:

Prof. Alexandre Sales Vasconcelos(Orientador)
Instituto Federal da Paraíba (IFPB)

Prof. Fagner de Araujo Pereira
Instituto Federal da Paraíba (IFPB)

Prof. Moacy Pereira da Silva
Instituto Federal da Paraíba (IFPB)

CAMPINA GRANDE – PB
2026

AGRADECIMENTOS

Agradeço primeiramente a Deus, por ter me permitido chegar até aqui.

À minha família, pelo apoio e pela paciência ao longo de todo o curso. Sei que nem sempre foi fácil, e vocês estiveram comigo do começo ao fim.

Aos amigos que fiz no curso, pela companhia ao longo desses anos. Em especial a Gabriel e Ryan, os irmãos que o curso me deu, que foram fundamentais para que eu não desistisse. Obrigado pelas conversas, pela parceria e pela ajuda em tantos momentos difíceis.

Ao professor Fagner, uma das minhas maiores inspirações como professor e como profissional, por ter me dado a primeira oportunidade em um projeto de PD&I. Aquela chance mudou muita coisa no meu caminho.

Ao professor Alexandre, meu orientador, pela paciência comigo e por ser um exemplo de trabalho bem feito e de entregas de alto nível.

E ao professor Moacy, pelos inúmeros conselhos ao longo dessa caminhada e por ser o exemplo do homem que eu quero ser no futuro.

RESUMO

A validação de *firmware* de sistemas embarcados costuma depender de sensores reais ligados ao *hardware* final, o que limita a reprodutibilidade dos testes e expõe componentes a danos, enquanto as plataformas comerciais de *Hardware-in-the-Loop* (HIL) têm custo inviável para laboratórios acadêmicos e pequenas equipes. Este trabalho teve como objetivo desenvolver uma jiga de testes de baixo custo, baseada no microcontrolador STM32F7, que simule sinais de sensores e periféricos e automatize a validação *black-box* do *firmware* de microcontroladores-alvo. A jiga gera estímulos analógicos, digitais e PWM, emula sensores comerciais nos barramentos I2C e SPI e adquire as respostas do dispositivo sob teste; uma aplicação *desktop* executa baterias de teste descritas em arquivo e emite o veredito de cada caso, que a jiga executa como uma rotina temporizada com resolução de 1 µs. A verificação foi feita nos níveis de unidade, integração, sistema e aceitação, com medições conferidas por multímetro e osciloscópio. Os conversores atenderam ao critério de 1% do fundo de escala, e as esperas das rotinas ficaram até 24 µs acima do pedido, contra variações de milissegundos quando comandadas pelo computador. Na validação com um ESP32-S3, o *firmware* correto foi aprovado em todas as execuções, as quatro versões com defeitos injetados foram reprovadas em todas as execuções, e o dispositivo leu os sensores emulados com os seus próprios *drivers*. Conclui-se que uma jiga de cerca de R$ 200 torna viável o teste automatizado de *firmware* com estímulos elétricos reais, em malha aberta, no contexto acadêmico.

Palavras-chave: Sistemas embarcados. *Hardware-in-the-loop*. Jiga de testes. Teste *black-box*. Validação de *firmware*.

ABSTRACT

Validating embedded-system firmware usually depends on real sensors connected to the final hardware, which limits test reproducibility and exposes components to damage, while commercial Hardware-in-the-Loop (HIL) platforms are too expensive for academic laboratories and small teams. This work aimed to develop a low-cost test jig, based on the STM32F7 microcontroller, that simulates sensor and peripheral signals and automates the black-box validation of target microcontroller firmware. The jig generates analog, digital and PWM stimuli, emulates commercial sensors on the I2C and SPI buses and acquires the responses of the device under test; a desktop application runs test batteries described in files and issues the verdict of each case, which the jig executes as a timed routine with 1 µs resolution. Verification covered the unit, integration, system and acceptance levels, with measurements checked against a multimeter and an oscilloscope. The converters met the 1% full-scale criterion, and routine waits were at most 24 µs longer than requested, compared with millisecond variations when commanded by the computer. In the validation with an ESP32-S3, the correct firmware passed every run, the four versions with injected defects were rejected in every run, and the device read the emulated sensors with its own drivers. The results show that a jig costing about US$ 36 makes automated firmware testing with real electrical stimuli, in open loop, feasible in academic settings.

Keywords: Embedded systems. Hardware-in-the-loop. Test jig. Black-box testing. Firmware validation.

LISTA DE ILUSTRAÇÕES

Figura 1 – Arquitetura em três camadas da jiga de testes. 34
Figura 2 – Esquema de ligação entre a jiga e o DUT. 37
Figura 3 – Arquitetura em camadas do firmware. 39
Figura 4 – Sequência de processamento de um comando direto no firmware. . 42
Figura 5 – Máquina de estados do receptor de quadros. 42
Figura 6 – Estados de execução de uma rotina na `TestExecTask`. 43
Figura 7 – Ciclo de execução de uma rotina entre o computador e a jiga. 47
Figura 8 – Formato do quadro do protocolo PC–Jiga. 48
Figura 9 – Diagrama de classes da aplicação *desktop*. 50
Figura 10 – Fluxograma de execução de uma bateria de testes. 52
Figura 11 – Janela da aplicação após a execução da bateria de *loopback*. 54
Figura 12 – Bancada de ensaios: jiga (NUCLEO-F767ZI), DUT (ESP32-S3), *jumpers* e matriz de contatos com os resistores. 55
Figura 13 – Erro da leitura do ADC no *loopback* DAC→ADC (média e faixa de 16 leituras por ponto). 58
Figura 14 – PWM2 em 20 kHz e 12,34% no osciloscópio (CT12). 59
Figura 15 – Pulso de 10 ms gerado no GPIO1 por uma rotina, no osciloscópio (CT19). 61
Figura 16 – Degrau do DAC1 (canal 1) e reação da saída do ventilador no ESP32-S3 (canal 2), no osciloscópio (CT22). 63
Figura 17 – Janela da aplicação após o CT22 com o *firmware* sem histerese. . . 64
Figura 18 – Distribuição do tempo de ida e volta dos comandos (intervalos de 0,1 ms). 65
Figura 19 – Senoide de 0,5 a 2,5 V e 1 kHz gerada no DAC1, no osciloscópio (CT27). 67
Figura 20 – Retransmissão após perda da resposta. 86

LISTA DE QUADROS

Quadro 1 – Comparação entre a jiga e trabalhos e ferramentas relacionados. . 25
Quadro 2 – Requisitos funcionais (RF). 28
Quadro 3 – Requisitos não funcionais (RNF). 29
Quadro 4 – Rastreabilidade entre requisitos e níveis de verificação. 30
Quadro 5 – Critérios de aceitação das grandezas medidas. 30
Quadro 6 – Instrumentos de medição. 33
Quadro 7 – Tarefas do FreeRTOS. 40
Quadro 8 – Atendimento aos requisitos. 68
Quadro 9 – Objetivos específicos e evidências. 70
Quadro 10 – Campos do quadro do protocolo. 79
Quadro 11 – Códigos de erro do NACK. 80
Quadro 12 – Comandos diretos de sistema, DAC, GPIO, PWM e ADC. 81
Quadro 13 – Comandos diretos de I2C, SPI e UART. 82
Quadro 14 – Comandos de rotina. 83
Quadro 15 – Passos admitidos em uma rotina. 84
Quadro 16 – Estados de execução de uma rotina. 85
Quadro 17 – Exemplos de quadros de comandos diretos (bytes em hexadecimal). 87
Quadro 18 – Exemplos de quadros de uma rotina (bytes em hexadecimal). . . . 88
Quadro 19 – Mapeamento de pinos dos conectores CN7 e CN8. 89
Quadro 20 – Mapeamento de pinos dos conectores CN9 e CN10. 89
Quadro 21 – Casos de teste de unidade e de integração com comandos diretos. 91
Quadro 22 – Casos de teste das rotinas. 92
Quadro 23 – Casos de teste de sistema, de aceitação, de desempenho e do gerador de ondas. 93
Quadro 24 – Itens de uma bateria de testes. 94

LISTA DE CÓDIGOS

Código 1 – Exemplo de bateria de testes. 51
Código 2 – Bateria do CT22 (ventilador com histerese). 95
Código 3 – Trecho do relatório CSV de uma execução do CT22 no ESP32-S3. 96

LISTA DE TABELAS

Tabela 1 – Lista de materiais da jiga de testes. 38
Tabela 2 – Testes automatizados e resultados na versão 1.0.0. 56
Tabela 3 – Cobertura dos testes executados no computador. 56
Tabela 4 – Tensão de saída do DAC1 (PA4) medida com multímetro, sem carga, nos 11 pontos do CT10. 57
Tabela 5 – Resumo do *loopback* DAC→ADC na faixa útil (33 pontos, 16 leituras por ponto). 58
Tabela 6 – PWM medido com osciloscópio (CT12). 59
Tabela 7 – Largura dos pulsos do GPIO1 medida com osciloscópio (CT19). . . 61
Tabela 8 – Vereditos do CT22 com defeitos injetados no *firmware* do DUT. . . . 63
Tabela 9 – Tempo de ida e volta dos comandos (CT23 e CT24). 64

LISTA DE ABREVIATURAS E SIGLAS

ACK	*Acknowledgement* (confirmação)
ADC	*Analog-to-Digital Converter* (conversor analógico-digital)
ARQ	*Automatic Repeat reQuest* (repetição automática)
CMSIS	*Common Microcontroller Software Interface Standard*
CRC	*Cyclic Redundancy Check* (verificação de redundância cíclica)
CS	*Chip Select* (seleção de escravo)
CSV	*Comma-Separated Values*
DAC	*Digital-to-Analog Converter* (conversor digital-analógico)
DMA	*Direct Memory Access* (acesso direto à memória)
DNL	*Differential Nonlinearity* (não linearidade diferencial)
DUT	*Device Under Test* (dispositivo sob teste)
FIFO	*First In, First Out*
FT	*Five-volt Tolerant* (tolerante a 5 V)
GPIO	*General Purpose Input/Output*
HAL	*Hardware Abstraction Layer* (camada de abstração de *hardware*)
HIL	*Hardware-in-the-Loop*
I2C	*Inter-Integrated Circuit*
IA	Inteligência Artificial
INL	*Integral Nonlinearity* (não linearidade integral)
JSON	*JavaScript Object Notation*
LSB	*Least Significant Bit* (bit menos significativo)
MCU	*Microcontroller Unit*
NACK	*Negative Acknowledgement* (recusa)
OTG	*On-The-Go*

PTAX	Taxa de câmbio de referência do Banco Central do Brasil
PWM	*Pulse Width Modulation* (modulação por largura de pulso)
RF	Requisito funcional
RNF	Requisito não funcional
RTOS	*Real-Time Operating System* (sistema operacional de tempo real)
SEQ	Número de sequência do quadro
SOF	*Start Of Frame* (início de quadro)
SPI	*Serial Peripheral Interface*
UART	*Universal Asynchronous Receiver-Transmitter*
USB	*Universal Serial Bus*
USB-CDC	*Universal Serial Bus – Communications Device Class*
YAML	*YAML Ain't Markup Language*

SUMÁRIO

1	**INTRODUÇÃO**	**17**
1.1	Contextualização e Descrição do Tema	17
1.2	Identificação da Lacuna e Formulação do Problema	17
1.3	Justificativa	18
1.4	Objetivos	18
1.4.1	Objetivo Geral	18
1.4.2	Objetivos Específicos	18
1.5	Escopo e Limites do Trabalho	19
1.6	Uso de Inteligência Artificial	19
1.7	Organização do Documento	20
2	**FUNDAMENTAÇÃO TEÓRICA E TRABALHOS RELACIONADOS**	**21**
2.1	Conceitos	21
2.2	Fundamentos Técnicos	22
2.2.1	Sistemas Operacionais de Tempo Real	22
2.2.2	Enquadramento, Detecção de Erros e Retransmissão	22
2.2.3	Barramentos Seriais e o Papel de Escravo	22
2.2.4	Conversores Analógico-Digitais e Digital-Analógicos	23
2.3	O Problema: Limitações da Validação com Sensores Reais	23
2.4	Trabalhos e Ferramentas Relacionados	24
2.4.1	Plataformas Comerciais de HIL	24
2.4.2	HIL de Baixo Custo	24
2.4.3	*Frameworks* e Equipamentos de Teste de *Hardware*	24
2.4.4	Quadro Comparativo	25
3	**METODOLOGIA**	**27**
3.1	Caracterização da Pesquisa	27
3.2	Requisitos	27
3.3	Estratégia de Verificação	28
3.3.1	Níveis de Teste	28
3.3.2	Critérios de Aceitação	30
3.3.3	Casos de Teste	31
3.3.4	Testes de Desempenho	32
3.3.5	Instrumentos de Medição	32
4	**DESENVOLVIMENTO DA JIGA**	**34**

4.1 Visão Geral da Solução . . . 34
4.2 Projeto de Hardware . . . 35
4.2.1 Escolha do Microcontrolador . . . 35
4.2.2 Mapeamento de Pinos . . . 35
4.2.3 Compatibilidade de Níveis de Tensão . . . 35
4.2.4 Conversores Analógicos . . . 36
4.2.5 Esquema de Ligação com o DUT . . . 37
4.2.6 Lista de Materiais e Custo . . . 38
4.3 Projeto de Firmware . . . 38
4.3.1 Arquitetura em Camadas . . . 39
4.3.2 Tarefas e Sincronização . . . 40
4.3.3 Processamento de um Comando . . . 41
4.3.4 Máquina de Estados do Receptor de Quadros . . . 41
4.3.5 Interpretador de Rotinas . . . 43
4.3.6 Emulação de Sensores . . . 44
4.3.7 Gerador de Ondas . . . 45
4.4 Protocolo de Comunicação PC–Jiga . . . 46
4.4.1 Meio de Transmissão . . . 46
4.4.2 Modelo de Comunicação . . . 46
4.4.3 Rotinas Executadas na Jiga . . . 46
4.4.4 Formato do Quadro . . . 48
4.4.5 Detecção de Erros e Retransmissão . . . 48
4.4.6 Comandos e Papéis nos Barramentos . . . 49
4.5 Aplicação Desktop . . . 49
4.5.1 Escolha da Linguagem . . . 49
4.5.2 Arquitetura da Aplicação . . . 49
4.5.3 Descrição dos Testes . . . 50
4.5.4 Execução de uma Bateria . . . 52

5 RESULTADOS E DISCUSSÕES . . . 54
5.1 Solução Implementada . . . 54
5.2 Testes Automatizados e Cobertura . . . 55
5.3 Testes de Unidade . . . 56
5.4 Testes de Integração com Comandos Diretos . . . 57
5.4.1 DAC e ADC . . . 57
5.4.2 PWM . . . 59
5.4.3 I2C, SPI e UART . . . 60
5.5 Rotinas e Base de Tempo . . . 60
5.6 Testes de Sistema e de Aceitação . . . 61
5.7 Desempenho da Comunicação . . . 63

5.8 Emulação de Sensores e Gerador de Ondas 65
5.9 Custo 66
5.10 Discussão 67
5.11 Atendimento aos Requisitos 68
5.12 Limitações da Avaliação 68

6 CONSIDERAÇÕES FINAIS E SUGESTÕES PARA TRABALHOS FUTUROS 70
6.1 Sugestões para Trabalhos Futuros 71

REFERÊNCIAS 73

APÊNDICES 77

APÊNDICE A – ESPECIFICAÇÃO COMPLETA DO PROTOCOLO DE COMUNICAÇÃO 78
A.1 Princípios e Convenções 78
A.2 Formato do Quadro 79
A.3 Verificação de Integridade 79
A.4 Respostas 79
A.4.1 ACK 79
A.4.2 NACK 80
A.5 Comandos Diretos 80
A.5.1 Emulação de dispositivos I2C e SPI 81
A.5.2 Emulação de sensores 82
A.6 Comandos de Rotina 83
A.6.1 Formato da Rotina 83
A.6.2 Passo ONDA 84
A.6.3 Formato dos Resultados 85
A.6.4 Estados de Execução 85
A.6.5 Regras de Execução 85
A.7 Sequenciamento e Retransmissão 86
A.8 Estado Seguro 87
A.9 Exemplos de Quadros 87

APÊNDICE B – MAPEAMENTO DE PINOS 89

APÊNDICE C – CASOS DE TESTE 90

APÊNDICE D – BATERIA DE TESTES E RELATÓRIO DE EXECUÇÃO 94

1 INTRODUÇÃO

1.1 CONTEXTUALIZAÇÃO E DESCRIÇÃO DO TEMA

Validar o *firmware* de um sistema embarcado é confirmar que o dispositivo responde como especificado aos estímulos que chegam às suas entradas. Para isso, o código é compilado, gravado no microcontrolador-alvo e exercitado com sinais de sensores e periféricos. Em projetos acadêmicos, de pesquisa e de pequenas empresas de tecnologia, esses sinais costumam vir de sensores reais ligados ao *hardware* final, e o resultado é observado manualmente (Euripedes Filho, 2017).

Este trabalho investiga uma alternativa a essa prática: uma jiga de testes que gera os sinais de sensores e periféricos de forma controlada, aplica-os ao dispositivo sob teste e decide automaticamente se a resposta está correta. O objetivo é tornar a validação mais rápida, segura e reprodutível, sem depender de sensores físicos.

1.2 IDENTIFICAÇÃO DA LACUNA E FORMULAÇÃO DO PROBLEMA

Soluções comerciais de *Hardware-in-the-Loop* (HIL), como as da dSPACE e da National Instruments, são eficazes e amplamente usadas nas indústrias automobilística e aeroespacial (dSPACE, [2026?]; National Instruments, [2026?]). Os fabricantes não publicam preços, mas registros públicos de compra mostram valores de dezenas a centenas de milhares de dólares por sistema (Frassinelli, 2026), o que os torna inacessíveis para laboratórios universitários e pequenas empresas (seção 2.4). Sem essas opções, a alternativa mais adotada é a validação manual, em que sensores reais são ligados ao *hardware* final e as variáveis de ambiente são alteradas fisicamente.

Esse método funciona, mas é pouco reprodutível, lento e limitado nos cenários que consegue cobrir. Condições ambientais extremas, como temperaturas ou pressões fora do comum, dificilmente são reproduzidas em bancada. Validar o tratamento de falhas exige sensores defeituosos ou sinais fora do padrão. Repetir manualmente os testes de regressão a cada alteração do *firmware* é demorado e expõe sensores caros ou frágeis a danos. Falta, portanto, uma forma acessível de simular sinais de sensores e periféricos no ciclo de desenvolvimento de *firmware* de laboratórios acadêmicos e pequenas equipes.

Diante do exposto, esta pesquisa busca responder ao seguinte problema: como validar o *firmware* de sistemas embarcados de forma reprodutível e de baixo custo, simulando sinais de sensores e periféricos, sem depender de sensores reais ou de plataformas comerciais de HIL?

1.3 JUSTIFICATIVA

Do ponto de vista acadêmico, o trabalho ocupa o espaço entre duas linhas já consolidadas, discutidas no Capítulo 2. De um lado, as técnicas de teste *off-target* validam a lógica do *software* sem o *hardware* real (Lins, 2016; Branam, 2026), mas não verificam o *firmware* diante de estímulos elétricos. De outro, as plataformas comerciais de HIL verificam esse comportamento, mas a um custo inviável para o meio acadêmico. Uma jiga de baixo custo, com protocolo documentado e rotinas de teste reprodutíveis, valida o *firmware* no próprio *hardware*-alvo sem recorrer a nenhum dos dois extremos. A especificação do protocolo (Apêndice A) e o código, publicados em repositório aberto, permitem reproduzir e estender a solução e usá-la no ensino de sistemas embarcados, a exemplo dos laboratórios descritos por Jacko *et al.* (2022).

Do ponto de vista prático, laboratórios acadêmicos e pequenas empresas, justamente os que não têm acesso às plataformas comerciais, são os que mais dependem da validação manual. Automatizar essas rotinas reduz o tempo dos testes de regressão, diminui o risco de dano a sensores e permite exercitar cenários de falha difíceis de reproduzir em bancada. O tema também reúne conteúdos centrais da formação em Engenharia de Computação: microcontroladores, sistemas operacionais de tempo real, protocolos de comunicação e verificação de *software*.

1.4 OBJETIVOS

1.4.1 Objetivo Geral

Desenvolver uma jiga de testes de baixo custo, baseada no microcontrolador STM32F7, que simule sinais de sensores e periféricos e automatize, por meio de uma aplicação *desktop*, a validação *black-box* do *firmware* de microcontroladores-alvo, de forma reprodutível e sem depender de sensores reais ou de plataformas comerciais de HIL.

1.4.2 Objetivos Específicos

Para alcançar o objetivo geral, foram definidos os seguintes objetivos específicos:

a) implementar no STM32F7 um *firmware* que gere estímulos analógicos (DAC), digitais (GPIO) e PWM, adquira as respostas do DUT (ADC e GPIO) e emule periféricos e sensores nos barramentos I2C, SPI e UART, incluindo formas de onda geradas pela própria jiga;

b) especificar e implementar um protocolo de comunicação entre o computador e a jiga, com detecção de erros, retransmissão e execução de rotinas temporizadas na própria jiga;

c) desenvolver uma aplicação *desktop* que descreva baterias de teste em arquivo, execute-as na jiga e emita o veredito de cada caso (passou ou falhou), com relatório;

d) verificar a jiga nos níveis de unidade, integração e sistema, conferindo as suas medições com instrumentos independentes;

e) validar a solução com um microcontrolador-alvo, o ESP32-S3, executando *firmware* de comportamento conhecido, e mostrar que a jiga reprova versões desse *firmware* com defeitos injetados.

1.5 ESCOPO E LIMITES DO TRABALHO

Delimitam-se, a seguir, os limites do trabalho:

- a jiga não substitui plataformas industriais de HIL: os estímulos seguem roteiros definidos no teste, em malha aberta, sem um modelo da planta que reaja às saídas do DUT (seção 2.1);
- não são desenvolvidos modelos matemáticos complexos de sensores; a simulação se baseia em perfis de sinal configuráveis e em mapas de registradores de sensores comerciais;
- a ligação entre a jiga e o DUT é feita por *jumpers*, sem placa de interface dedicada;
- os testes são *black-box*, sem instrumentação interna do *firmware* do DUT;
- a temporização se limita à resolução dos periféricos do STM32F7, de 1 µs na base de tempo das rotinas.

1.6 USO DE INTELIGÊNCIA ARTIFICIAL

O desenvolvimento deste trabalho observa princípios de integridade acadêmica e de uso responsável de ferramentas de Inteligência Artificial (IA). As ferramentas de IA foram empregadas exclusivamente como assistentes de apoio, não substituindo a autoria, o julgamento técnico ou a responsabilidade do autor sobre o conteúdo produzido: a IA Gemini foi utilizada para a revisão ortográfica e gramatical dos textos e para o auxílio na geração de tabelas, enquanto a IA Claude foi utilizada como agente de apoio à construção do código, à solução de *bugs* e à sugestão de correções na lógica do *software* e do *firmware*. Em ambos os casos, todo o conteúdo gerado com auxílio dessas ferramentas foi revisado, validado e adaptado pelo autor, que assume total responsabilidade pela originalidade, integridade e por todo o conteúdo final apresentado neste trabalho.

1.7 ORGANIZAÇÃO DO DOCUMENTO

O Capítulo 2 apresenta os fundamentos técnicos usados no trabalho e compara a proposta com trabalhos e ferramentas relacionados. O Capítulo 3 descreve como o trabalho foi conduzido e avaliado: requisitos, estratégia de verificação, critérios de aceitação e instrumentos. O Capítulo 4 apresenta a jiga construída: *hardware*, *firmware*, protocolo e aplicação. O Capítulo 5 apresenta e discute os resultados, e o Capítulo 6 traz as considerações finais. A especificação completa do protocolo está no Apêndice A.

2 FUNDAMENTAÇÃO TEÓRICA E TRABALHOS RELACIONADOS

Este capítulo apresenta os conceitos e os fundamentos técnicos em que a jiga se apoia, o problema que ela enfrenta e os trabalhos e ferramentas com que se relaciona.

2.1 CONCEITOS

Um **sistema embarcado** é um sistema computacional dedicado, com *hardware* e *software* projetados para executar uma função específica dentro de um dispositivo maior, em geral sob restrições de custo, energia e tempo de resposta (Chagas, 2020). O *software* que o controla, gravado no microcontrolador, é o *firmware*. **Validar** o *firmware* é confirmar que o dispositivo se comporta como especificado diante de estímulos controlados nas suas entradas.

As técnicas de teste se dividem conforme o acesso à implementação. No teste ***white-box***, os casos são derivados da estrutura interna do código. No teste ***black-box***, adotado neste trabalho, o dispositivo sob teste (DUT, *Device Under Test*) é avaliado apenas pelas respostas que dá aos estímulos aplicados, sem acesso à lógica interna (Myers; Badgett; Sandler, 2011; Euripedes Filho, 2017). Como não depende do código do DUT, o teste *black-box* se aplica a microcontroladores de fabricantes e arquiteturas diferentes.

Os testes também se organizam em **níveis**, cada um com um objeto e um objetivo próprios. Os testes de unidade verificam componentes isolados; os de integração, a interação entre componentes; os de sistema, o produto completo; e os de aceitação, se o produto atende ao uso pretendido (International Software Testing Qualifications Board, 2024). A estratégia de verificação deste trabalho (seção 3.3) segue essa divisão.

A técnica de ***Hardware-in-the-Loop*** (HIL) combina um equipamento real, em geral o controlador embarcado, com um modelo simulado do restante do sistema, executado em tempo real (Fathy *et al.*, 2006). No HIL clássico, a malha é fechada: o simulador lê as saídas do controlador, calcula a resposta da planta e devolve os sinais de sensor correspondentes. A técnica nasceu na indústria aeroespacial e automobilística e hoje é usada também em eletrônica de potência e acionamentos (Mihalič; Truntič; Hren, 2022).

A jiga deste trabalho aplica o princípio do HIL de forma simplificada: o DUT real recebe sinais simulados de sensores e periféricos, mas os estímulos seguem um roteiro definido no teste. As saídas do DUT são registradas e comparadas com o esperado, sem realimentar um modelo da planta. Trata-se, portanto, de um **HIL de**

malha aberta, ou de um testador funcional com emulação de periféricos. O termo HIL é mantido porque o princípio, um *hardware* real exercitado por um ambiente simulado em tempo real, é o mesmo; a ausência do modelo em malha fechada é uma limitação declarada (seção 1.5).

2.2 FUNDAMENTOS TÉCNICOS

2.2.1 Sistemas Operacionais de Tempo Real

Um sistema operacional de tempo real (RTOS, *Real-Time Operating System*) divide o *firmware* em tarefas, cada uma com uma prioridade, e garante que a tarefa pronta de maior prioridade seja executada primeiro. No FreeRTOS, usado neste trabalho, o escalonador é preemptivo: uma tarefa de maior prioridade que fica pronta interrompe a que está em execução (Barry, 2016). As tarefas se comunicam por filas, que copiam mensagens entre elas, e se sincronizam por notificações e semáforos. Um *mutex* protege um recurso compartilhado e, com herança de prioridade, evita que uma tarefa de baixa prioridade bloqueie indefinidamente uma de alta (Barry, 2016). Esses mecanismos permitem separar a comunicação com o computador, a execução dos testes e a emulação dos barramentos em tarefas independentes, cada uma com a urgência adequada.

2.2.2 Enquadramento, Detecção de Erros e Retransmissão

Em um canal serial, os bytes chegam em sequência, e o receptor precisa saber onde cada mensagem começa e termina. O **enquadramento** resolve isso, por exemplo com um byte de início seguido de um campo de comprimento (Tanenbaum; Wetherall, 2011). Para detectar bytes corrompidos, o transmissor acrescenta ao quadro um código de redundância cíclica (CRC), que o receptor recalcula. Um CRC de 16 bits detecta todos os erros em rajada de até 16 bits, e a escolha do polinômio determina quantos erros isolados são detectados para cada tamanho de mensagem (Koopman; Chakravarty, 2004). Quando um quadro se perde ou chega corrompido, o protocolo ***stop-and-wait*** com repetição automática (ARQ, *Automatic Repeat reQuest*) resolve: o transmissor envia um quadro, espera a confirmação e, se ela não chega no tempo limite, retransmite. Um número de sequência permite ao receptor reconhecer uma retransmissão e não executar o mesmo comando duas vezes (Tanenbaum; Wetherall, 2011).

2.2.3 Barramentos Seriais e o Papel de Escravo

Os sensores e periféricos que a jiga emula se comunicam por três barramentos seriais. O **I2C** usa duas linhas, dados (SDA) e relógio (SCL), em dreno aberto com

resistores de *pull-up*. O controlador (mestre) inicia cada transferência com o endereço do alvo (escravo), e os modos padrão e rápido operam a até 100 e 400 kbit/s (NXP Semiconductors, 2021). O escravo pode estender o pulso de relógio (*clock stretching*) enquanto prepara o próximo byte. O **SPI** usa linhas separadas para relógio, dados em cada sentido e seleção do escravo (CS). É síncrono, sem endereçamento nem confirmação, e o escravo não controla o ritmo da transferência, o que exige que ele tenha o dado pronto quando o mestre gera o relógio (STMicroelectronics, 2018). A **UART** é assíncrona, sem relógio comum: os dois lados combinam a taxa de transmissão, e cada byte é delimitado por bits de início e de parada (STMicroelectronics, 2018).

Emular um sensor significa fazer a jiga ocupar o papel de escravo, respondendo ao DUT como o sensor real responderia. Para isso, ela deve expor um mapa de registradores com o mesmo endereçamento e o mesmo formato de dados da folha de dados do sensor, e responder dentro dos tempos do barramento.

2.2.4 Conversores Analógico-Digitais e Digital-Analógicos

Os conversores digital-analógico (DAC) e analógico-digital (ADC) são caracterizados por erros estáticos. O erro de *offset* é o desvio da saída no código que deveria produzir zero, e o erro de ganho é o desvio da inclinação da curva de transferência, depois de removido o *offset*. As não linearidades diferencial (DNL) e integral (INL) medem o quanto cada degrau e a curva inteira se afastam da reta ideal (Kester, 2005). Esses erros são expressos em LSB ou em porcentagem do fundo de escala, e é nesses termos que se definem os critérios de exatidão do DAC e do ADC da jiga (subseção 3.3.2).

2.3 O PROBLEMA: LIMITAÇÕES DA VALIDAÇÃO COM SENSORES REAIS

Ligar sensores reais ao *hardware* final e observar o comportamento do dispositivo é simples, mas pouco reprodutível. Variações de temperatura, luminosidade ou pressão são difíceis de controlar em bancada, e a repetição de um cenário raramente produz condições idênticas (Euripedes Filho, 2017).

Parte da literatura propõe alternativas que dispensam o *hardware*. Lins (2016) investigam o desenvolvimento dirigido por testes para sistemas embarcados de tempo real, separando o código das dependências físicas por substitutos (*mocks*, *stubs* e *fakes*). Grenning (2011) sistematiza a mesma abordagem para C embarcado, e Branam (2026) descreve o uso do Ceedling, que combina Unity e CMock, em testes unitários *off-target*. Essas abordagens validam a lógica do *software*, mas não o *firmware* executando no *hardware*-alvo e reagindo a estímulos elétricos, que é o que este trabalho pretende tornar acessível.

Barbosa (2011) e Mendonça (2019), ao definirem metodologias de teste

para sistemas embarcados críticos e para medidores inteligentes, destacam a dificuldade de manter um ambiente de teste padronizado quando a validação depende de instrumentação montada manualmente para cada cenário. Cabral (2021) relata dificuldade semelhante para simular condições de falha durante o desenvolvimento de um sistema embarcado de detecção de anomalias.

2.4 TRABALHOS E FERRAMENTAS RELACIONADOS

2.4.1 Plataformas Comerciais de HIL

A dSPACE (dSPACE, [2026?]) e a National Instruments (National Instruments, [2026?]) oferecem plataformas de HIL que simulam em tempo real a planta completa à qual o controlador seria ligado, integradas a ferramentas de modelagem como o MATLAB/Simulink. Os fabricantes não divulgam preços. Os registros públicos de compra reunidos por Frassinelli (2026) vão de cerca de 24 mil dólares, para um sistema compacto da dSPACE adquirido pela NASA, a cerca de 194 mil dólares, para um sistema da National Instruments. Esses valores tornam as plataformas inviáveis para laboratórios acadêmicos e pequenas equipes, que são o público desta jiga.

2.4.2 HIL de Baixo Custo

A literatura registra várias tentativas de levar o HIL a plataformas baratas, quase sempre em domínios específicos. Lu *et al.* (2007) montaram um ambiente de HIL em tempo real para controles de eletrônica de potência com *software* livre e *hardware* de prateleira. Jamaludin *et al.* (2018) substituíram um HIL comercial por um equipamento baseado em microcontrolador para testar painéis de instrumentos automotivos, gerando sinais de frequência, de resistência e digitais. Tejado *et al.* (2016) ligaram um simulador de robô móvel em MATLAB/Simulink a uma placa Arduino para o ensino de controle. Qu e Lin (2012) construíram sobre um STM32 um simulador de sensores de aceleração para teste, e Jacko *et al.* (2022) descrevem laboratórios remotos de ensino com STM32 e ESP8266 que geram tensões e PWM.

Esses trabalhos mostram que um microcontrolador de médio custo consegue gerar sinais de teste com a temporização necessária, inclusive na família STM32. Cada um, porém, atende a um sensor ou a uma aplicação, e nenhum emula os barramentos digitais em que a maioria dos sensores atuais se comunica, nem automatiza a decisão de aprovar ou reprovar o *firmware*.

2.4.3 *Frameworks* e Equipamentos de Teste de *Hardware*

No *software* de automação, há *frameworks* abertos para testes com *hardware*. O OpenHTF, desenvolvido no Google, estrutura procedimentos de teste em

fases, com medições, limites e registro dos resultados (OpenHTF, [2026?]). O labgrid controla placas embarcadas, sua alimentação e suas portas seriais a partir de testes escritos em pytest (Labgrid, 2026), e o pytest-embedded, da Espressif, grava e se comunica com placas pela porta serial (Espressif Systems, [2026?]). O Pluma, da The Embedded Kit, automatiza testes de *hardware* e *software* em produtos com Linux embarcado (The Embedded Kit, 2026). The Embedded Kit (2024) discute alternativas ao Robot Framework para orquestrar esses testes. Na produção em série, jigas como a da Engenharia Híbrida ([2026?]) integram gravação, teste funcional e rastreabilidade. Essas ferramentas organizam e executam os testes, mas dependem de instrumentos que gerem os estímulos: não simulam, por si, sensores nem periféricos.

Do lado dos instrumentos, os equipamentos USB multifunção são o concorrente mais próximo em custo. O Analog Discovery 3, da Digilent, reúne osciloscópio de dois canais, gerador de funções, analisador lógico e fonte em um só aparelho, por cerca de 379 dólares (DigiKey, 2026a). Ele gera e mede sinais com resolução e banda superiores às da jiga, mas é um instrumento de uso geral. A sequência do teste é comandada pelo computador, por *scripts* escritos pelo usuário, e o aparelho não traz sensores comerciais emulados nem um formato de teste com veredito.

2.4.4 Quadro Comparativo

O Quadro 1 compara a jiga com os trabalhos e ferramentas discutidos, segundo as características que a definem.

Quadro 1 – Comparação entre a jiga e trabalhos e ferramentas relacionados.

Trabalho ou ferramenta	**Baixo custo**	***Black-box***	**Emula barramento**	**Emula sensor**	**Veredito automático**	**Protocolo aberto**
dSPACE e NI (dSPACE, [2026?]; National Instruments, [2026?])		•	•	•	•	
Lu *et al.* (2007)	•	•				•
Jamaludin *et al.* (2018)	•	•		•		
Tejado *et al.* (2016)	•					
Qu e Lin (2012)	•	•		•		
Jacko *et al.* (2022)	•					
OpenHTF, labgrid, pytest-embedded	•	•			•	•
Analog Discovery 3	•	•				
Jiga deste trabalho	•	•	•	•	•	•

Fonte: elaborado pelo autor.

As plataformas comerciais reúnem quase todas as características, mas não o custo. Os trabalhos de baixo custo emulam, no máximo, um sensor analógico de uma aplicação, e os *frameworks* automatizam o veredito sem gerar estímulos. A lacuna que a jiga ocupa é a combinação dessas características em um único equipamento

de baixo custo: estímulos analógicos e digitais, emulação de barramentos e de sensores comerciais, e baterias de teste com veredito automático, sobre um protocolo documentado.

3 METODOLOGIA

Este capítulo descreve como o trabalho foi conduzido e avaliado: a caracterização da pesquisa, os requisitos da jiga e a estratégia de verificação, com os critérios de aceitação, os casos de teste e os instrumentos. A jiga construída é apresentada no Capítulo 4.

3.1 CARACTERIZAÇÃO DA PESQUISA

Quanto à natureza, trata-se de uma pesquisa aplicada: busca resolver um problema prático, a validação de *firmware* sem sensores reais, por meio de um artefato tecnológico (Prodanov; Freitas, 2013). A abordagem é predominantemente quantitativa, pois a solução é avaliada por grandezas mensuráveis, como tensões, frequências, tempos e vereditos, comparadas a valores de referência.

Quanto aos procedimentos, a fundamentação (Capítulo 2) usa pesquisa bibliográfica, e a validação usa o procedimento experimental: definem-se o objeto de estudo, as variáveis que o influenciam e as formas de controlá-las e de observar os seus efeitos (Gil, 2017). As variáveis manipuladas são os estímulos aplicados pela jiga e, nos ensaios com falhas injetadas, o *firmware* do DUT; os efeitos observados são as respostas do DUT e os vereditos emitidos.

O objeto de estudo é a própria jiga. O DUT de referência é o ESP32-S3, escolhido por reunir todos os periféricos que a jiga estimula e emula: conversor analógico-digital, entradas e saídas digitais e os barramentos I2C, SPI e UART. Ele opera no mesmo nível lógico de 3,3 V da jiga. A jiga acessa o DUT apenas pelos pinos, sem depender do seu *firmware* ou da sua arquitetura (RNF02). Os dados são coletados pela própria jiga, nas respostas adquiridas e nos vereditos registrados pela aplicação, e conferidos com instrumentos independentes (subseção 3.3.5).

O trabalho foi conduzido em cinco etapas: (i) levantamento dos requisitos a partir do problema e dos objetivos (seção 3.2); (ii) projeto do *hardware* e escolha do microcontrolador; (iii) implementação do *firmware*, do protocolo e da aplicação *desktop* (Capítulo 4); (iv) verificação nos níveis de unidade, integração, sistema e aceitação, além de testes de desempenho (seção 3.3); e (v) análise dos resultados (Capítulo 5). A implementação foi incremental: cada versão só era fechada quando os casos de teste correspondentes passavam na placa.

3.2 REQUISITOS

Os requisitos foram derivados do problema (Capítulo 1) e dos objetivos específicos. Cada um recebe um identificador, usado na seção 3.3 para rastrear o teste

que o comprova. O Quadro 2 apresenta os requisitos funcionais, e o Quadro 3, os não funcionais.

Quadro 2 – Requisitos funcionais (RF).

ID	Requisito	Critério de aceitação
RF01	Gerar tensão analógica configurável em dois canais (DAC1 e DAC2), na faixa útil de 0,1 a 3,2 V.	Tensão medida coincide com a comandada, dentro do Quadro 5.
RF02	Gerar níveis lógicos em quatro saídas digitais (GPIO1–4).	Nível medido coincide com o comandado.
RF03	Gerar PWM em três canais, de 1 Hz a 100 kHz, com razão cíclica de 0 a 100% em passos de 0,01%.	Frequência e razão cíclica medidas dentro do Quadro 5.
RF04	Emular escravos I2C (100 e 400 kHz) e SPI (modos 0 a 3) e atuar como mestre nesses barramentos (SPI a partir de 211 kHz); comunicar-se por UART de 1 200 a 1 000 000 *baud*.	Dado escrito por uma extremidade é lido íntegro na outra.
RF05	Adquirir respostas do DUT: quatro entradas digitais (GPIO5–8), três canais analógicos (ADC) e dados pela UART.	Valor adquirido coincide com o aplicado ao pino.
RF06	Receber comandos do computador por USB-CDC e confirmar cada um com ACK ou NACK.	Todo comando válido recebe ACK e todo inválido recebe NACK.
RF07	Permitir, na aplicação, descrever baterias de casos de teste em arquivo.	Bateria salva é reexecutada sem alteração manual.
RF08	Executar baterias automaticamente e emitir o veredito (passou, falhou ou inconclusivo) de cada caso.	Veredito coincide com o esperado em casos de resposta conhecida, inclusive com *firmware* defeituoso.
RF09	Registrar os resultados de cada execução.	Relatório contém estímulos, respostas e vereditos.
RF10	Executar na própria jiga um caso de teste completo por comando, registrando o instante de cada medição com resolução de 1 µs.	Instantes, esperas e tempos de reação dentro do Quadro 5.
RF11	Emular sensores comerciais (MPU-6500 e BME280) no I2C e no SPI, de modo que o DUT os leia com o seu *driver* normal.	Valores lidos pelo DUT iguais aos simulados, dentro da tolerância do caso.
RF12	Variar no tempo os estímulos e as grandezas dos sensores emulados, com formas de onda definidas na bateria.	Valores nos instantes previstos, conferidos na jiga e no osciloscópio.

Fonte: elaborado pelo autor.

3.3 ESTRATÉGIA DE VERIFICAÇÃO

A verificação foi planejada nos quatro níveis da seção 2.1, de acordo com o que cada camada permite comprovar, e complementada por testes de desempenho.

3.3.1 Níveis de Teste

Testes de unidade. Verificam isoladamente a codificação e a decodificação do protocolo, a compilação dos casos em rotinas, o motor de testes da aplicação e os módulos do *firmware*. Os que não dependem do *hardware* rodam também no

Quadro 3 – Requisitos não funcionais (RNF).

ID	Requisito	Critério de aceitação
RNF01	Baixo custo, compatível com laboratórios acadêmicos e pequenas equipes.	Custo total da lista de materiais registrado e comparado às alternativas comerciais.
RNF02	Portabilidade: testar DUTs sem modificar o *firmware* deles.	A bateria de aceitação usa apenas os pinos do DUT, sem enviar comandos a ele, e o *firmware* sob teste não contém código para a jiga (inspeção).
RNF03	Confiabilidade da comunicação PC–jiga, sem perda silenciosa de comandos.	Comandos corrompidos ou perdidos são detectados e retransmitidos ou sinalizados.
RNF04	Reprodutibilidade: a mesma bateria deve produzir os mesmos estímulos.	Execuções repetidas produzem os mesmos estímulos e vereditos.
RNF05	Segurança elétrica do DUT e da jiga.	Níveis e correntes dentro dos limites das folhas de dados; riscos de ligação errada documentados (subseção 4.2.3).
RNF06	Modularidade: camadas independentes e protocolo documentado.	Protocolo especificado no Apêndice A e módulos testáveis isoladamente.
RNF07	Desempenho da comunicação: resposta rápida o bastante para não provocar retransmissões indevidas.	Latência máxima de ida e volta inferior a 50 ms.

Fonte: elaborado pelo autor.

computador a cada alteração, na abordagem *off-target* descrita por Grenning (2011) e Branam (2026).

Testes de integração. Verificam a comunicação entre a aplicação e o *firmware*, cada periférico da jiga e a execução de rotinas, inclusive as regras de ocupação, interrupção e tempo máximo. Para os periféricos, a jiga é ligada a si mesma em *loopback*, sem DUT, por *jumpers* entre as suas saídas e as suas entradas, o que mostra se o que ela gera é o que ela adquire.

Testes de sistema. Executam baterias completas com a jiga ligada a um DUT de referência, cujo *firmware* foi escrito para o teste e tem comportamento totalmente conhecido; assim, uma divergência pode ser atribuída à jiga, e não ao DUT. Esse nível cobre os barramentos com o DUT e a emulação dos sensores.

Testes de aceitação. Validam a solução no uso pretendido: uma bateria *black-box* verifica um comportamento conhecido do DUT apenas pelos seus pinos. Para mostrar que a jiga reprova um *firmware* errado no *hardware* real, a mesma bateria é executada também sobre versões do *firmware* do DUT com defeitos injetados de propósito.

O Quadro 4 relaciona os requisitos aos níveis de teste que os comprovam.

Quadro 4 – Rastreabilidade entre requisitos e níveis de verificação.

Requisito	Unidade	Integração	Sistema	Aceitação	Desempenho
RF01 a RF03		•			
RF04		•	•		
RF05		•			
RF06	•	•			
RF07	•				
RF08	•		•	•	
RF09			•		
RF10	•	•	•		
RF11	•	•	•		
RF12	•	•	•		
RNF01	análise da lista de materiais				
RNF02				•	
RNF03	•	•			•
RNF04			•		
RNF05		•	inspeção do projeto (subseção 4.2.3)		
RNF06	inspeção do projeto				
RNF07					•

Fonte: elaborado pelo autor.

3.3.2 Critérios de Aceitação

Os critérios numéricos das grandezas analógicas e temporais estão no Quadro 5. Para níveis lógicos e dados de barramento, o critério é a igualdade exata, pois qualquer diferença indica erro.

Quadro 5 – Critérios de aceitação das grandezas medidas.

Grandeza	Critério	Requisito
Tensão de saída do DAC	$\lvert V_{\text{medida}} - V_{\text{comandada}} \rvert \leq 33$ mV (1% do fundo de escala)	RF01
Leitura do ADC	$\lvert V_{\text{lida}} - V_{\text{aplicada}} \rvert \leq 33$ mV (1% do fundo de escala)	RF05
Frequência do PWM	desvio de até 1% do valor comandado	RF03
Razão cíclica do PWM	desvio de até 1 ponto percentual	RF03
Níveis digitais e dados de barramento	igualdade exata	RF02, RF04, RF05
Esperas, instantes e tempos de reação em uma rotina	$\lvert t_{\text{jiga}} - t_{\text{referência}} \rvert \leq 50$ µs	RF10
Latência de ida e volta de um comando	máxima inferior a 50 ms	RNF07

Fonte: elaborado pelo autor.

A tolerância analógica de 1% do fundo de escala é cerca de quarenta vezes a resolução dos conversores de 12 bits ($3{,}3\ \text{V}/4096 \approx 0{,}8$ mV). A margem absorve os erros de *offset* e de ganho dos conversores e a incerteza do instrumento de referência, e ainda é pequena diante das diferenças de tensão que um teste de emulação de sensor precisa distinguir. Quando um caso exige mais precisão, a tolerância de cada

verificação pode ser reduzida no arquivo da bateria. Esses critérios valem na faixa útil do DAC, de 0,1 a 3,2 V; em códigos do ADC, os 33 mV correspondem a ± 41 códigos.

No PWM, a frequência deriva do oscilador da placa e é quantizada pelos divisores do temporizador; o limite de 1% é folgado para os dois efeitos. A tolerância temporal de 50 µs é cinquenta vezes a resolução da base de tempo (1 µs), para acomodar a latência das interrupções e a execução de cada passo, e vinte vezes menor que a resolução de 1 ms do relógio do sistema operacional, que seria a precisão obtida sem a base de tempo dedicada. O limite de latência é a metade do tempo limite de resposta do protocolo, de 100 ms, para que atrasos normais nunca provoquem retransmissões.

Nos casos com sensores emulados, o valor esperado é o que a jiga aplica no instante em que o DUT lê o sensor. Quando o valor varia no tempo, como numa rampa, o esperado considera a latência entre o pedido da bateria e a leitura feita pelo *driver* do DUT. A tolerância cobre apenas a resolução do sensor e a variação dessa latência.

3.3.3 Casos de Teste

Foram definidos 27 casos de teste, descritos no Apêndice C com o procedimento, o resultado esperado e os requisitos que cada um comprova. Os casos CT01 a CT07 são de unidade e cobrem o protocolo, a compilação das baterias e o motor de testes da aplicação. Os casos CT08 a CT19 são de integração: comandos diretos, conversores, PWM, barramentos em *loopback* e rotinas, incluindo as regras de ocupação, interrupção e tempo máximo e a temporização das esperas. O CT27, também de integração, verifica o gerador de ondas. Os casos CT20, CT21, CT25 e CT26 são de sistema, com o DUT de referência e os sensores emulados; o CT22 é o teste de aceitação; e o CT23 e o CT24 são de desempenho. Nos casos de integração, as medições da própria jiga são conferidas com instrumentos independentes (subseção 3.3.5), pois não seria válido comprovar a jiga apenas com as suas próprias leituras.

O barramento I2C é verificado no nível de sistema (CT20), e não em *loopback*. O STM32F767ZI tem quatro controladores I2C (STMicroelectronics, [20–]), mas a jiga usa um só, o I2C2, como escravo ou mestre conforme o caso. Ligar um segundo controlador ao primeiro verificaria apenas a comunicação entre dois periféricos do mesmo *chip*, enquanto o DUT exercita o barramento com um controlador de outro fabricante, que é o uso pretendido. A escolha de um único I2C é uma decisão de projeto, que mantém livres os pinos dos demais controladores, e não uma limitação do microcontrolador.

No CT20, o DUT de referência atende comandos de texto pela sua UART, ligada à UART4 da jiga: o computador envia o comando com UART_WRITE e lê a

resposta com UART_READ, e o caso inteiro roda pela própria jiga. O mesmo *firmware* executa, numa tarefa própria e sem receber comandos, o comportamento do CT22: lê a tensão de um “sensor” a cada 1 ms, liga uma saída acima de 1,80 V e a desliga abaixo de 1,50 V. A bateria do CT22 depende apenas desse comportamento, observado nos pinos. Os pontos da histerese, 1,58 e 1,72 V, ficam a 70 a 80 mV dos limiares e do ponto médio. Assim, um limiar único em 1,65 V falharia nos dois casos, e o erro do ADC do DUT não altera o veredito.

Os defeitos injetados no CT22 são escolhidos na compilação do *firmware* do DUT e representam erros comuns de implementação: (1) histerese ausente, com limiar único em 1,65 V; (2) limiar de ligar deslocado para 2,10 V; (3) saída atrasada em 50 ms; e (4) saída presa em nível baixo. Cada versão é gravada no ESP32-S3 e testada com a mesma bateria, sem nenhuma alteração.

Os casos CT25 e CT26 usam o mesmo DUT como cliente dos sensores emulados. Ele lê o MPU-6500 e o BME280 com *drivers* escritos como os de um produto, com *reset*, identificação, configuração, leitura e a compensação da Bosch no BME280, e informa os valores pela UART. No CT27, os valores das ondas no sensor emulado são conferidos pela própria jiga, lendo o mapa da escrava em instantes conhecidos da rotina.

3.3.4 Testes de Desempenho

Os testes de desempenho (CT23 e CT24) avaliam a comunicação sob uso contínuo. A latência de ida e volta é medida no computador, com um relógio de alta resolução, porque é ela que determina a duração de uma bateria e o risco de retransmissões indevidas. Além da média, registram-se o percentil 95 e o máximo, pois o escalonamento do sistema operacional do computador introduz atrasos ocasionais que a média esconderia. O CT24 verifica se, numa sequência longa de comandos, alguma falha escapa à detecção e à retransmissão (RNF03). Esses atrasos não afetam a temporização dentro de um caso, que é executado na jiga e verificado pelo CT19.

3.3.5 Instrumentos de Medição

As medições independentes foram feitas com um multímetro digital, para as tensões contínuas do DAC, e com um osciloscópio digital, para o PWM, as formas de onda e os tempos. O Quadro 6 identifica os instrumentos e a incerteza de cada medição, segundo os manuais dos fabricantes.

Quadro 6 – Instrumentos de medição.

Instrumento	**Modelo**	**Uso**	**Resolução e exatidão**
Multímetro digital	Fluke 117	Tensão do DAC (CT10)	Faixa de 6 V: resolução de 1 mV, exatidão de $\pm$(0,5% da leitura + 2 dígitos)
Osciloscópio digital	Agilent DSO-X 2014A (100 MHz, 4 canais), ponta N2862B (10:1)	PWM, ondas e tempos (CT12, CT19, CT22, CT27)	Base de tempo: $\pm$25 ppm; ganho vertical DC: $\pm$3% do fundo de escala; deslocamento: $\pm$(0,1 div + 2 mV + 1% do ajuste)

Fonte: elaborado pelo autor.

4 DESENVOLVIMENTO DA JIGA

Este capítulo apresenta a jiga construída para atender aos requisitos da seção 3.2: a arquitetura geral, o *hardware*, o *firmware*, o protocolo de comunicação e a aplicação *desktop*. Cada escolha de projeto é acompanhada do motivo e da alternativa descartada.

4.1 VISÃO GERAL DA SOLUÇÃO

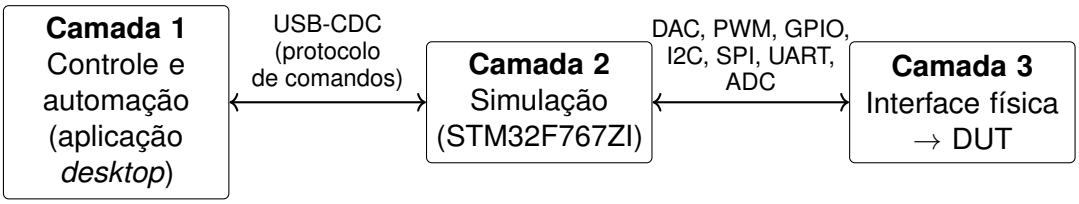
A solução proposta é uma jiga de testes composta por três camadas, conforme a Figura 1:

a) **camada de controle e automação**: uma aplicação *desktop*, executada em um computador, que define as rotinas de teste, envia comandos à jiga, recebe as respostas e emite o veredito de cada caso (passou/falhou);

b) **camada de simulação**: o *firmware* do microcontrolador STM32F767ZI, que executa os comandos e as rotinas de teste recebidos, gera os estímulos elétricos (analógicos, digitais, PWM e de barramento) e adquire as respostas do dispositivo sob teste (DUT);

c) **camada de interface física**: os conectores e *jumpers* que ligam os pinos da jiga aos pinos do DUT.

Figura 1 – Arquitetura em três camadas da jiga de testes.



Fonte: elaborado pelo autor.

A separação em camadas foi adotada por duas razões. A primeira é a portabilidade: como a jiga se relaciona com o DUT apenas por sinais elétricos, sem instrumentá-lo, ela mantém a abordagem *black-box* definida na seção 2.1 e pode ser aplicada a microcontroladores de fabricantes e arquiteturas distintas (Euripedes Filho, 2017). A segunda é a testabilidade: com um protocolo de comandos bem definido entre as camadas 1 e 2, cada uma pode ser verificada isoladamente, o que corresponde à prática de *design for test* recomendada por Tamari (2023).

4.2 PROJETO DE HARDWARE

4.2.1 Escolha do Microcontrolador

O núcleo da camada de simulação é o microcontrolador STM32F767ZI, da família STM32F7 (STMicroelectronics, [2026?]), utilizado em uma placa de desenvolvimento com o formato Nucleo-144. A escolha se justifica por três motivos. Primeiro, a família STM32 já foi empregada com sucesso para gerar sinais de simulação de sensores (Qu; Lin, 2012) e de estímulo em laboratórios remotos (Jacko *et al.*, 2022), o que reduz o risco técnico do projeto. Segundo, o dispositivo reúne em um só *chip* todos os periféricos exigidos pelos requisitos: dois canais de DAC, três conversores analógico-digitais, temporizadores com saída PWM, I2C, SPI, UART e USB, dispensando componentes externos e mantendo o baixo custo (RNF01). Terceiro, a placa de desenvolvimento já integra o programador/depurador e expõe os pinos em conectores padronizados, o que dispensa o desenvolvimento de uma placa dedicada, conforme o escopo do trabalho (Capítulo 1).

4.2.2 Mapeamento de Pinos

Os sinais foram distribuídos pelos conectores CN7 a CN10 da placa. Para cada função, escolheu-se um pino ligado ao periférico necessário e, sempre que possível, vizinho dos demais sinais da mesma função; as entradas digitais ficaram em pinos tolerantes a 5 V (subseção 4.2.3). A Figura 2 mostra os sinais e o sentido de cada um, e o Apêndice B traz a posição de cada pino nos conectores.

4.2.3 Compatibilidade de Níveis de Tensão

Os pinos do STM32F767ZI operam com níveis lógicos de 3,3 V. O DUT de referência, o ESP32-S3, opera no mesmo nível, de modo que a ligação é direta. Ainda assim, como a jiga pode ser ligada a DUTs de outras famílias, a compatibilidade de níveis é o principal cuidado para atender ao RNF05, e as diretrizes adotadas são:

- as saídas da jiga (GPIO1–4, PWM, DAC e SPI/I2C quando geradas por ela) operam em 3,3 V, nível compatível com o ESP32 e normalmente reconhecido como nível alto por entradas de 5 V, embora com pouca margem de ruído;
- as entradas digitais GPIO5–8 foram alocadas em pinos tolerantes a 5 V (FT), para que saídas de DUTs de 5 V possam ser adquiridas sem conversor de nível;
- as entradas analógicas (ADC) e as saídas do DAC operam na faixa de 0 a 3,3 V; sinais analógicos de 5 V não podem ser ligados diretamente às entradas do ADC e exigem divisor de tensão;

- nenhuma saída de 5 V de um DUT deve ser conectada a um pino da jiga que não seja tolerante a 5 V, nem a um DUT de 3,3 V, como o ESP32, que não tolera 5 V em seus pinos;
- em barramentos I2C entre dispositivos de tensões distintas, é necessário conversor de nível ou *pull-ups* referenciados a 3,3 V.

A tolerância dos pinos FT tem duas condições, ambas da folha de dados do microcontrolador (STMicroelectronics, [20–]). Com a jiga alimentada (VDD = 3,3 V), a tensão máxima em um pino FT é o menor valor entre VDD + 3,6 V e 5,5 V, isto é, 5,5 V. Com a jiga desligada, o limite cai para 3,6 V. Por isso, a ordem de energização é uma restrição de uso: a jiga deve ser ligada antes do DUT de 5 V e desligada depois dele. Isso se cumpre naturalmente quando os dois são alimentados pelo mesmo computador e a jiga é conectada primeiro. A segunda condição é que, acima de 4 V, os resistores internos de *pull-up* e *pull-down* do pino precisam estar desligados. As entradas GPIO5–8 são, portanto, configuradas sem resistor interno; quando o DUT exigir um resistor, como em uma saída de dreno aberto, ele é externo e escolhido conforme a tensão do DUT. Sem resistor e sem ligação, uma entrada solta lê um valor indefinido, o que é esperado e não indica defeito. A ligação a um DUT de 5 V não foi validada em bancada.

Além dos níveis, há o risco de ligação errada. Cada pino do STM32F767ZI fornece ou absorve no máximo 25 mA (STMicroelectronics, [20–]). Se uma saída da jiga for ligada a uma saída do DUT em nível oposto, ou se a jiga e o DUT forem ambos mestres no SPI, as duas saídas entram em curto, e a corrente pode passar desse limite. A jiga não tem proteção contra esse erro, pois as saídas vão direto aos pinos da placa. A medida recomendada no guia de ligação é um resistor de 220 a 330 Ω em série com cada saída. Ele limita a corrente de um curto entre 3,3 V e 0 V a cerca de 15 mA sem afetar sinais de até alguns megahertz. Na montagem validada, as ligações foram conferidas antes de energizar o DUT, e a inclusão desses resistores numa placa de interface fica como trabalho futuro.

4.2.4 Conversores Analógicos

O DAC opera com o *buffer* de saída ligado. Com o *buffer*, a saída tem baixa impedância, mas não alcança os trilhos de alimentação: perto de 0 e de 3,3 V ela satura. Desligar o *buffer* aproximaria a saída dos trilhos, mas deixaria o pino com impedância de saída da ordem de quilo-ohms. A carga de uma entrada do DUT derrubaria a tensão justamente no meio da faixa, onde ela é mais usada. Manteve-se o *buffer* e definiu-se uma **faixa útil** de 0,1 a 3,2 V (códigos 124 a 3971), na qual vale o critério de 1% do fundo de escala (subseção 3.3.2). Fora dela, a saturação é uma limitação conhecida e documentada. O estado seguro usa o código 0, a menor tensão

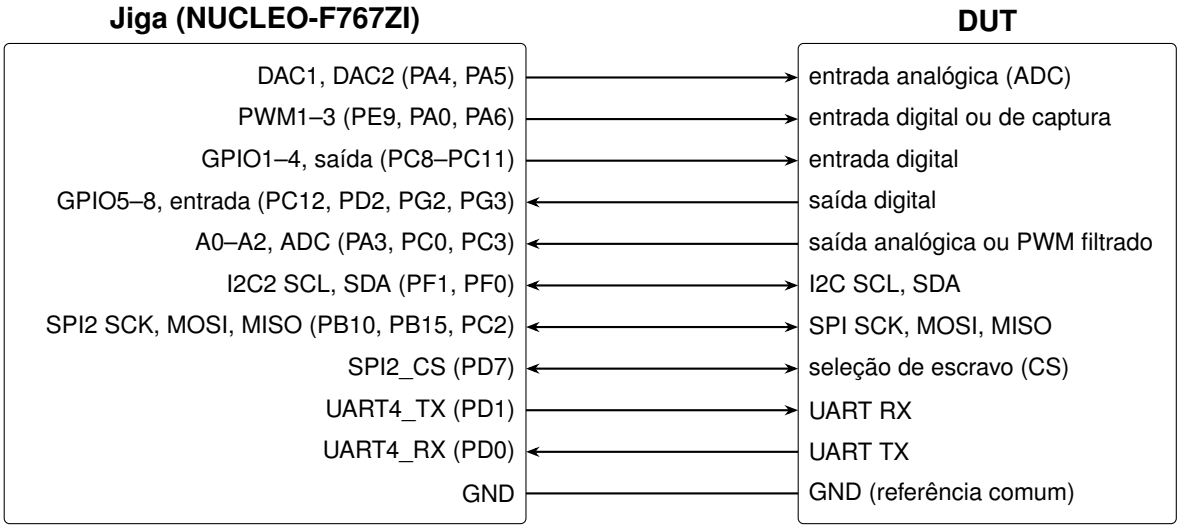
que o DAC fornece, valor fixo e inofensivo para o DUT, em vez de desligar o DAC, o que deixaria o pino flutuando.

Cada canal de ADC usa um conversor próprio (ADC1, ADC2 e ADC3), com resolução de 12 bits, referência de 3,3 V e tempo de amostragem de 480 ciclos, o maior disponível: com ele, o capacitor de amostragem tem tempo de carregar mesmo quando a fonte do sinal, no DUT, tem impedância de alguns quilo-ohms. Cada ADC_READ faz uma única conversão, por varredura (*polling*), pois o tempo de conversão, de poucos microssegundos, é desprezível diante da comunicação.

4.2.5 Esquema de Ligação com o DUT

O esquema de ligação da Figura 2 define, para cada grupo de sinais, o pino da jiga, a função correspondente no DUT e o sentido do sinal. Apenas o lado da jiga é fixo. Os pinos do lado do DUT dependem do *firmware* avaliado e são indicados pelo usuário na bateria de testes.

Figura 2 – Esquema de ligação entre a jiga e o DUT.



Fonte: elaborado pelo autor.

Algumas regras de ligação decorrem do esquema. O GND da jiga e o do DUT devem ser interligados, pois todas as tensões aplicadas e medidas são referenciadas a ele; sem essa ligação, os níveis lidos por ambos os lados deixam de ter significado. Na UART, as linhas são cruzadas: a transmissão de um lado vai à recepção do outro. No barramento I2C, as linhas SCL e SDA exigem resistores de *pull-up*, referenciados a 3,3 V pelas razões da subseção 4.2.3. No SPI, o sentido de SCK, MOSI e CS depende do papel da jiga. Como escrava, o DUT gera esses sinais e o pino PD7 é configurado como entrada; como mestra, a jiga os gera e PD7 é configurado como saída.

Os únicos componentes externos da montagem são os resistores de *pull-up* do I2C.

4.2.6 Lista de Materiais e Custo

A Tabela 1 apresenta os materiais que compõem a jiga e seu custo, que é o critério de aceitação do RNF01. A lista inclui apenas o que é necessário para montar e operar a jiga. O DUT usado na validação (ESP32-S3), os instrumentos de medição, o computador e os cabos USB não fazem parte dela, pois não integram o produto e são recursos que o laboratório já precisa ter para desenvolver *firmware*. O preço da placa é o de um distribuidor oficial, em dólares e sem os impostos de importação, convertido pela taxa PTAX de venda do dia da consulta (DigiKey, 2026b; Banco Central do Brasil, 2026). O preço dos *jumpers* é o valor pago na compra.

Tabela 1 – Lista de materiais da jiga de testes.

Item	**Qtd.**	**Preço unit. (R$)**	**Subtotal (R$)**
Placa de desenvolvimento NUCLEO-F767ZI, com depurador ST-LINK integrado (US$ 34,39 a R$ 5,4621)	1	187,84	187,84
Kit de cabos *jumper* macho-fêmea, 20 cm (40 unidades)	1	9,00	9,00
Resistor 4,7 kΩ, 1/4 W (*pull-up* do I2C)	2	–	–
Total			**196,84**

Fonte: elaborado pelo autor, com o preço da placa de DigiKey (2026b), consultado em 5 nov. 2026.

A composição da lista decorre das decisões anteriores. A placa de desenvolvimento concentra o microcontrolador, o depurador e os conectores (seção 4.2), e a jiga é usada em bancada. Os *jumpers* são do tipo macho-fêmea porque os conectores da placa usados na ligação são fêmeas e o ESP32-S3, em placa de desenvolvimento, expõe pinos machos. Os dois resistores de *pull-up* do I2C e o resistor de 220 Ω em série no CS do SPI, único sinal em que a jiga e o DUT podem ser saídas ao mesmo tempo, só são necessários quando esses barramentos são usados. Como têm custo desprezível diante do total, não foram cotados. Um DUT de 5 V exigiria ainda os divisores de tensão nas entradas analógicas e um conversor de nível nos barramentos (subseção 4.2.3), componentes de baixo custo que não fazem parte da montagem validada.

O Capítulo 5 compara esse custo com o das alternativas comerciais discutidas na seção 2.4.

4.3 PROJETO DE FIRMWARE

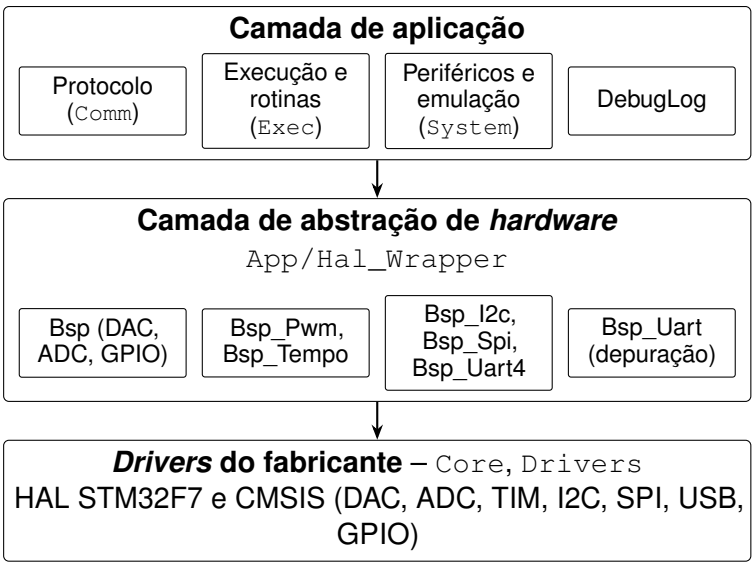
O *firmware* é responsável por interpretar os comandos recebidos do computador e traduzi-los em estímulos elétricos, além de adquirir e devolver as respostas do DUT. Esta seção descreve sua arquitetura em camadas, a divisão de tarefas no

sistema operacional de tempo real e a lógica de processamento de um comando, com as respectivas justificativas.

4.3.1 Arquitetura em Camadas

O código é organizado em três camadas, ilustradas na Figura 3. Na base fica a camada de *drivers* do fabricante (`Core` e `Drivers`, gerados pelo STM32CubeMX, com a HAL e o CMSIS). Acima dela, a camada de abstração de *hardware* (`App/Hal_Wrapper`) tem um módulo `Bsp` por periférico. No topo, a camada de aplicação se apoia no FreeRTOS para organizar a sua execução em tarefas. A camada de aplicação tem quatro grupos. O protocolo (`App/Comm`) recebe os quadros e monta as respostas. A execução (`App/Exec`) trata os comandos diretos, interpreta as rotinas e aplica o estado seguro. Os módulos de periférico (`App/System`) validam os parâmetros e contêm a lógica do DAC, das entradas e saídas digitais, do PWM e dos barramentos, inclusive a emulação dos escravos. Por fim, o `DebugLog` registra as mensagens de depuração.

Figura 3 – Arquitetura em camadas do firmware.



Fonte: elaborado pelo autor.

A justificativa para essa separação é a mesma que fundamenta as abordagens de teste *off-target* discutidas na seção 2.3 (Lins, 2016; Branam, 2026): isolando o acesso direto aos periféricos em um módulo único (`Bsp`), a camada de aplicação passa a depender de uma interface própria, e não diretamente da HAL do fabricante. Isso traz dois benefícios práticos para este trabalho. Primeiro, a portabilidade: se a jiga for futuramente portada para outro microcontrolador, apenas a camada `Bsp` precisa ser reescrita, mantendo o protocolo e a lógica de dispatch inalterados. Segundo, a testabilidade: os testes de unidade (seção 3.3) substituem o `Bsp` por periféricos simulados, e

os módulos de protocolo, de execução e de periférico rodam também no computador, compilados com o `gcc`, sem depender do *hardware* da jiga.

4.3.2 Tarefas e Sincronização

Os sinais estáticos, como uma tensão de DAC ou um PWM configurado, são mantidos pelos próprios periféricos. O que exige concorrência é o restante: a comunicação espera bytes da USB, um teste pode aguardar a reação do DUT e, como escrava I2C, SPI ou UART, a jiga atende no instante que o DUT escolhe. Num laço principal único (*superloop*), essas esperas virariam verificações periódicas, e o atraso de uma atividade afetaria as demais. Com um sistema operacional de tempo real, cada atividade é uma tarefa que bloqueia à espera de um evento sem ocupar o processador. O FreeRTOS foi escolhido por já estar integrado ao STM32CubeMX e ser de código aberto, sob licença MIT.

O *firmware* é organizado nas três tarefas do Quadro 7, todas criadas na inicialização e permanentes. A `CommTask` é dedicada exclusivamente à comunicação com o computador, e a `TestExecTask` interpreta os comandos de teste e aciona os estímulos e as aquisições. A terceira, `defaultTask`, é gerada pelo STM32CubeMX: inicializa a pilha USB e, depois, apenas pisca o LED de atividade da placa.

Quadro 7 – Tarefas do FreeRTOS.

Tarefa	**Responsabilidade**	**Ciclo de vida**
`CommTask`	Comunicação com o computador pela USB OTG FS: recebe e valida os quadros (subseção 4.3.4), atende diretamente PING, ROUTINE_STATUS e ABORT, repassa os demais comandos à `TestExecTask` e envia as respostas.	Criada na inicialização; permanente.
`TestExecTask`	Executa os comandos diretos e interpreta as rotinas de teste (subseção 4.3.5): aplica os estímulos (DAC, saídas digitais, PWM e barramentos), realiza as aquisições (ADC, entradas digitais e barramentos) e aplica o estado seguro.	Criada na inicialização; permanente.
`defaultTask`	Gerada pelo STM32CubeMX: inicializa o USB Device e pisca o LED de atividade.	Criada na inicialização; permanente.

Fonte: elaborado pelo autor.

A separação entre `CommTask` e `TestExecTask` garante que uma não atrase a outra. Enquanto a `TestExecTask` executa uma rotina, a `CommTask` continua recebendo e validando quadros: responde às consultas de estado lendo as informações que a `TestExecTask` publica a cada passo, transmite um ABORT como sinal à `TestExecTask` e recusa os demais, conforme as regras da subseção 4.3.4. No sentido inverso, a chegada de bytes pela USB não interrompe um estímulo em andamento. As duas tarefas trocam comandos e resultados por filas do FreeRTOS;

como o protocolo só admite um comando pendente por vez (seção 4.4), cada fila precisa comportar apenas um elemento. Os bytes recebidos pela USB passam da interrupção à `CommTask` por um *stream buffer* do FreeRTOS.

As prioridades seguem a urgência de cada atividade. As transações iniciadas pelo DUT, que não espera, são atendidas nas interrupções dos periféricos, com precedência sobre qualquer tarefa. Entre as tarefas, a `TestExecTask` vem primeiro, para que a comunicação não atrase os passos de uma rotina. A `CommTask` fica abaixo das duas, pois só precisa responder dentro do tempo limite de 100 ms do protocolo, folga muito maior que qualquer passo isolado. Como a `TestExecTask` passa a maior parte do tempo bloqueada à espera de eventos, a `CommTask` continua sendo atendida com frequência.

No projeto, previu-se uma tarefa para cada emulação de escravo, mas elas foram dispensadas na implementação. O escravo I2C é atendido byte a byte na interrupção do I2C2. O SPI é atendido na interrupção do SPI2 e na do pino de seleção (PD7), que marca o início e o fim de cada transação; e a UART, na interrupção da UART4. Como é o DUT quem dita o tempo de cada byte e o tratamento é curto, uma tarefa intermediária só acrescentaria uma troca de contexto e uma pilha própria. O critério foi transformar em tarefa apenas o que precisa bloquear à espera de eventos.

4.3.3 Processamento de um Comando

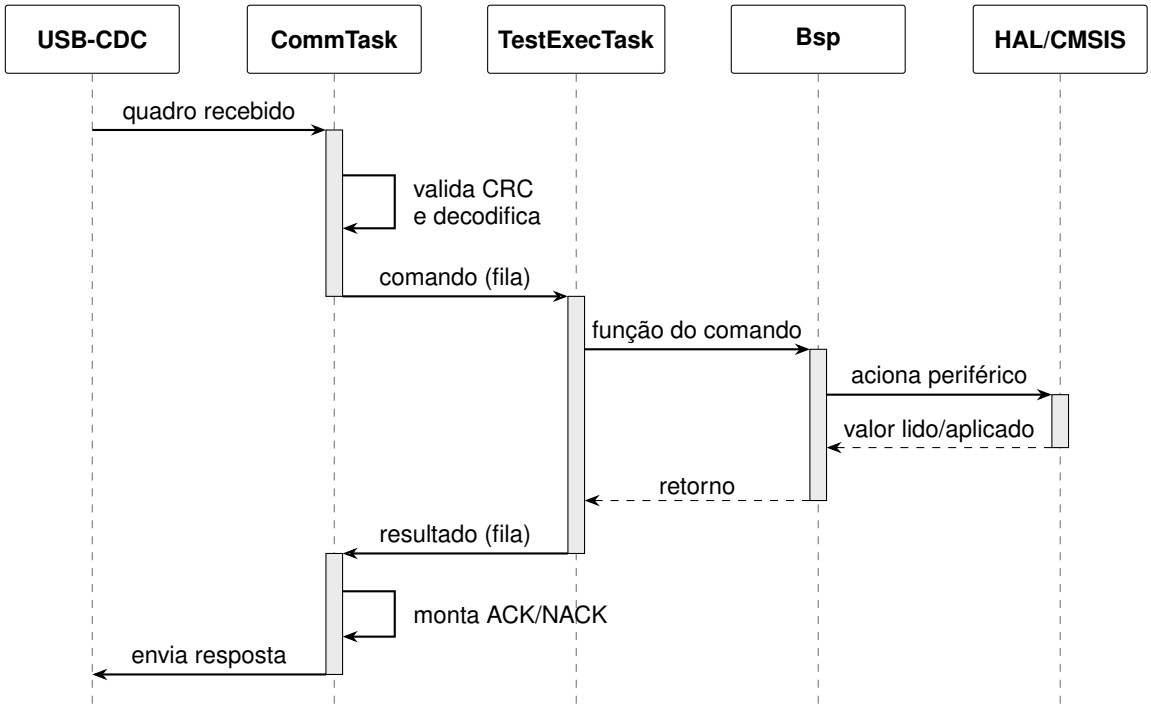
A Figura 4 apresenta o percurso de um comando direto, da recepção do quadro à resposta. A `CommTask` valida o quadro e o repassa pela fila à `TestExecTask`, que executa o comando por meio do `Bsp` e devolve o resultado pela fila de retorno. Enquanto isso, a `CommTask` continua lendo a USB, com tempo limite de 20 ms, e confere a fila de resultado a cada volta. Como a `TestExecTask` tem prioridade maior, um comando rápido já está concluído na conferência seguinte; só um comando demorado, como um UART_WRITE longo, pode esperar até 20 ms a mais, pouco diante do tempo limite de 100 ms. Num RUN_ROUTINE, o resultado imediato é apenas a aceitação da rotina (subseção 4.3.5).

4.3.4 Máquina de Estados do Receptor de Quadros

Como o USB-CDC entrega um fluxo contínuo de bytes (seção 4.4), a `CommTask` usa um receptor de quadros implementado como uma máquina de estados finita, em vez de assumir que cada leitura da porta corresponde a um quadro completo. O receptor é um módulo à parte, sem dependência do *hardware* nem do FreeRTOS: recebe um byte por vez e devolve um quadro completo e válido, ou o motivo do descarte. A Figura 5 apresenta os estados e as transições.

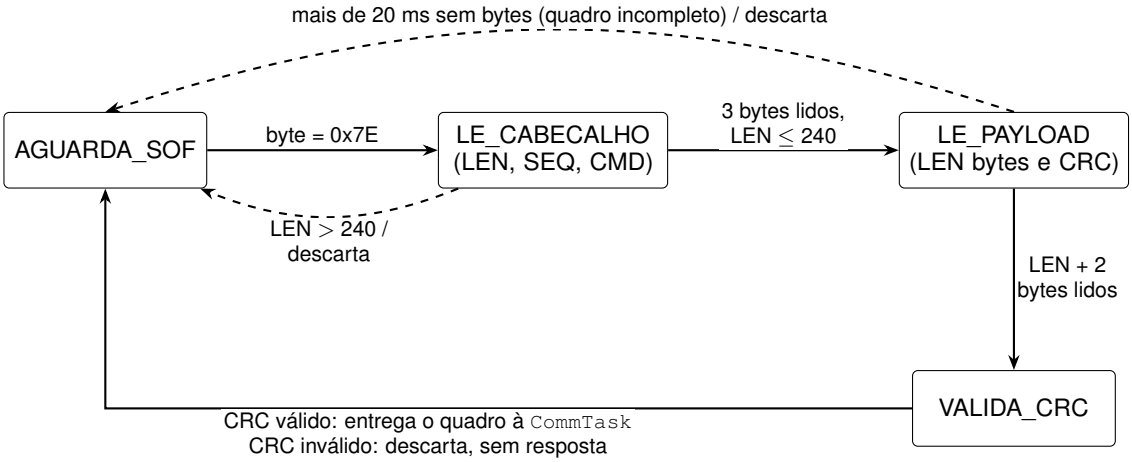
A máquina de estados foi adotada, em vez de uma função bloqueante que aguarda o quadro inteiro, porque a recepção de um byte por vez, acionada pela

Figura 4 – Sequência de processamento de um comando direto no firmware.



Fonte: elaborado pelo autor.

Figura 5 – Máquina de estados do receptor de quadros.



Fonte: elaborado pelo autor.

interrupção ou pela fila da USB-CDC, não pode travar a tarefa à espera de bytes que ainda não chegaram; a máquina de estados registra o progresso da leitura e retoma de onde parou a cada novo byte recebido. O estado `VALIDA_CRC` realiza a verificação descrita na seção A.3 antes de qualquer efeito sobre os periféricos, de modo que um quadro corrompido nunca chega à `TestExecTask` nem ao `Bsp`, o que atende ao RNF03.

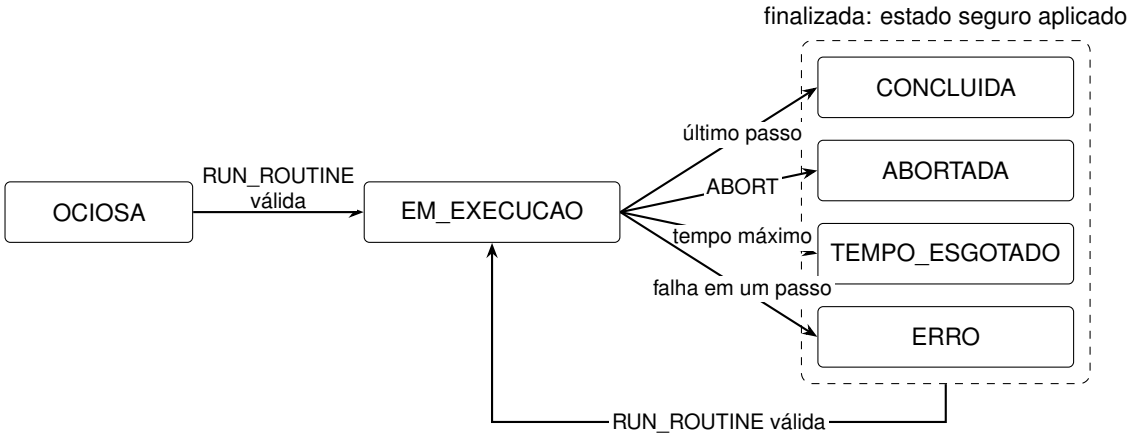
Cada quadro válido segue para a lógica de resposta da `CommTask`, separada do receptor. A retransmissão do comando que ainda está em execução é ignorada, e

um quadro com o SEQ do último comando respondido recebe a resposta guardada, sem nova execução (Apêndice A.7). PING, ROUTINE_STATUS e ABORT são respondidos na hora, sem passar pela `TestExecTask`, o que garante a resposta mesmo com uma rotina em execução. Qualquer outro comando que chegue com um comando pendente ou durante uma rotina recebe OCUPADO, e os demais são repassados à `TestExecTask`; a resposta é enviada quando o resultado volta pela fila.

4.3.5 Interpretador de Rotinas

Ao receber um RUN_ROUTINE, a `TestExecTask` primeiro valida a rotina inteira, conforme a especificação do Apêndice A; só depois de validada a rotina é aceita e executada, passo a passo, na ordem em que foi escrita. O estado da execução evolui conforme a Figura 6.

Figura 6 – Estados de execução de uma rotina na `TestExecTask`.



Fonte: elaborado pelo autor.

A precisão temporal, que motivou a execução das rotinas na jiga (subseção 4.4.3), depende de três decisões. A primeira é a base de tempo: instantes e esperas são contados pelo TIM5, de 32 bits, como contador livre a 1 MHz, e não pelo relógio do sistema operacional, de 1 ms. O TIM5 é, com o TIM2, que já gera o PWM2, o único temporizador de 32 bits do microcontrolador, e a 1 MHz só se repete após cerca de 71 minutos. A segunda é a detecção de transições no ESPERA_GPIO: a borda é detectada por interrupção externa, que registra o valor do temporizador naquele momento, independentemente do escalonamento da `TestExecTask`. Como PD2 (GPIO6) e PG2 (GPIO7) compartilham a linha EXTI2, a linha é associada à entrada monitorada no início de cada passo. A terceira é a forma de esperar. A tarefa bloqueia à espera de uma notificação, disparada por uma comparação do TIM5, pela entrada monitorada ou por um ABORT, de modo que a espera não ocupa o processador e pode ser interrompida imediatamente.

A cada passo concluído, a `TestExecTask` atualiza o estado publicado para a `CommTask` (estado, número de passos concluídos e código de erro) e verifica se o tempo máximo da rotina foi ultrapassado. Ao final, qualquer que seja o motivo, leva as saídas ao estado seguro e só então publica o estado final. Assim, o computador, ao ver a rotina finalizada, sabe que nenhum estímulo continua aplicado. Os resultados permanecem disponíveis para leitura. Os passos são executados pelas mesmas funções dos comandos diretos, o que garante que um estímulo aplicado em uma rotina se comporta exatamente como o comando direto equivalente, já verificado nos testes de integração.

4.3.6 Emulação de Sensores

Além de emular um dispositivo genérico (um mapa de registradores ou um *buffer*), a jiga emula dois sensores comerciais completos, o acelerômetro e giroscópio MPU-6500 e o sensor ambiental BME280 (temperatura, pressão e umidade), de modo que o DUT os lê com o seu *driver* normal, sem saber que está falando com a jiga. A emulação aproveita o mapa de 256 registradores da escrava I2C e acrescenta três recursos ao *firmware*. O primeiro é uma **máscara de bits graváveis** por registrador. Um registrador só de leitura, como o de identificação ou os de dados, não muda quando o DUT escreve nele. Um bit de *reset* que, no sensor real, volta sozinho a zero é marcado como não gravável: o DUT escreve 1 e lê 0, como se o *reset* tivesse terminado. Sem isso, um *driver* que espera o fim do *reset* ficaria preso. O segundo é um **segundo endereço** de escrava, recurso do próprio periférico I2C do STM32F767ZI, que permite à jiga responder como os dois sensores ao mesmo tempo. Os dois endereços dividem o mesmo mapa, o que é possível porque os registradores dos dois sensores não se sobrepõem (de `0x0D` a `0x75` no MPU-6500 e de `0x88` a `0xFE` no BME280). O terceiro é o **modo registradores da escrava SPI**, que usa o mesmo mapa: em cada transação, o primeiro byte é o endereço, com o bit 7 indicando leitura, e os seguintes são os dados, com auto-incremento.

O modo SPI tem uma restrição que decorre do próprio barramento. No I2C, a escrava pode segurar a linha de *clock* (*clock stretching*) até ter o dado pronto, e o mestre espera. No SPI, o mestre gera o *clock* sem esperar a escrava, e o primeiro dado precisa começar a sair logo após o último bit do endereço, que é justamente quando a jiga descobre qual registrador foi pedido. Por isso, o mestre precisa deixar uma pausa de alguns microssegundos entre o byte de endereço e o primeiro dado. Daí em diante, a interrupção do SPI mantém a FIFO de transmissão de 4 bytes cheia com os registradores seguintes, e a leitura segue na velocidade do mestre. Um mestre que emenda o endereço e os dados sem pausa, como os que usam DMA, recebe o primeiro dado errado. A restrição foi aceita e documentada, porque cobri-la exigiria um circuito externo dedicado; como a jiga tem um único pino de seleção, apenas um

sensor por vez pode ficar no SPI, enquanto o outro pode ficar no I2C.

O *firmware* não conhece os sensores: ele só tem o mapa, as máscaras e os endereços, e os modelos ficam na aplicação. No MPU-6500, as grandezas são inteiros de 16 bits proporcionais ao fundo de escala. No BME280, o DUT lê valores brutos e os compensa com os coeficientes de calibração gravados no mapa; a aplicação faz a operação inversa, por busca binária, com erro de até 0,006 °C, 0,003 hPa e 0,006 ponto percentual. A configuração inicial de um sensor passa de 100 bytes e não cabe com os passos nos 240 bytes de uma rotina. Por isso, ela é enviada como **preparo** do caso, em comandos diretos antes do RUN_ROUTINE, e a rotina contém apenas as mudanças de valor.

4.3.7 Gerador de Ondas

Muitos comportamentos de interesse dependem de como um sinal varia no tempo. São exemplos um termostato que liga ao passar de um limiar durante um aquecimento, um filtro que deve ignorar uma vibração, um controle que reage a uma rampa. Para isso, a jiga tem um gerador de ondas que varia um valor sozinho, em segundo plano, enquanto a rotina continua; assim, as esperas e as verificações do caso acontecem com a onda em andamento. Os destinos possíveis são os dois canais do DAC, a razão cíclica dos três canais de PWM e os registradores dos sensores emulados, e até seis ondas podem rodar ao mesmo tempo.

O *firmware* conhece apenas uma tabela de 2 a 32 pontos igualmente espaçados no período, com interpolação linear ou em degraus, um número de ciclos (zero significa até o fim da rotina) e um ruído uniforme opcional; a forma da onda é montada pela aplicação. Com isso, qualquer forma custa o mesmo para o *firmware*, e a aplicação oferece rampa, senoide, triangular, quadrada, dente de serra, exponencial (a resposta de um sistema de primeira ordem, como um aquecimento real) e uma sequência livre de pontos. Uma senoide com 32 pontos e interpolação linear se afasta menos de 0,5% da amplitude da senoide verdadeira, e as formas triangular, dente de serra e rampa são exatas.

Com passos de 100 µs ou mais, cada onda é atualizada numa interrupção agendada no canal 2 de comparação do TIM5. O passo padrão de 1 ms basta para os sensores, pois o BME280 entrega no máximo cerca de 180 medidas por segundo e os *drivers* costumam ler o MPU-6500 a até 1 kHz. Para o DAC, porém, 1 ms limitaria uma senoide a dezenas de hertz. Com passos de 1 a 99 µs, um período de até 1024 amostras é tocado por DMA, disparado pelo TIM7 (DAC1) ou TIM4 (DAC2), sem roubar tempo das interrupções que atendem o DUT. No PWM, a onda altera apenas o registrador de comparação, sem reiniciar a contagem. No fim da rotina, no ABORT e no RESET_ALL, as ondas são paradas antes do estado seguro.

Uma onda na temperatura do BME280 tem um cuidado adicional: como a

compensação da pressão e da umidade depende da temperatura, mudar só o valor bruto da temperatura faria o DUT ler também uma pressão e uma umidade diferentes. A aplicação acrescenta, por isso, duas ondas que corrigem os valores brutos da pressão e da umidade ao longo da rampa, calculadas pelo mesmo modelo inverso.

4.4 PROTOCOLO DE COMUNICAÇÃO PC–JIGA

A comunicação entre a aplicação *desktop* e o *firmware* é feita por um protocolo próprio de comandos, transportado sobre USB-CDC, que o computador enxerga como uma porta serial virtual. A especificação completa, com formato do quadro, códigos de comando, erros e exemplos, está no Apêndice A. Esta seção resume as escolhas e as justifica.

4.4.1 Meio de Transmissão

A placa oferece três caminhos até o computador: a UART do depurador integrado (USART3), a porta Ethernet e o USB OTG *full-speed* do STM32F767ZI na classe CDC (*Communications Device Class*). Escolheu-se o USB-CDC porque o computador o reconhece como porta serial virtual com os *drivers* nativos, e a aplicação o acessa com a mesma biblioteca de uma UART. A vazão também não fica presa a uma taxa de *baud*, pois os dados trafegam em pacotes USB de 12 Mbit/s. Em relação à Ethernet, ele dispensa endereçamento IP e uma pilha TCP/IP no *firmware*, sem benefício para uma ligação ponto a ponto. A UART do depurador fica reservada ao registro de depuração (`DebugLog`), separado do canal de comandos.

4.4.2 Modelo de Comunicação

O protocolo segue o modelo requisição–resposta: o computador inicia toda a comunicação, e cada comando recebe exatamente uma resposta, ACK ou NACK (RF06). O computador só envia um novo comando depois da resposta do anterior, o que dispensa fila de comandos e controle de fluxo no *firmware*. A vazão menor não limita, pois o tempo de um teste é dominado pela resposta do DUT.

Há dois tipos de comando. Os **comandos diretos** executam uma única operação – aplicar uma tensão no DAC, ler uma entrada digital – e servem à operação manual da jiga e aos testes de integração. Os **comandos de rotina** transferem à jiga um caso de teste completo, isto é, uma sequência de estímulos, esperas e aquisições, que a jiga executa sozinha (subseção 4.4.3).

4.4.3 Rotinas Executadas na Jiga

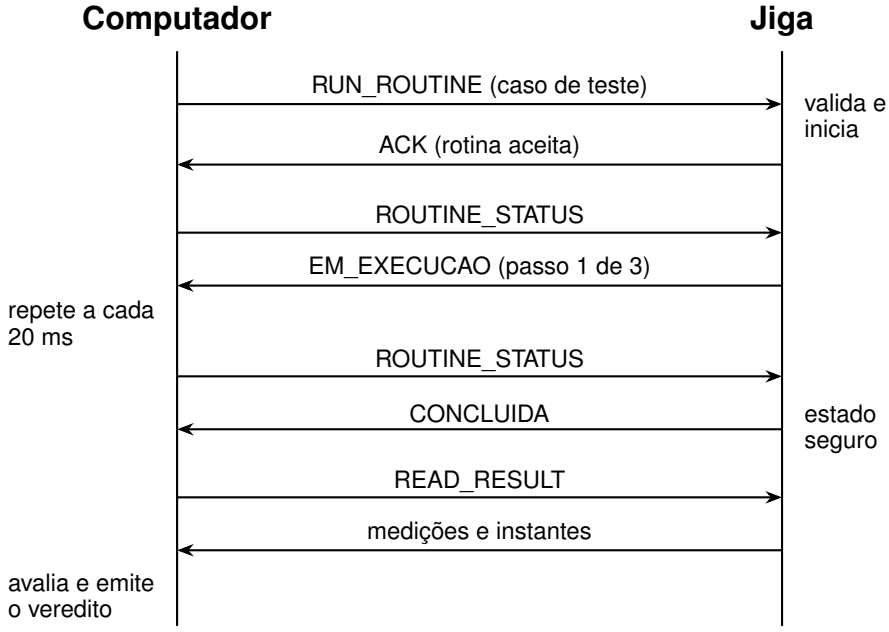
Cada caso de teste é convertido pela aplicação em uma rotina e enviado em um único comando, RUN_ROUTINE. A execução na própria jiga, em vez de

o computador comandar cada estímulo e cada leitura, foi adotada principalmente pela precisão temporal. Comandados pelo computador, os passos sofreriam a ida e volta pela USB e o escalonamento do sistema operacional, que variam na escala de milissegundos. Na jiga, eles são temporizados pelo relógio do microcontrolador, e cada medição é registrada com resolução de microssegundos (subseção 4.3.5), o que permite verificar tempos de reação do DUT e aumenta a reprodutibilidade (RNF04). A rotina também é mais robusta: se a comunicação cair no meio de um caso, a jiga a conclui e volta sozinha ao estado seguro.

Usa-se um caso por comando, e não a bateria inteira, porque cada caso tem o seu próprio veredito e começa do estado seguro, uma falha não compromete os casos seguintes e a aplicação pode mostrar o progresso. Uma bateria inteira também não caberia em um quadro.

Como uma rotina pode durar vários segundos, mais do que o tempo limite de resposta do protocolo, o ACK de RUN_ROUTINE é enviado assim que a rotina é validada e começa a ser executada, e não ao seu término. Enquanto a rotina é executada, o computador consulta periodicamente o estado com ROUTINE_STATUS e, ao final, lê as medições com READ_RESULT, conforme a Figura 7.

Figura 7 – Ciclo de execução de uma rotina entre o computador e a jiga.



Fonte: elaborado pelo autor.

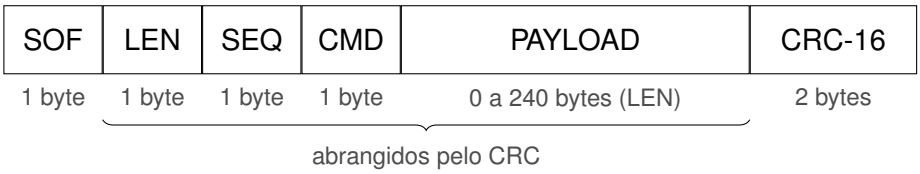
O acompanhamento por consultas mantém o computador como único a iniciar a comunicação, de modo que o mesmo tempo limite e a mesma retransmissão valem para todas as mensagens. O tráfego, de poucos bytes a cada 20 ms, é desprezível para o USB. Durante a execução, os comandos que poderiam interferir na rotina são recusados (subseção 4.3.4).

O comando ABORT interrompe a rotina a qualquer momento, e cada rotina informa um tempo máximo, fiscalizado pela jiga. Nos dois casos, e ao término normal, a jiga volta ao estado seguro (RNF05). A jiga devolve apenas as medições, e o veredito é emitido pelo computador; assim, o interpretador permanece simples, os critérios ficam em unidades físicas no arquivo da bateria (subseção 4.5.3) e o relatório preserva os dados brutos (RF09).

4.4.4 Formato do Quadro

Como o USB-CDC entrega um fluxo contínuo de bytes, sem delimitar mensagens, cada mensagem é encapsulada em um quadro composto por um byte de início (SOF), o tamanho dos dados (LEN), um número de sequência (SEQ), o código do comando (CMD), os dados (PAYLOAD) e uma verificação de integridade (CRC-16), conforme a Figura 8.

Figura 8 – Formato do quadro do protocolo PC–Jiga.



Fonte: elaborado pelo autor.

O formato é **binário**, e não texto, porque o cabeçalho de tamanho fixo é decodificado pelo *firmware* sem análise de *strings*, e o lado Python o monta com a biblioteca padrão.

O enquadramento é feito pelo comprimento: o byte `0x7E` só marca o início quando o receptor procura o SOF e, dentro do quadro, é um byte comum delimitado pelo LEN. A inserção de bytes de escape (*byte stuffing*) foi descartada porque faria o tamanho do quadro depender do conteúdo, podendo quase dobrá-lo. A perda de alinhamento, risco do enquadramento por comprimento, é coberta pelo CRC, pelo tempo limite de 20 ms entre bytes, que devolve o receptor à busca do SOF, e pela retransmissão do computador.

4.4.5 Detecção de Erros e Retransmissão

O USB já detecta erros em cada pacote, mas o fluxo de bytes pode perder o alinhamento das mensagens, por exemplo no reinício de um dos lados. Um comando montado com bytes desalinhados aplicaria um estímulo errado sem aviso (RNF03). Por isso o quadro inclui um CRC-16/CCITT, de baixo custo e adequado ao quadro máximo de 246 bytes.

Sem resposta válida, o computador retransmite o comando com o mesmo SEQ. A jiga guarda a última resposta e, ao receber um SEQ repetido, a reenvia sem executar o comando de novo; assim, uma escrita I2C não é aplicada duas vezes, e um RUN_ROUTINE retransmitido não reinicia a rotina.

4.4.6 Comandos e Papéis nos Barramentos

Os comandos diretos estão organizados por periférico: sistema, DAC, GPIO, PWM, ADC, I2C, SPI e UART (RF01 a RF05). Os passos de uma rotina usam a mesma codificação, acrescida de uma espera fixa e de uma espera por nível numa entrada digital, que registra o instante da transição. Como a principal aplicação é fornecer ao DUT os dados de um sensor, nos barramentos I2C e SPI a jiga é **escrava** por padrão. Ela mantém um mapa de registradores (I2C) ou *buffers* (SPI) que emulam o sensor, e o DUT a consulta como faria com o dispositivo real. O papel de mestre é oferecido para quando o DUT é quem responde. O **estado seguro** (DAC no código 0, saídas em nível baixo, PWM parado e barramentos desligados) é aplicado na inicialização, no RESET_ALL e ao fim de cada rotina (RNF05), conforme a seção A.8.

4.5 APLICAÇÃO DESKTOP

4.5.1 Escolha da Linguagem

A aplicação *desktop* concentra a descrição dos testes, a comunicação com a jiga e o veredito. Como a comunicação é por uma porta serial virtual, o *firmware* independe da linguagem do computador. Foram consideradas Python, C# e C++.

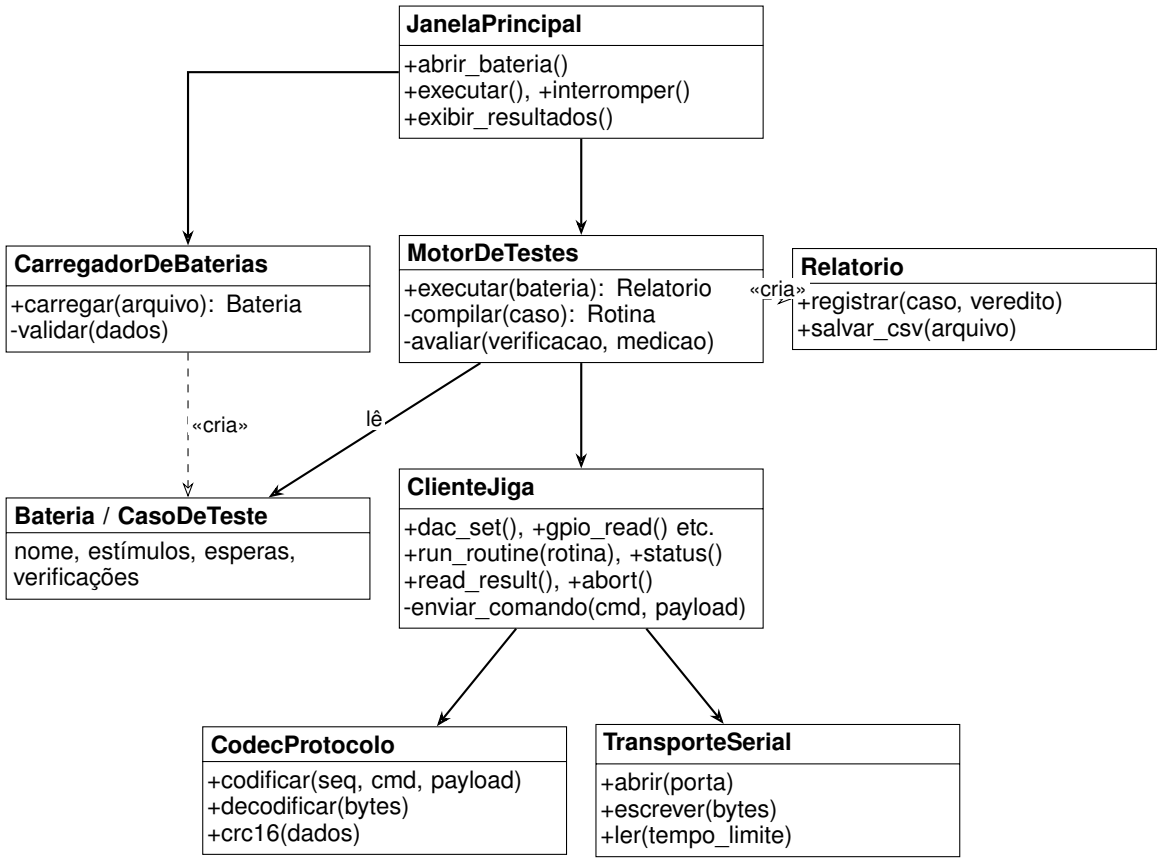
Escolheu-se o **Python** porque: (i) a comunicação serial é resolvida por bibliotecas maduras e multiplataforma; (ii) a linguagem facilita a interpretação dos arquivos de bateria e os testes automatizados da própria aplicação, com o *pytest*, sem *frameworks* externos como o Robot Framework (The Embedded Kit, 2024); (iii) há bibliotecas prontas para gráficos e relatórios; e (iv) o desempenho do computador não limita, pois o tempo de um teste é dominado pela USB e pelo DUT. C# simplificaria a interface no Windows, mas WinForms e WPF só rodam nesse sistema. C++ não traria ganho perceptível e exigiria mais bibliotecas e mais esforço de desenvolvimento.

4.5.2 Arquitetura da Aplicação

A aplicação segue a mesma ideia de camadas do *firmware* (Figura 9). `TransporteSerial` troca bytes com a jiga pela biblioteca *pyserial*; `CodecProtocolo` monta e decodifica os quadros (Apêndice A); e `ClienteJiga` oferece uma operação por comando e concentra o número de sequência, o tempo limite e as retransmissões. No topo, `CarregadorDeBaterias` lê e valida os arquivos de teste, e

`MotorDeTestes` converte cada caso numa rotina, acompanha a execução e emite os vereditos. `Relatorio` registra os resultados e `JanelaPrincipal` forma a interface gráfica.

Figura 9 – Diagrama de classes da aplicação *desktop*.



Fonte: elaborado pelo autor.

Essa divisão foi adotada por três motivos. O primeiro é a testabilidade. Os testes de unidade trocam o `TransporteSerial` por um transporte simulado, ligado a uma jiga simulada que espelha o *firmware* e que pode descartar respostas de propósito, de modo que o cliente, o motor e a janela são testados sem a placa. O segundo é a separação entre interface e lógica: a mesma bateria roda pela janela ou por linha de comando, o que permite testes de regressão automáticos. O terceiro é o isolamento do protocolo, que muda apenas no `CodecProtocolo`. A interface usa a biblioteca PySide6 (Qt), multiplataforma e de licença LGPL, e executa o motor numa *thread* própria, para que o botão de interromper tenha efeito imediato. O cliente usa tempo limite de 100 ms e três tentativas por comando.

4.5.3 Descrição dos Testes

Os testes são descritos em um arquivo de texto no formato YAML, chamado aqui de **bateria de testes**, composto por **casos de teste**. Cada caso define os

estímulos a aplicar, as esperas e as verificações a fazer sobre as respostas do DUT, e dá origem a uma rotina executada pela jiga (subseção 4.4.3). A Código 1 ilustra uma bateria para um *firmware* hipotético que deve ligar uma saída quando a tensão de um sensor, simulada pelo DAC, ultrapassa um limiar. O primeiro caso verifica também o tempo de reação: a saída do DUT deve ir a nível alto em até 100 ms após o estímulo.

Código 1 – Exemplo de bateria de testes.

```
bateria: controle-ventilador
dut: ESP32
casos:
  - nome: liga acima do limiar em até 100 ms
    estimulos:
      - dac: {canal: 1, tensao: 2.0}
    verificar:
      - reacao_gpio: {pino: 5, nivel: 1, max_ms: 100}
  - nome: desliga abaixo do limiar
    estimulos:
      - dac: {canal: 1, tensao: 1.0}
    espera_ms: 500
    verificar:
      - gpio_in: {pino: 5, esperado: 0}
      - adc: {canal: 0, esperado: 1.65, tolerancia: 0.05}
```

Fonte: elaborado pelo autor.

A descrição declarativa, em vez de testes escritos como código, foi escolhida para que um usuário sem conhecimento de Python consiga criar e ler uma bateria, e para que as baterias possam ser versionadas junto com o *firmware* do DUT, apoiando os testes de regressão a cada alteração (RF07). O YAML foi preferido ao JSON por admitir comentários e ser mais legível para arquivos editados à mão.

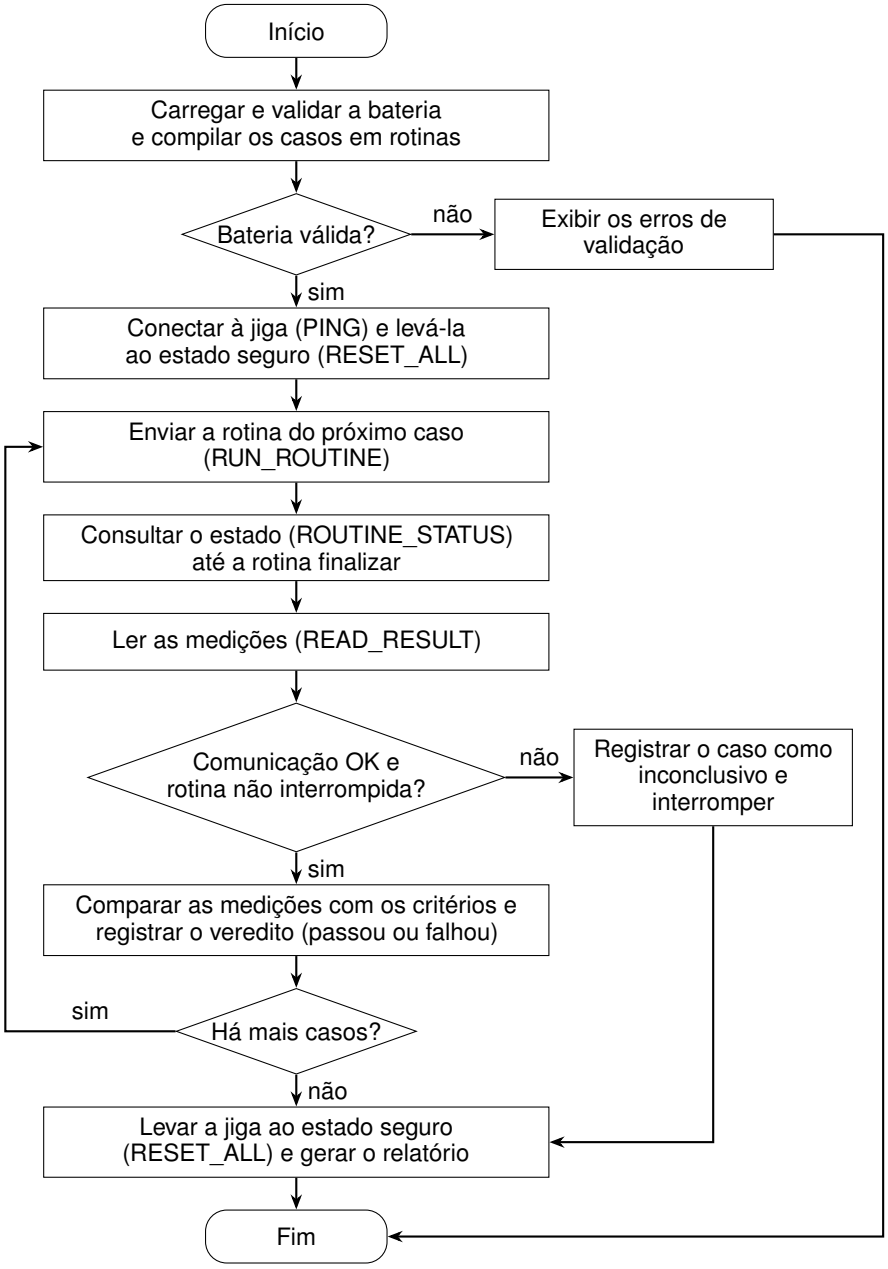
Antes da execução, a aplicação compila cada caso numa rotina binária. Cada estímulo vira o passo correspondente, com as grandezas físicas convertidas para os valores do protocolo; por exemplo, uma tensão V no DAC corresponde ao código $\mathrm{round}(4095 \cdot V/3{,}3)$. Cada espera vira um passo ESPERA, cada verificação vira a aquisição correspondente, e a de tempo de reação vira um ESPERA_GPIO. Os valores esperados e as tolerâncias ficam na aplicação, que avalia as medições devolvidas. O tempo máximo da rotina é a soma das esperas, dos tempos limite e da transmissão pela UART, multiplicada por 1,1 e acrescida de 100 ms, e pode ser fixado no arquivo. O Quadro 24, no Apêndice D, resume os itens de uma bateria.

Como cada rotina termina no estado seguro, cada caso configura os periféricos que usa, sem depender do caso anterior. A bateria inteira é validada antes da execução, e todos os erros encontrados são informados de uma vez, cada um com o caso e o item correspondentes. Entre as validações estão as mesmas que a jiga aplica à rotina, de modo que uma rotina recusada pela jiga indicaria um defeito da própria aplicação, e não da bateria.

4.5.4 Execução de uma Bateria

A Figura 10 apresenta a lógica com que o `MotorDeTestes` executa uma bateria.

Figura 10 – Fluxograma de execução de uma bateria de testes.



Fonte: elaborado pelo autor.

Três decisões dessa lógica merecem justificativa. Primeiro, todos os casos são validados antes da primeira comunicação, para que um erro no último caso não apareça só depois de metade da bateria executada. Segundo, a jiga vai ao estado seguro no início e no fim da bateria, além de ao fim de cada rotina, para que nenhum estímulo anterior influencie a execução atual (RNF04 e RNF05). Terceiro, existe o veredito “inconclusivo”, dado quando a comunicação falha mesmo após as retransmissões

ou quando a rotina é interrompida, por ABORT ou tempo máximo. Nesses casos, não se sabe se todos os estímulos foram aplicados, e culpar o DUT seria injusto; por isso a bateria é interrompida (RNF03). Já uma rotina em ERRO indica que o DUT não respondeu num barramento, e o caso é registrado como falho. Interromper a bateria pela interface envia ABORT, e o caso em andamento fica inconclusivo.

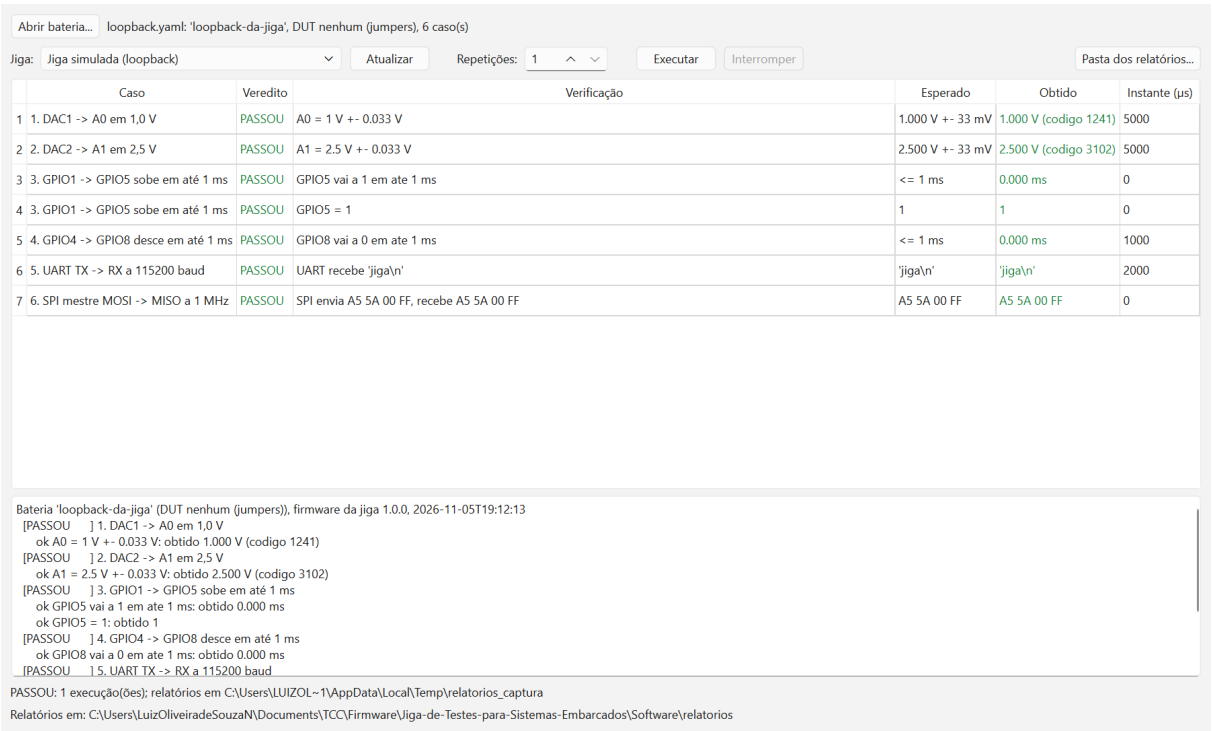
5 RESULTADOS E DISCUSSÕES

Este capítulo apresenta a jiga construída e os resultados da verificação planejada na seção 3.3. Os resultados são os da versão 1.0.0 do *firmware* e da aplicação, cujo código está publicado em repositório aberto (Neto, 2026). As medições de bancada foram feitas com a jiga, o DUT e os instrumentos da subseção 3.3.5 sobre uma mesma bancada.

5.1 SOLUÇÃO IMPLEMENTADA

O *firmware* implementa todos os comandos do Apêndice A: os 19 comandos diretos de sistema, DAC, GPIO, PWM, ADC, I2C, SPI e UART, os quatro comandos de rotina e os passos de espera. A aplicação implementa as classes da Figura 9 e grava, a cada execução, um relatório em CSV e em JSON; o Apêndice D traz a bateria do CT22 e um trecho do relatório de uma execução. Uma mesma bateria pode ser executada pela janela da Figura 11 ou pela linha de comando. Para a validação, foi escrito ainda o *firmware* do DUT de referência, para o ESP32-S3.

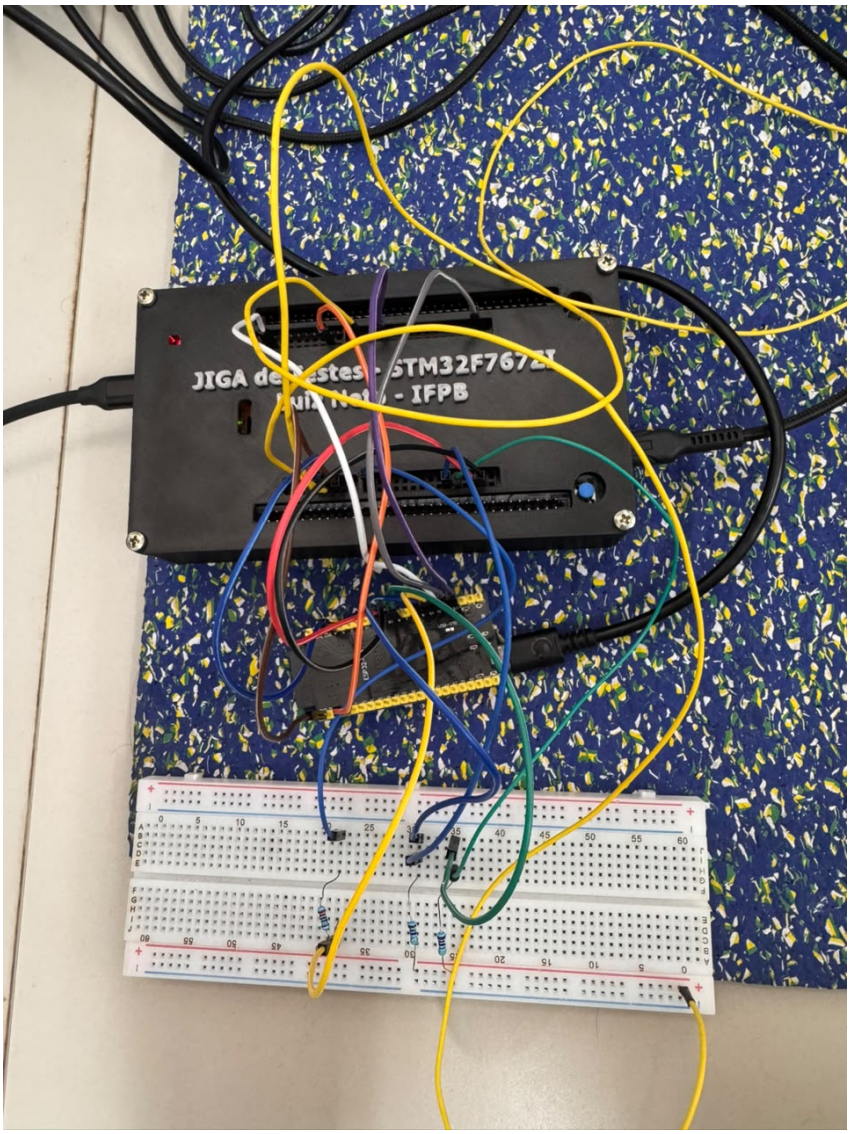
Figura 11 – Janela da aplicação após a execução da bateria de *loopback*.



Fonte: elaborado pelo autor. Execução contra a jiga simulada; na placa, a janela é a mesma.

A Figura 12 mostra a bancada usada nos ensaios de sistema, com a jiga ligada ao ESP32-S3 por *jumpers*.

Figura 12 – Bancada de ensaios: jiga (NUCLEO-F767ZI), DUT (ESP32-S3), *jumpers* e matriz de contatos com os resistores.



Fonte: elaborado pelo autor.

5.2 TESTES AUTOMATIZADOS E COBERTURA

A Tabela 2 resume os conjuntos de testes automatizados. Os testes de unidade do *firmware* usam o Unity e rodam na própria placa; os módulos que não dependem do *hardware* (execução, rotinas, ondas e periféricos, com periféricos simulados) rodam também no computador, assim como os do *firmware* do DUT. Os testes da aplicação usam o *pytest*. Os de integração são executados por um *script* que envia os quadros à placa e confere as respostas e o registro de depuração da jiga. Os testes que rodam no computador são repetidos por integração contínua a cada alteração enviada ao repositório. As medições com instrumentos e os ensaios com o DUT, descritos nas seções seguintes, são feitos à parte, em bancada.

Tabela 2 – Testes automatizados e resultados na versão 1.0.0.

Conjunto	**Onde roda**	**Passaram**
Unidade do *firmware* (Unity)	placa	211 de 211
Unidade do *firmware*: execução, rotinas e ondas	computador	89 de 89
Unidade do *firmware* do DUT, com as falhas injetadas	computador	25 de 25
Aplicação e casos de teste (*pytest*)	computador	625 de 625
Integração com a placa, sem ligações	placa e computador	73 de 73
Integração com os *jumpers* de *loopback*	placa e computador	74 de 74
Integração com o DUT (CT20)	placa, computador e ESP32-S3	5 de 5

Fonte: elaborado pelo autor.

O número de testes não indica, sozinho, o quanto do código eles exercitam. A Tabela 3 mostra a cobertura de linhas e de ramos medida no computador. Na aplicação, usou-se o `coverage.py`, excluídos os *scripts* de bancada e o terminal serial; no *firmware*, o `gcov` nos módulos do *firmware* que rodam no computador. Os módulos que dependem do *hardware* (protocolo na USB, `Bsp` e emulação nas interrupções) só rodam na placa, onde a cobertura não foi medida. Eles são exercitados pelos testes de integração e de sistema.

Tabela 3 – Cobertura dos testes executados no computador.

Código	**Linhas**	**Ramos**
Aplicação (protocolo, baterias, cliente, motor, ondas, sensores, relatório e janela)	92,6%	86,2%
Firmware: execução, rotinas, ondas e módulos de periférico	85,9%	69,4%

Fonte: elaborado pelo autor.

5.3 TESTES DE UNIDADE

Os casos CT01 a CT07 passaram. O CRC-16 da sequência `123456789` resultou em `0x29B1` no periférico de CRC do microcontrolador e na implementação em Python (CT01). Os 17 quadros dos Quadros 17 e 18 foram montados byte a byte e decodificados nos mesmos campos (CT02). A alteração de cada um dos 64 bits de um quadro de DAC_SET, fora do SOF, fez o quadro ser descartado sem resposta, e o cliente recuperou o comando por retransmissão (CT03). Bytes espúrios antes do SOF foram ignorados, e um quadro truncado foi descartado pelo tempo limite entre bytes, com ressincronização no quadro seguinte (CT04). Com a primeira resposta descartada, o cliente retransmitiu o comando com o mesmo SEQ, executado uma única vez; com três respostas perdidas seguidas, sinalizou a falha de comunicação (CT05). A bateria do Código 1 foi compilada nos passos esperados, e um caso com 80 estímulos foi recusado antes de qualquer comunicação (CT06). Por fim, o motor de testes emitiu os vereditos previstos para rotinas concluídas, em erro, com tempo esgotado, interrompidas e com perda de comunicação (CT07).

Esses testes usam uma jiga simulada que espelha a lógica do *firmware*, conferida pelos mesmos vetores de teste. Ela permitiu desenvolver a aplicação sem a placa e define, para cada caso de integração, a resposta esperada antes do envio à placa.

5.4 TESTES DE INTEGRAÇÃO COM COMANDOS DIRETOS

O PING foi respondido com as versões do protocolo e do *firmware* (CT08), e os comandos inexistentes e com parâmetros fora da faixa receberam os NACKs previstos, sem alterar nenhuma saída (CT09). Os 73 casos de integração sem ligações cobrem cada comando direto, com parâmetros válidos e inválidos, o receptor diante de quadros corrompidos, truncados e com bytes espúrios, a retransmissão e a resposta OCUPADO.

5.4.1 DAC e ADC

A Tabela 4 apresenta a tensão do DAC1 (PA4) medida com um multímetro Fluke 117, sem carga, nos 11 pontos do CT10. Todos os pontos ficaram dentro do critério de 33 mV: o maior erro, em módulo, foi de 2,2 mV, cerca de 0,07% do fundo de escala, e os erros são menores que a própria incerteza do multímetro, de 2 a 18 mV ao longo da faixa (Quadro 6). Mesmo somando essa incerteza ao erro medido, cada ponto fica a menos de 21 mV do ideal. O DAC2 (PA5) foi conferido pelo *loopback* descrito a seguir e, com o multímetro, no ponto médio, com erro de -2 mV. Nas pontas da escala, fora da faixa útil, a saída satura a cerca de 50 mV de cada trilho nos dois canais, efeito do *buffer* de saída que motivou a faixa útil (subseção 4.2.4): os códigos 0 e 1 produziram 0,052 V, e o código 4095, 3,254 V, 46 mV abaixo do ideal.

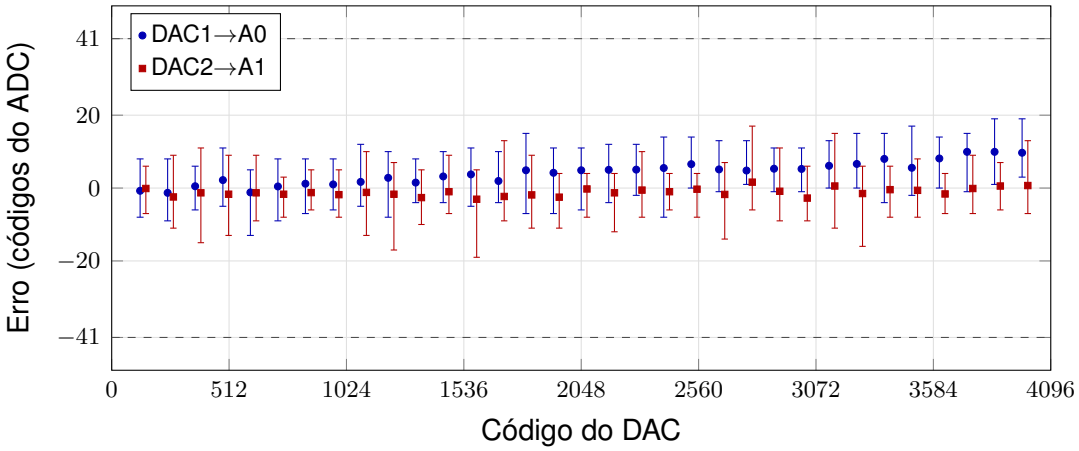
Tabela 4 – Tensão de saída do DAC1 (PA4) medida com multímetro, sem carga, nos 11 pontos do CT10.

Código	Ideal (V)	Medida (V)	Erro (mV)	Incerteza do multímetro (mV)
124	0,0999	0,098	$-1{,}9$	± 2
509	0,4102	0,408	$-2{,}2$	± 4
893	0,7196	0,719	$-0{,}6$	± 6
1278	1,0299	1,029	$-0{,}9$	± 7
1663	1,3401	1,340	$-0{,}1$	± 9
2048	1,6504	1,651	+0,6	± 10
2432	1,9599	1,961	+1,1	± 12
2817	2,2701	2,270	$-0{,}1$	± 13
3202	2,5804	2,581	+0,6	± 15
3586	2,8898	2,892	+2,2	± 16
3971	3,2001	3,202	+1,9	± 18

Fonte: elaborado pelo autor. Incerteza de $\pm$(0,5% da leitura + 2 dígitos) na faixa de 6 V, segundo o fabricante.

No *loopback* DAC→ADC (CT10), cada canal do DAC foi ligado a uma entrada analógica e varrido em 33 pontos da faixa útil, com 16 leituras do ADC por ponto. A Figura 13 mostra o erro de cada ponto, e a Tabela 5 resume os dois canais. Nenhuma das 1 056 leituras ficou fora da tolerância de ± 41 códigos (33 mV); o pior caso isolado foi de 19 códigos (15,3 mV). O erro sistemático é pequeno: ganho de +0,25% no canal 1, que chega a cerca de +10 códigos no topo da faixa, e de +0,04% no canal 2. Parte da diferença entre os canais vem do DAC: no código 2048, o multímetro mediu de +0,7 a +3,7 códigos no canal 1, em duas medições, e $-2{,}5$ no canal 2, e o *loopback*, +4,9 e $-0{,}2$. O restante do ganho do canal 1 vem do ADC, pois o multímetro não mediu mais que 2,2 mV de erro no DAC1 ao longo da faixa (Tabela 4). O que define a pior leitura isolada é o ruído, com 15 a 30 códigos entre o mínimo e o máximo das 16 leituras de um ponto. O critério de 1% é atendido com margem de cerca de duas vezes. O CT11, com as 16 combinações de saídas e entradas digitais, também passou.

Figura 13 – Erro da leitura do ADC no *loopback* DAC→ADC (média e faixa de 16 leituras por ponto).



Fonte: elaborado pelo autor. As linhas tracejadas marcam a tolerância de ± 41 códigos (33 mV); os pontos do canal 2 foram deslocados 25 códigos para a direita, para não se sobreporem.

Tabela 5 – Resumo do *loopback* DAC→ADC na faixa útil (33 pontos, 16 leituras por ponto).

Canal	Reta ajustada (leitura)	Erro médio por ponto	Pior leitura	Fora de ± 41
DAC1→A0	$1{,}00253 \cdot \mathrm{DAC} - 1{,}00$	$-1{,}3$ a +9,9 códigos	+19 códigos (+15,3 mV)	0 de 528
DAC2→A1	$1{,}00042 \cdot \mathrm{DAC} - 1{,}99$	$-3{,}1$ a +1,6 códigos	-19 códigos ($-15{,}3$ mV)	0 de 528

Fonte: elaborado pelo autor.

5.4.2 PWM

A Tabela 6 apresenta as medições do CT12 com osciloscópio, em seis combinações que cobrem os três canais, os extremos da faixa de frequência e razões cíclicas de 12,34% a 90%. Todas ficaram dentro do critério. A coluna “esperado” mostra o valor que os registradores do temporizador efetivamente produzem: em 20 kHz no TIM2, 12,34% resulta em 12,33%, pois o período tem 5 400 contagens; em 50 Hz no TIM3, de 16 bits, o divisor necessário dá 49,9997 Hz. Esses arredondamentos são duas ordens de grandeza menores que a tolerância.

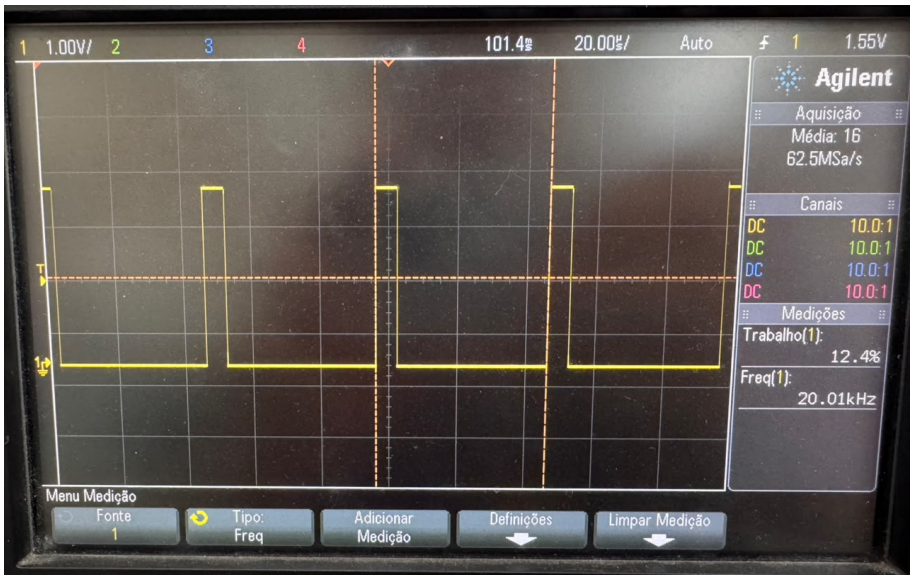
Tabela 6 – PWM medido com osciloscópio (CT12).

		Pedido		Esperado		Medido	
Canal	**Pino**	**Hz**	**%**	**Hz**	**%**	**Hz**	**%**
PWM1 (TIM1)	PE9	1 000	50,00	1 000,0000	50,00	1 000	50,00
PWM1 (TIM1)	PE9	100 000	90,00	100 000,0000	90,00	100 000	90,00
PWM2 (TIM2)	PA0	20 000	12,34	20 000,0000	12,33	20 000	12,34
PWM2 (TIM2)	PA0	1	50,00	1,0000	50,00	1	50,00
PWM3 (TIM3)	PA6	50	75,00	49,9997	75,00	50	75,00
PWM3 (TIM3)	PA6	100 000	25,00	100 000,0000	25,00	100 000	25,00

Fonte: elaborado pelo autor.

A Figura 14 mostra uma das medições, a do PWM2 em 20 kHz com 12,34%. Na captura, as medições automáticas indicam 20,01 kHz e 12,4%, diferenças de 0,05% e 0,06 ponto percentual, abaixo da resolução dessas medições e muito dentro do critério de 1% e 1 ponto percentual.

Figura 14 – PWM2 em 20 kHz e 12,34% no osciloscópio (CT12).



Fonte: elaborado pelo autor. Medições automáticas do osciloscópio, com média de 16 aquisições: 20,01 kHz e 12,4%.

5.4.3 I2C, SPI e UART

Sem ligações, os casos de integração verificaram as configurações, o mapa do escravo I2C, os *buffers* do SPI e da UART, as validações de parâmetros e, com a jiga mestra sem dispositivo no barramento I2C, a falta de reconhecimento, informada como ERRO_PERIFERICO sem travar a jiga. Com os *jumpers* do CT13, os dados do SPI e da UART voltaram iguais aos enviados em todos os modos e taxas.

Essa etapa revelou um defeito que os testes sem ligações não mostravam: na primeira transferência do SPI como mestre depois de a jiga ter sido escrava, os dados chegavam atrasados em 4 bytes. A causa era a FIFO de recepção do SPI2, que guarda até 4 bytes e não é esvaziada quando o HAL desliga o periférico. A correção, reiniciar o periférico pelo controlador de *reset* ao desligá-lo, foi validada na mesma montagem.

5.5 ROTINAS E BASE DE TEMPO

Os casos CT14 a CT18 passaram na placa. Uma rotina com um passo inválido foi recusada com o índice do passo, sem executar nenhum (CT14). Durante uma rotina, um comando direto e um novo RUN_ROUTINE receberam OCUPADO, enquanto ROUTINE_STATUS e PING continuaram atendidos (CT15). Um ABORT no meio de uma espera de 10 s interrompeu a rotina de imediato, com a jiga no estado seguro (CT16), e uma rotina mais longa que o tempo máximo terminou em TEMPO_ESGOTADO (CT17). Um RUN_ROUTINE retransmitido com o mesmo SEQ recebeu a resposta guardada, sem reiniciar a rotina (CT18).

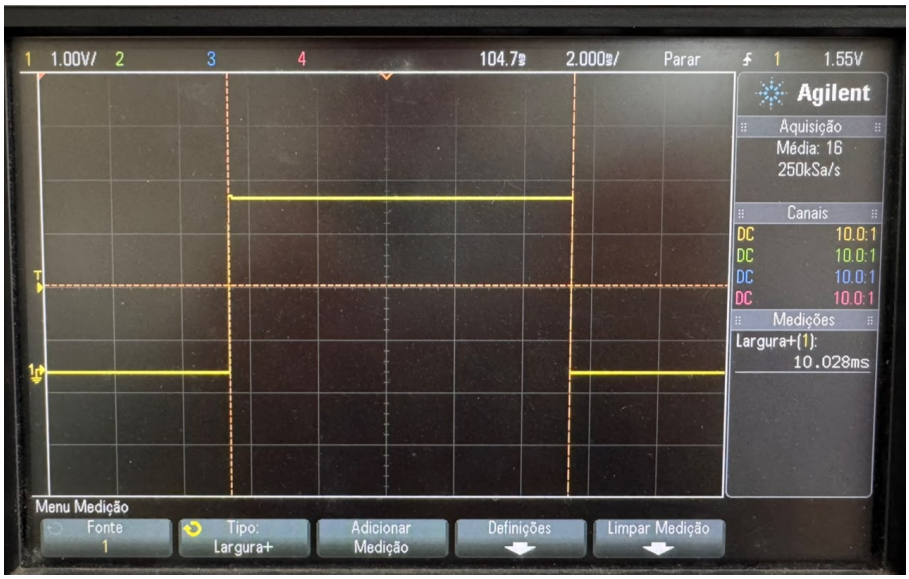
O CT19 verificou a base de tempo em três partes. Sem ligações, uma rotina fez leituras separadas por ESPERAs de 1, 10 e 100 ms, e a jiga registrou intervalos de 1 028, 10 029 e 100 029 µs. O excesso de cerca de 28 µs é constante e corresponde ao custo da leitura e do retorno da notificação que acorda a `TestExecTask`. Com a saída GPIO1 ligada à entrada GPIO5, o ESPERA_GPIO detectou as duas bordas e esgotou no tempo limite quando a entrada não mudou. Por fim, a largura de pulsos gerados no GPIO1 foi medida com o osciloscópio (Tabela 7 e Figura 15): os três pulsos ficaram de 19 a 24 µs acima da espera pedida, dentro da tolerância de 50 µs. O excesso não cresce com a duração, o que mostra que ele vem do custo fixo de cada passo, e não de um erro de frequência da base de tempo, que em 100 ms apareceria como desvio proporcional. A Figura 15 mostra o pulso de 10 ms numa repetição da medida, com a rotina gerando o pulso a cada 50 ms para facilitar o enquadramento. A largura foi de 10,028 ms, igual ao excesso de cerca de 28 µs que a própria jiga registrou na parte (a).

Tabela 7 – Largura dos pulsos do GPIO1 medida com osciloscópio (CT19).

Espera pedida (ms)	Largura medida (ms)	Excesso (µs)
1	1,024	24
10	10,019	19
100	100,02	20

Fonte: elaborado pelo autor.

Figura 15 – Pulso de 10 ms gerado no GPIO1 por uma rotina, no osciloscópio (CT19).



Fonte: elaborado pelo autor. Pulsos de 10 ms repetidos a cada 50 ms pela mesma rotina, com média de 16 aquisições: largura de 10,028 ms.

5.6 TESTES DE SISTEMA E DE ACEITAÇÃO

O CT20 passou nos cinco casos, em três execuções seguidas, com o ESP32-S3 ligado à UART4, ao I2C2 e ao SPI2 da jiga: o DUT leu e escreveu a jiga escrava nos dois barramentos e respondeu como escravo à jiga mestra. A primeira execução em bancada revelou quatro defeitos que a simulação e os testes sem ligações não mostravam, todos corrigidos:

- na escrava SPI da jiga, uma carga de dados feita depois da configuração só valia a partir da segunda transação do DUT, e o rearme da escrava travava a jiga por cerca de 200 ms;
- na escrava SPI, o último byte de uma transação de tamanho ímpar se perdia, porque a FIFO só avisava a cada dois bytes;
- no *firmware* do DUT, a escrava I2C devolvia sempre `7F FF FF`, porque a função do ESP-IDF que esvazia a FIFO de transmissão também reinicia o periférico no meio da leitura;

- com a UART da jiga desligada entre os casos, o DUT acumulava bytes soltos e recusava o primeiro comando; as baterias passaram a enviar um fim de linha antes de cada caso que comanda o DUT.

A montagem também mostrou a importância da ligação: os primeiros testes do I2C falharam por um *pull-up* de 4,7 Ω no lugar de 4,7 kΩ e pelo SCL ligado ao pino errado do DUT. O segundo erro foi localizado pela própria jiga, que, com os seus pinos como entradas, contou as bordas de cada linha durante uma transmissão do DUT: 96 bordas no SDA e nenhuma no SCL.

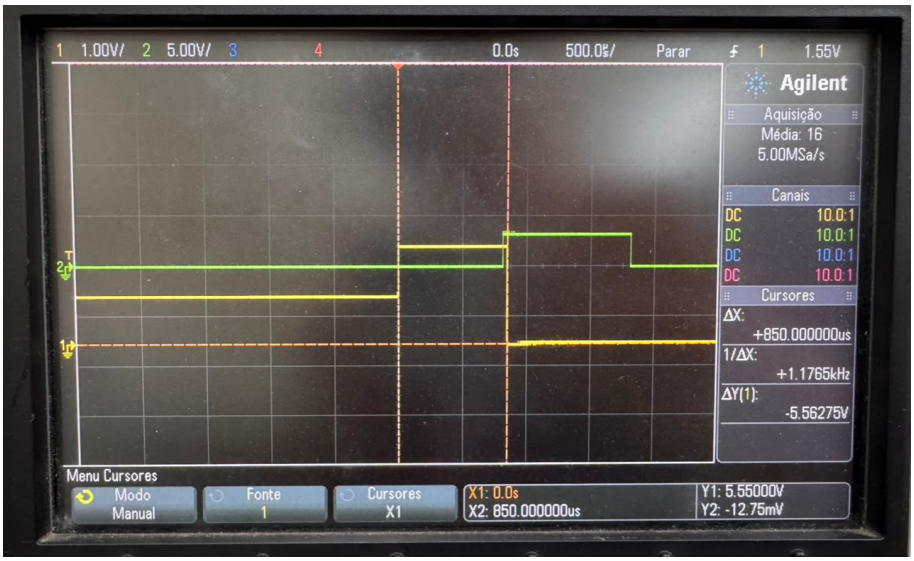
No CT21, a bateria de *loopback* passou nas dez execuções, com os mesmos vereditos e nenhuma retransmissão em 240 comandos. Uma bateria sem ligações, com o mapa do escravo I2C, estímulos e esperas, executada também dez vezes, deu os mesmos vereditos e as mesmas medições em todas as execuções, sem retransmissões em 140 comandos. Nos relatórios das dez execuções com *loopback*, a leitura do A0 para 1,0 V no DAC1 variou de 0,993 a 1,011 V, e a do A1 para 2,5 V no DAC2, de 2,501 a 2,509 V, dentro do ruído do ADC visto no CT10. O tempo de reação do GPIO5 e do GPIO8 ficou entre 16 e 17 µs em todas, e os dados da UART e do SPI foram idênticos. Os estímulos, portanto, se repetiram dentro da resolução da medida.

O CT22 executou a bateria do ventilador com histerese dez vezes no ESP32-S3 com o *firmware* correto: os cinco casos passaram em todas as execuções, sem retransmissões em 370 comandos. O tempo de reação registrado pelo ESPERA_GPIO ficou entre 0,07 e 1,01 ms ao ligar e entre 0,04 e 0,84 ms ao desligar, bem abaixo do limite de 20 ms. A dispersão corresponde ao período de 1 ms com que o DUT lê o sensor. Para conferir essa medida com um instrumento independente, o degrau de 1,0 para 2,0 V foi repetido com o osciloscópio no PA4 e na saída do DUT (Figura 16). Os cursores marcaram 850 µs entre as duas subidas, e a jiga registrou 0,814 ms na mesma execução. A diferença de 36 µs está dentro da resolução da leitura por cursores na base de tempo de 500 µs/div. A captura mostra também o fim da rotina. Assim que a jiga detecta a reação, a rotina termina e o estado seguro leva o DAC a 0 V; o DUT desliga a saída cerca de 0,8 ms depois.

A Tabela 8 mostra o resultado da mesma bateria sobre as quatro versões do *firmware* com defeitos injetados, cada uma gravada no ESP32-S3 e executada três vezes. Os vereditos foram os mesmos nas três execuções de cada versão, e todas foram reprovadas. Antes e depois delas, o *firmware* correto foi gravado de novo e passou nas três execuções, o que mostra que a reprovação vem do defeito, e não da montagem.

Cada defeito foi apanhado pelos casos em que muda o comportamento. Sem histerese (1), a saída desligou em 1,58 V e ligou em 1,72 V, o contrário do esperado nos casos 3 e 5. Com o limiar de ligar em 2,10 V (2) e com a saída presa em 0 (4), a saída não ligou em 2,0 V dentro dos 20 ms, e o caso 3, que parte da saída ligada,

Figura 16 – Degrau do DAC1 (canal 1) e reação da saída do ventilador no ESP32-S3 (canal 2), no osciloscópio (CT22).



Fonte: elaborado pelo autor. Canal 1 a 1 V/div e canal 2 a 5 V/div; os cursores marcam 850 µs entre o degrau e a reação.

também falhou. O atraso de 50 ms (3) só foi apanhado pelo caso 2, que mede o tempo de reação: a saída não mudou em 20 ms. No caso 4 dessa versão, a saída ainda não tinha ligado quando o DAC desceu para 1,2 V, e a espera pelo nível 0 terminou em 17 µs, o custo do próprio passo. O caso passou, portanto, sem exercitar o desligamento. A bateria seria mais rigorosa se conferisse que a saída está em 1 antes da descida, ajuste simples que fica registrado como melhoria da bateria. A Figura 17 mostra a janela da aplicação ao fim da execução com o defeito (1).

Tabela 8 – Vereditos do CT22 com defeitos injetados no *firmware* do DUT.

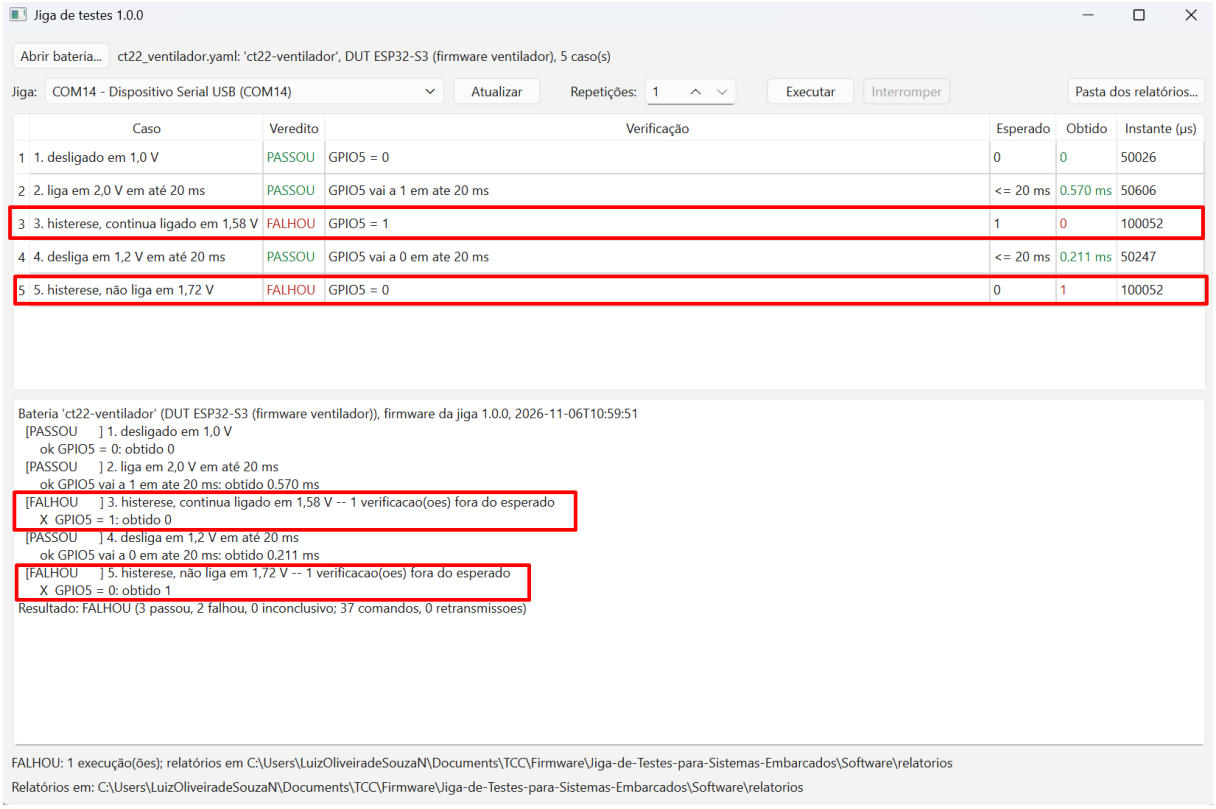
	Caso da bateria				
***Firmware* do DUT**	**1,0 V**	**liga 2,0 V**	**mantém 1,58 V**	**desliga 1,2 V**	**não liga 1,72 V**
Correto	passou	passou	passou	passou	passou
(1) sem histerese	passou	passou	**falhou**	passou	**falhou**
(2) limiar em 2,10 V	passou	**falhou**	**falhou**	passou	passou
(3) saída atrasada 50 ms	passou	**falhou**	passou	passou	passou
(4) saída presa em 0	passou	**falhou**	**falhou**	passou	passou

Fonte: elaborado pelo autor. Cada versão foi executada três vezes, com os mesmos vereditos. O *firmware* correto, gravado antes e depois das versões com defeito, passou nas seis execuções.

5.7 DESEMPENHO DA COMUNICAÇÃO

A Tabela 9 e a Figura 18 apresentam os tempos de ida e volta medidos no computador (Windows 11, Python 3.14), com a jiga ligada à porta USB OTG. O CT23 e

Figura 17 – Janela da aplicação após o CT22 com o *firmware* sem histerese.



Fonte: elaborado pelo autor. Os retângulos vermelhos, acrescentados à captura, destacam os casos reprovados.

o CT24 passaram: o maior tempo entre os 11 000 comandos foi de 3,1 ms, cerca de 16 vezes menor que o limite de 50 ms do RNF07, sem erros não recuperados nem retransmissões. Os testes foram executados duas vezes. A tabela traz a segunda, cujos dados foram gravados; a primeira diferiu apenas por um comando isolado, discutido adiante.

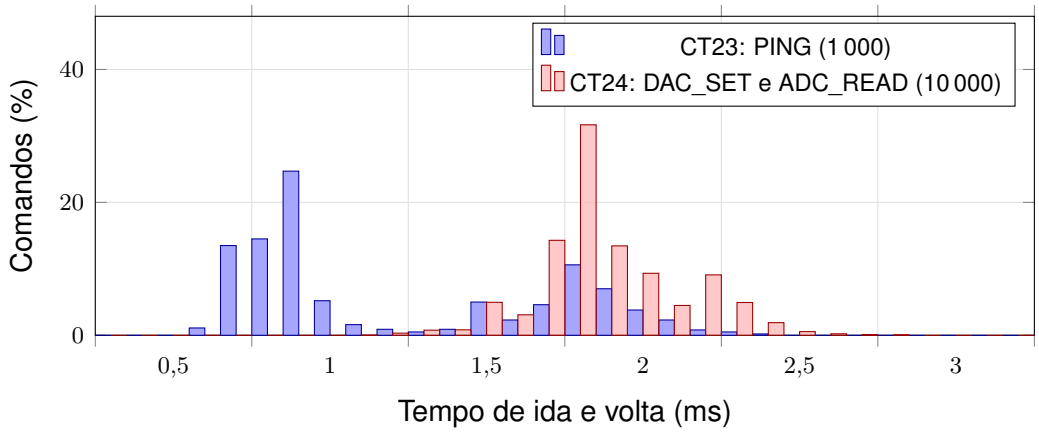
Tabela 9 – Tempo de ida e volta dos comandos (CT23 e CT24).

		Tempo de ida e volta (ms)						
Caso	**Comandos**	**Média**	**Mediana**	**P95**	**P99**	**Máx.**	**Erros**	**Retr.**
CT23	1 000	1,46	1,17	2,26	2,47	2,67	0	0
CT24	10 000	2,13	2,07	2,54	2,69	3,08	0	0

Fonte: elaborado pelo autor. CT23: PING; CT24: DAC_SET e ADC_READ alternados. Erros: não recuperados; Retr.: retransmissões.

Dois aspectos merecem destaque. O primeiro é a diferença entre os casos: o PING é respondido pela própria `CommTask`, enquanto os comandos do CT24 passam pela fila e pela `TestExecTask`, o que acrescenta cerca de 0,7 ms em média. O segundo é a cauda: na primeira execução do CT24, um único comando levou 15 ms, efeito do escalonamento do sistema operacional do computador, que a média e o

Figura 18 – Distribuição do tempo de ida e volta dos comandos (intervalos de 0,1 ms).



Fonte: elaborado pelo autor.

percentil 99 não revelam e que justifica o registro do máximo. Mesmo esse valor ficou bem abaixo do tempo limite de 100 ms. A ausência de retransmissões indica que, na ligação USB direta, o mecanismo quase nunca é acionado; ele continua necessário para a perda de alinhamento e o reinício de um dos lados, verificados do CT03 ao CT05.

Esses tempos confirmam a escolha de executar as rotinas na jiga (subseção 4.4.3). Comandado pelo computador, o intervalo entre um estímulo e uma leitura variaria de cerca de 1,4 a 3 ms, com casos isolados acima de 15 ms. Numa rotina, ele tem um custo fixo de cerca de 28 µs por passo, de 50 a 107 vezes menor, e é registrado com resolução de 1 µs.

5.8 EMULAÇÃO DE SENSORES E GERADOR DE ONDAS

A conversão do BME280 foi conferida contra a folha de dados. Com os coeficientes e os valores brutos do exemplo do fabricante, a aplicação e o *driver* do DUT resultaram em 25,08 °C e 100 653,27 Pa, os valores publicados. A conversão inversa acertou a temperatura, a pressão e a umidade com erro de até 0,006 °C, 0,003 hPa e 0,006 ponto percentual, em toda a faixa do sensor.

Com o ESP32-S3, o CT25 e o CT26 passaram em dez execuções cada, sem retransmissões. O DUT leu os dois sensores emulados com os seus *drivers*, sem nenhum código para a jiga, e informou os valores aplicados, como aceleração de 0,866 g, giro de -250 °/s, temperatura de -5 °C, pressão de 780 hPa e umidade de 82,5%. Com o BME280 no SPI e o MPU-6500 no I2C ao mesmo tempo, os dois barramentos funcionaram juntos.

O caso da rampa do CT26 merece registro, porque o seu critério foi revisto. A rampa vai de 10 a 30 °C em 1 s, e a bateria pede a leitura aos 500 ms. O critério inicial esperava 20 °C, o valor da rampa naquele instante, com tolerância de 0,05 °C,

e a primeira execução falhou: o DUT informou 20,38 °C. A diferença vem da latência do *driver*, que lê a calibração antes dos dados e obtém a temperatura cerca de 19 ms depois do pedido; nesse instante, a rampa vale $10 + 20 \times 0{,}519 = 20{,}38$ °C. O critério passou, então, a considerar essa latência, conforme a subseção 3.3.2, com o valor esperado de 20,38 °C e a mesma tolerância de 0,05 °C, e a bateria revisada passou nas dez execuções, sem retransmissões em 1 300 comandos. A repetição exata do valor em todas as execuções mostra que a onda é atualizada nos instantes previstos.

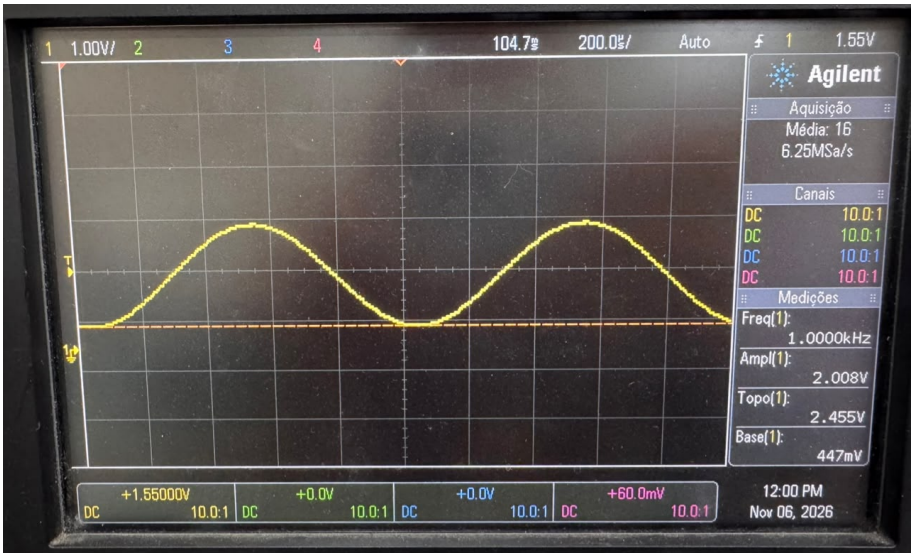
No CT27, uma bateria sem ligações conferiu ondas num registrador do MPU-6500 emulado, lendo o mapa da escrava em instantes conhecidos. Uma rampa de 0 a 1 g em 1 s valia 0,25, 0,5 e 1 g aos 250, 500 e 1 000 ms, e uma onda quadrada de 200 ms alternou entre 35 e 25 °C nos instantes previstos, em seis execuções seguidas. No osciloscópio, com as ondas mantidas por dois minutos, foi medida a senoide programada no DAC1, de 0,5 a 2,5 V a 1 kHz com passo de 10 µs. A primeira medição deu 1 kHz e 2,015 V de pico a pico, erro de 0,75% na amplitude. Numa segunda medição (Figura 19), a frequência foi de 1,0000 kHz e a amplitude, de 2,008 V, erro de 0,4%. Os extremos, medidos com média de aquisições, ficaram em 2,467 e 0,452 V na primeira medição e em 2,455 e 0,447 V na segunda, cerca de 40 a 50 mV abaixo dos programados; como a amplitude está correta e os dois extremos se deslocaram juntos, o desvio é atribuído ao deslocamento vertical do osciloscópio, cuja exatidão em tensão contínua é inferior à do multímetro (subseção 3.3.5). No PWM1, a frequência se manteve em 1 kHz enquanto a razão cíclica percorria a triangular, de 0 a 100%. O período de 100 ms da triangular não pôde ser lido na tela, porque, na base de tempo necessária, os cerca de 200 pulsos se sobrepõem; essa medida exigiria um filtro passa-baixas e fica como verificação complementar.

5.9 CUSTO

O custo da jiga, dado pela Tabela 1, foi de R$ 196,84, cerca de 36 dólares: R$ 187,84 da placa, cotada num distribuidor oficial sem os impostos de importação, e R$ 9,00 dos *jumpers*. Os impostos de importação aumentam esse valor, mas o total permanece na ordem de centenas de reais.

As plataformas comerciais de HIL custam de 24 a 194 mil dólares nos registros públicos de compra (Frassinelli, 2026), mas não são equipamentos equivalentes: simulam a planta completa em tempo real, com muito mais canais, resolução e velocidade. O concorrente mais próximo em custo é um instrumento USB multifunção como o Analog Discovery 3, de cerca de 379 dólares (DigiKey, 2026a), dez vezes o custo da jiga. Ele mede e gera sinais com mais resolução e banda, mas a jiga oferece o que ele não traz pronto. São eles os sensores comerciais emulados como escravos I2C e SPI, rotinas executadas no próprio equipamento com instantes registrados em microssegundos e baterias de teste com veredito automático. Os dois se complementam,

Figura 19 – Senoide de 0,5 a 2,5 V e 1 kHz gerada no DAC1, no osciloscópio (CT27).



Fonte: elaborado pelo autor. Medições automáticas, com média de 16 aquisições: 1,0000 kHz, amplitude (topo menos base) de 2,008 V, topo de 2,455 V e base de 0,447 V.

e a jiga cobre esse escopo dentro do orçamento de uma disciplina ou de um projeto acadêmico, o que atende ao RNF01.

5.10 DISCUSSÃO

Os resultados confirmam, em escopo maior, o que os trabalhos de baixo custo da seção 2.4 indicavam. Qu e Lin (2012) mostraram que um STM32 consegue simular um acelerômetro analógico. Aqui, o mesmo tipo de microcontrolador emulou dois sensores digitais completos, lidos por *drivers* comuns nos dois barramentos ao mesmo tempo. Jamaludin *et al.* (2018) substituíram um HIL comercial por um equipamento com microcontrolador e conferiram as saídas do painel de instrumentos dentro da tolerância da especificação. A jiga generaliza essa ideia para um DUT qualquer e acrescenta a descrição do teste numa bateria com veredito automático. Lu *et al.* (2007) defenderam o HIL de baixo custo com componentes de prateleira; a jiga leva o argumento ao extremo de uma única placa de desenvolvimento, ao custo de não fechar a malha com um modelo da planta. Por fim, os laboratórios de Jacko *et al.* (2022) geram tensões e PWM para o ensino. A jiga acrescenta a temporização em microssegundos e a emulação de barramentos, o que permite usá-la também na verificação de *firmware*, e não só na demonstração de conceitos.

A principal contribuição observada nos ensaios, porém, foi a revelação de defeitos. Cinco defeitos só apareceram em bancada, com a jiga ligada a si mesma ou ao DUT, embora a simulação e os testes sem ligações passassem. Foram três no SPI da jiga, um na escrava I2C do DUT de referência e um na comunicação pela UART

entre os casos. Isso mostra, em pequena escala, o valor do teste com *hardware* real que motiva o trabalho.

5.11 ATENDIMENTO AOS REQUISITOS

O Quadro 8 relaciona cada requisito aos resultados obtidos.

Quadro 8 – Atendimento aos requisitos.

Req.	**Evidência**	**Situação**
RF01	DAC medido com multímetro (Tabela 4) e CT10	atendido na faixa útil de 0,1 a 3,2 V
RF02	CT11, 16 combinações	atendido
RF03	CT12 (Tabela 6)	atendido
RF04	casos sem ligação; CT13 com *jumpers*; CT20 com o ESP32-S3	atendido
RF05	CT10, CT11; UART no CT13	atendido
RF06	CT02, CT08, CT09 e os 73 casos de integração	atendido
RF07	CT06; baterias versionadas com o projeto	atendido
RF08	CT07; CT22 com o *firmware* correto e com defeitos injetados	atendido
RF09	relatórios CSV e JSON em cada execução (CT21)	atendido
RF10	CT14 a CT19	atendido
RF11	CT25 e CT26 com o ESP32-S3	atendido
RF12	CT27 na placa e no osciloscópio	atendido
RNF01	lista de materiais (seção 5.9)	atendido
RNF02	CT22: bateria só com os pinos do DUT; *firmware* do DUT sem código para a jiga	atendido para o ESP32-S3
RNF03	CT03 a CT05, CT18 e CT24	atendido
RNF04	CT21 (dez execuções com os mesmos vereditos)	atendido
RNF05	estado seguro (CT16, CT17); níveis e correntes (subseção 4.2.3)	atendido, sem proteção contra ligação errada
RNF06	Apêndice A; módulos testados isoladamente	atendido
RNF07	CT23 e CT24 (máximo de 3,1 ms)	atendido

Fonte: elaborado pelo autor.

5.12 LIMITAÇÕES DA AVALIAÇÃO

Os resultados devem ser lidos com as seguintes limitações, que restringem o quanto deles se pode generalizar:

- **um único DUT**: a validação usou apenas o ESP32-S3, de 3,3 V. A portabilidade para outras arquiteturas e para DUTs de 5 V foi tratada no projeto, mas não demonstrada em bancada;
- ***firmware* do próprio autor**: o DUT de referência e as suas falhas injetadas foram escritos pelo autor da jiga. As falhas representam erros comuns, mas não esgotam os defeitos possíveis, e uma avaliação com *firmware* de terceiros daria evidência mais forte;

- **uma única bancada**: todas as medições foram feitas com uma placa, um DUT e um conjunto de instrumentos, sem repetição em outra montagem;
- **cobertura parcial**: a cobertura do *firmware* foi medida só nos módulos que rodam no computador (Tabela 3); o código das interrupções foi verificado pelos testes de integração, sem medida de cobertura.

No *hardware*, as limitações conhecidas são a faixa útil do DAC, de 0,1 a 3,2 V; as entradas digitais sem resistor interno, imposição da tolerância a 5 V; a ausência de proteção contra ligação errada; e a pausa exigida entre o endereço e os dados na escrava SPI em modo de registradores.

6 CONSIDERAÇÕES FINAIS E SUGESTÕES PARA TRABALHOS FUTUROS

Este trabalho partiu da pergunta: como validar o *firmware* de sistemas embarcados de forma reprodutível e de baixo custo, simulando sinais de sensores e periféricos, sem depender de sensores reais ou de plataformas comerciais de HIL? A resposta construída é uma jiga de testes de cerca de R$ 200, formada por uma placa NUCLEO-F767ZI e uma aplicação *desktop*. Ela aplica ao DUT estímulos analógicos, digitais e PWM, emula sensores e periféricos nos barramentos I2C, SPI e UART e decide, a partir de uma bateria descrita em arquivo, se o *firmware* se comportou como o esperado. Os ensaios no ESP32-S3 mostraram que essa abordagem funciona. A jiga aprovou o *firmware* correto em todas as execuções, reprovou as versões com defeitos injetados em todas as execuções, nos casos que cada defeito afeta, e repetiu os mesmos estímulos e vereditos de uma execução para outra.

O Quadro 9 relaciona cada objetivo específico à evidência que o comprova.

Quadro 9 – Objetivos específicos e evidências.

	Objetivo	**Evidência**
(a)	*Firmware* com estímulos, aquisição, emulação de periféricos e sensores e formas de onda	CT08 a CT13 com instrumentos e *loopback*; CT20, CT25 e CT26 com o ESP32-S3; CT27 (seção 5.4 e seção 5.8)
(b)	Protocolo com detecção de erros, retransmissão e rotinas temporizadas	CT01 a CT05, CT14 a CT19, CT23 e CT24; especificação no Apêndice A
(c)	Aplicação com baterias em arquivo, veredito e relatório	CT06, CT07 e CT21; Figura 11 e Apêndice D
(d)	Verificação em níveis, com instrumentos independentes	Tabela 2; multímetro e osciloscópio nos CT10, CT12, CT19, CT22 e CT27
(e)	Validação no ESP32-S3, inclusive com defeitos injetados	CT22 com o *firmware* correto e com quatro defeitos (Tabela 8)

Fonte: elaborado pelo autor.

Duas decisões de projeto se mostraram decisivas. A primeira foi executar cada caso de teste na própria jiga, e não por comandos isolados do computador. O intervalo entre um estímulo e uma leitura deixou de depender da USB e do sistema operacional do computador, que variam na escala de milissegundos, e passou a um custo fixo de dezenas de microssegundos, conferido com o osciloscópio. A segunda foi emular os sensores como escravos de barramento, com os mapas de registradores das folhas de dados. O DUT os leu com *drivers* comuns, sem nenhum código para a jiga, o que é a condição para um teste *black-box* de *firmware* real.

O teste em bancada também mostrou o seu próprio valor: cinco defeitos só apareceram com a jiga ligada a si mesma ou ao DUT, embora a simulação e os testes sem ligações passassem. A verificação incremental, em que cada versão só era fechada com os testes passando na placa, permitiu corrigi-los com segurança.

As limitações estão discutidas na seção 5.12. As principais são a validação com um único DUT, de 3,3 V, com *firmware* do próprio autor, e a ausência de proteção contra ligação errada. A jiga também não substitui uma plataforma comercial de HIL: os seus estímulos seguem um roteiro, em malha aberta, sem um modelo da planta. O seu papel é outro, o de tornar o teste automatizado de *firmware* com estímulos elétricos reais acessível a laboratórios acadêmicos e pequenas equipes, que é o público visado por este trabalho.

6.1 SUGESTÕES PARA TRABALHOS FUTUROS

A partir dos resultados e das limitações observadas, sugerem-se as seguintes continuações:

- **validação com outros DUTs e com *firmware* de terceiros**, de arquiteturas e níveis lógicos diferentes, inclusive de 5 V, reaproveitando a bateria do CT22 e avaliando a jiga sobre defeitos que o seu autor não conhece;
- **placa de interface dedicada**, com conectores identificados, resistores em série nas saídas, *pull-ups* e *pull-downs* selecionáveis nas entradas e divisores para sinais analógicos de 5 V, que protejam a jiga e o DUT contra ligação errada;
- **malha fechada**, com um modelo simples da planta executado na jiga, que calcule os sinais de sensor a partir das saídas do DUT, aproximando a jiga do HIL clássico;
- **mais modelos de sensores**, como magnetômetros, sensores de distância e conversores analógico-digitais externos, aproveitando o mapa, as máscaras e o gerador de ondas existentes;
- **SPI sem a pausa depois do endereço**, com uma pequena lógica programável que entregue o registrador pedido dentro do próprio byte, para atender mestres que usam DMA;
- **captura de sinais PWM do DUT**, medindo frequência e razão cíclica com os temporizadores da jiga;
- **integração contínua com a jiga**, executando as baterias a cada alteração do *firmware* do DUT e publicando os relatórios, o que a linha de comando e os códigos de saída da aplicação já permitem;
- **editor gráfico de baterias**, para que o usuário monte os casos sem editar o arquivo;

- **portabilidade da jiga** para outros microcontroladores, reescrevendo apenas a camada de abstração de *hardware*.

REFERÊNCIAS

BANCO CENTRAL DO BRASIL. **Cotações e boletins: taxa PTAX de venda do dólar**. Brasília, 2026. R$ 5,4621 em 5 nov. 2026. Disponível em: https://www.bcb.gov.br/estabilidadefinanceira/historicocotacoes. Acesso em: 5 nov. 2026. Citado na página 38.

BARBOSA, J. R. **Estudo e definição de uma metodologia de teste de software no contexto de sistemas embarcados críticos**. Dissertação (Dissertação (Mestrado em Ciência da Computação)) — Universidade Federal de Goiás, Goiânia, 2011. Disponível em: https://ww2.inf.ufg.br/ppgcc/sites/www.inf.ufg.br.mestrado/files/uploads/Dissertacoes/JacsonRodrigues.pdf. Acesso em: 5 nov. 2026. Citado na página 23.

BARRY, R. **Mastering the FreeRTOS real time kernel: a hands-on tutorial guide**. [S.l.]: Real Time Engineers, 2016. Citado na página 22.

BRANAM, S. **Unit testing for embedded software development**. 2026. Dojo Five. Disponível em: https://dojofive.com/blog/unit-testing-for-embedded-software-development/. Acesso em: 5 nov. 2026. Citado 4 vezes nas páginas 18, 23, 29 e 39.

CABRAL, T. F. **Sistema de detecção de anomalias em sistemas embarcados**. Dissertação (Trabalho de Conclusão de Curso (Bacharelado em Engenharia de Computação)) — Escola Politécnica de Pernambuco, Universidade de Pernambuco, Recife, 2021. Disponível em: https://www.ecomp.poli.br/ListaTCC/20201/Tiago%20Farias%20Cabral.pdf. Acesso em: 5 nov. 2026. Citado na página 24.

CHAGAS, J. **Teste de software em sistemas embarcados automotivos**. 2020. Embarcados. Disponível em: https://embarcados.com.br/teste-de-software-em-sistemas-embarcados-automotivos/. Acesso em: 5 nov. 2026. Citado na página 21.

DIGIKEY. **410-415: Analog Discovery 3 – Digilent**. 2026. Preço unitário de US$ 379,00. Disponível em: https://www.digikey.com.br/pt/products/detail/digilent-inc/410-415/19235661. Acesso em: 5 nov. 2026. Citado 2 vezes nas páginas 25 e 66.

DIGIKEY. **NUCLEO-F767ZI: STMicroelectronics**. 2026. Preço unitário de US$ 34,39. Disponível em: https://www.digikey.com.br/pt/products/detail/stmicroelectronics/NUCLEO-F767ZI/6004740. Acesso em: 5 nov. 2026. Citado na página 38.

DSPACE. **SCALEXIO: HIL simulator hardware**. Paderborn, [2026?]. Disponível em: https://www.dspace.com/en/pub/home/products/hw/simulator_hardware/scalexio.cfm. Acesso em: 5 nov. 2026. Citado 3 vezes nas páginas 17, 24 e 25.

ENGENHARIA HíBRIDA. **Jiga de teste automatizada com software integrado**. [2026?]. Disponível em: https://www.engenhariahibrida.com.br/jiga-de-testes-automatizados. Acesso em: 5 nov. 2026. Citado na página 25.

ESPRESSIF SYSTEMS. **pytest-embedded: a pytest plugin for embedded testing**. [2026?]. Disponível em: https://github.com/espressif/pytest-embedded. Acesso em: 5 nov. 2026. Citado na página 25.

Euripedes Filho. **Testes automatizados: como faço com o hardware?** 2017. Embarcados. Disponível em: https://embarcados.com.br/testes-automatizados-e-o-hardware/. Acesso em: 5 nov. 2026. Citado 4 vezes nas páginas 17, 21, 23 e 34.

FATHY, H. K. *et al.* Review of hardware-in-the-loop simulation and its prospects in the automotive area. In: **Modeling and Simulation for Military Applications**. Bellingham: SPIE, 2006. (Proceedings of SPIE, v. 6228), p. 62280E. Citado na página 21.

FRASSINELLI, D. **Embedded simulation cost: published prices stop where the checkout stops**. 2026. Veecle. Disponível em: https://veecle.ai/blog/embedded-simulation-cost. Acesso em: 5 nov. 2026. Citado 3 vezes nas páginas 17, 24 e 66.

GIL, A. C. **Como elaborar projetos de pesquisa**. 6. ed. São Paulo: Atlas, 2017. Citado na página 27.

GRENNING, J. W. **Test-driven development for embedded C**. Raleigh: Pragmatic Bookshelf, 2011. Citado 2 vezes nas páginas 23 e 29.

INTERNATIONAL SOFTWARE TESTING QUALIFICATIONS BOARD. **Certified tester foundation level syllabus: v4.0.1**. [S.l.], 2024. Disponível em: https://www.istqb.org/wp-content/uploads/2024/11/ISTQB_CTFL_Syllabus_v4.0.1.pdf. Acesso em: 5 nov. 2026. Citado na página 21.

JACKO, P. *et al.* Remote IoT education laboratory for microcontrollers based on the STM32 chips. **Sensors**, v. 22, n. 4, p. 1440, 2022. Citado 5 vezes nas páginas 18, 24, 25, 35 e 67.

JAMALUDIN, W. S. W. *et al.* Adopting hardware-in-the-loop for testing vehicle instrument panel using economical approach. **Indonesian Journal of Electrical Engineering and Computer Science**, v. 10, n. 1, p. 50–58, 2018. Citado 3 vezes nas páginas 24, 25 e 67.

KESTER, W. (Ed.). **Data conversion handbook**. Burlington: Newnes, 2005. Citado na página 23.

KOOPMAN, P.; CHAKRAVARTY, T. Cyclic redundancy code (CRC) polynomial selection for embedded networks. In: **International Conference on Dependable Systems and Networks**. Florença: IEEE, 2004. p. 145–154. Citado na página 22.

LABGRID. **Labgrid: embedded board control python library**. 2026. Disponível em: https://labgrid.readthedocs.io/. Acesso em: 5 nov. 2026. Citado na página 25.

LINS, F. C. **Estudo de metodologia de desenvolvimento baseado em testes aplicado a sistemas embarcados de tempo real**. Dissertação (Trabalho de Conclusão de Curso (Bacharelado em Engenharia Elétrica)) — Universidade Federal de Campina Grande, Campina Grande, 2016. Disponível em: https://dspace.sti.ufcg.edu.br/handle/riufcg/18630. Acesso em: 5 nov. 2026. Citado 3 vezes nas páginas 18, 23 e 39.

LU, B. *et al.* A low-cost real-time hardware-in-the-loop testing approach of power electronics controls. **IEEE Transactions on Industrial Electronics**, v. 54, n. 2, p. 919–931, 2007. Citado 3 vezes nas páginas 24, 25 e 67.

MENDONÇA, R. N. **Metodologia de teste de sistemas embarcados aplicada a medidores inteligentes e sistemas de medição centralizada**. Dissertação (Monografia (Graduação em Engenharia Elétrica)) — Universidade Federal do Ceará, Fortaleza, 2019. Disponível em: https://repositorio.ufc.br/handle/riufc/45316. Acesso em: 5 nov. 2026. Citado na página 23.

MIHALIČ, F.; TRUNTIČ, M.; HREN, A. Hardware-in-the-loop simulations: a historical overview of engineering challenges. **Electronics**, v. 11, n. 15, p. 2462, 2022. Citado na página 21.

MYERS, G. J.; BADGETT, T.; SANDLER, C. **The art of software testing**. 3. ed. Hoboken: John Wiley & Sons, 2011. Citado na página 21.

NATIONAL INSTRUMENTS. **Hardware-in-the-loop (HIL) testing**. Austin, [2026?]. Disponível em: https://www.ni.com/en/solutions/hardware-in-the-loop-testing.html. Acesso em: 5 nov. 2026. Citado 3 vezes nas páginas 17, 24 e 25.

NETO, L. O. d. S. **Jiga de testes para sistemas embarcados: firmware, aplicação desktop e DUT de referência**. 2026. Versão 1.0.0. Disponível em: https://github.com/1luizneto/Jiga-de-Testes-para-Sistemas-Embarcados. Acesso em: 5 nov. 2026. Citado na página 54.

NXP SEMICONDUCTORS. **UM10204: I2C-bus specification and user manual**. Eindhoven, 2021. Rev. 7.0. Disponível em: https://www.nxp.com/docs/en/user-guide/UM10204.pdf. Acesso em: 5 nov. 2026. Citado na página 23.

OPENHTF. **OpenHTF: the open-source hardware testing framework**. [2026?]. Disponível em: https://github.com/google/openhtf. Acesso em: 5 nov. 2026. Citado na página 25.

PRODANOV, C. C.; FREITAS, E. C. de. **Metodologia do trabalho científico: métodos e técnicas da pesquisa e do trabalho acadêmico**. 2. ed. Novo Hamburgo: Feevale, 2013. Citado na página 27.

QU, B.; LIN, H. Design of acceleration sensor simulation system based on STM32. **Advanced Materials Research**, v. 588–589, p. 940–943, 2012. Citado 4 vezes nas páginas 24, 25, 35 e 67.

STMICROELECTRONICS. **STM32F765xx, STM32F767xx, STM32F768Ax, STM32F769xx: datasheet DS11532**. Genebra, [20–]. Disponível em: https://www.st.com/resource/en/datasheet/stm32f767zi.pdf. Acesso em: 5 nov. 2026. Citado 2 vezes nas páginas 31 e 36.

STMICROELECTRONICS. **RM0410 reference manual: STM32F76xxx and STM32F77xxx advanced Arm-based 32-bit MCUs**. Genebra, 2018. Rev. 4. Disponível em: https://www.st.com/resource/en/reference_manual/rm0410-stm32f76xxx-and-stm32f77xxx-advanced-armbased-32bit-mcus-stmicroelectronics.pdf. Acesso em: 5 nov. 2026. Citado na página 23.

STMICROELECTRONICS. **STM32F7 series: Arm Cortex-M7 MCUs**. Genebra, [2026?]. Disponível em: https://www.st.com/en/microcontrollers-microprocessors/stm32f7-series.html. Acesso em: 5 nov. 2026. Citado na página 35.

TAMARI, A. **Como projetar para teste em sistemas embarcados**. 2023. Altium Resources. Disponível em: https://resources.altium.com/pt/p/how-design-test-embedded-systems. Acesso em: 5 nov. 2026. Citado na página 34.

TANENBAUM, A. S.; WETHERALL, D. J. **Computer networks**. 5. ed. Boston: Pearson Prentice Hall, 2011. Citado na página 22.

TEJADO, I. *et al.* Low-cost hardware-in-the-loop testbed of a mobile robot to support learning in automatic control and robotics. **IFAC-PapersOnLine**, v. 49, n. 6, p. 242–247, 2016. Citado 2 vezes nas páginas 24 e 25.

THE EMBEDDED KIT. **Alternative to Robot Framework for embedded testing**. 2024. Disponível em: https://theembeddedkit.io/blog/alternative-to-robot-framework/. Acesso em: 5 nov. 2026. Citado 2 vezes nas páginas 25 e 49.

THE EMBEDDED KIT. **How we built a test bench for embedded system testing**. 2026. Elektor Labs. Disponível em: https://www.elektormagazine.com/labs/how-we-built-a-test-bench-for-embedded-system-testing. Acesso em: 5 nov. 2026. Citado na página 25.

Apêndices

APÊNDICE A – ESPECIFICAÇÃO COMPLETA DO PROTOCOLO DE COMUNICAÇÃO

Este apêndice especifica o protocolo de comandos entre a aplicação *desktop* (computador) e o *firmware* da jiga, transportado sobre USB-CDC. O resumo do protocolo e as justificativas de projeto estão na seção 4.4.

A.1 PRINCÍPIOS E CONVENÇÕES

O protocolo segue o modelo requisição–resposta: o computador é o único a iniciar a comunicação, e cada comando enviado recebe exatamente uma resposta da jiga, um ACK (confirmação, com dados quando aplicável) ou um NACK (recusa, com o código do erro). O computador só envia um novo comando depois de receber a resposta do anterior.

Há dois tipos de comando. Os **comandos diretos** (Apêndice A.5) executam uma única operação sobre um periférico e respondem ao seu término; servem à operação manual e aos testes de integração. Os **comandos de rotina** (Apêndice A.6) transferem à jiga uma rotina de teste completa – uma sequência de estímulos, esperas e aquisições –, que a jiga executa sozinha; o computador acompanha a execução por consultas e, ao final, lê as medições registradas.

As convenções adotadas são:

- campos de vários bytes são transmitidos em ordem *little-endian*, isto é, o byte menos significativo primeiro;
- os tipos são indicados por `u8`, `u16` e `u32`, para inteiros sem sinal de 8, 16 e 32 bits;
- os canais de DAC são numerados de 1 a 2, os de PWM de 1 a 3, os de ADC de 0 a 2 (A0 a A2) e as saídas digitais GPIO1 a GPIO4 por 1 a 4;
- as entradas digitais GPIO5 a GPIO8 são lidas em conjunto, na forma de uma máscara de 8 bits, em que o bit 0 corresponde ao GPIO5 e o bit 3 ao GPIO8; os bits 4 a 7 são sempre zero;
- as numerações de pinos e canais correspondem ao mapeamento dos Quadros 19 e 20.

A.2 FORMATO DO QUADRO

Como o USB-CDC entrega um fluxo contínuo de bytes, sem fronteiras entre mensagens, cada mensagem é encapsulada em um quadro com o formato da Figura 8 e do Quadro 10.

Quadro 10 – Campos do quadro do protocolo.

Campo	Tamanho	Descrição
SOF	1 byte	Início de quadro, sempre `0x7E`.
LEN	1 byte	Número de bytes do campo PAYLOAD (0 a 240).
SEQ	1 byte	Número de sequência. O computador o incrementa a cada novo comando (volta a 0 após 255), e a jiga o repete na resposta.
CMD	1 byte	Código do comando (Quadros 12 a 14). Na resposta de sucesso, é o código do comando com o bit 7 ligado; na recusa, é `0xFF`.
PAYLOAD	0 a 240 bytes	Parâmetros do comando ou dados da resposta.
CRC-16	2 bytes	Verificação de integridade calculada sobre LEN, SEQ, CMD e PAYLOAD (Apêndice A.3).

Fonte: elaborado pelo autor.

O tamanho máximo de um quadro é de 246 bytes. O receptor sincroniza-se procurando o byte SOF e valida o quadro pelo CRC. Se o CRC falhar, ou se o LEN for maior que 240, o receptor descarta o quadro, **sem responder**, e volta a procurar o próximo SOF; o mesmo acontece com um quadro incompleto, descartado quando o intervalo entre dois bytes passa de 20 ms. A jiga não responde a um quadro corrompido porque, nele, nem o SEQ nem o CMD são confiáveis: uma resposta com um SEQ errado poderia ser associada pelo computador a outro comando. A recuperação fica a cargo do computador, que retransmite o comando ao esgotar o tempo limite de resposta (Apêndice A.7).

A.3 VERIFICAÇÃO DE INTEGRIDADE

O CRC é o CRC-16/CCITT-FALSE: polinômio `0x1021`, valor inicial `0xFFFF`, sem inversão de bits na entrada ou na saída e sem operação final de XOR. O resultado de 16 bits é transmitido em *little-endian*. Uma implementação correta produz `0x29B1` para a sequência ASCII `123456789`, valor que pode ser usado como teste de unidade.

A.4 RESPOSTAS

A.4.1 ACK

Quando o comando é aceito, a jiga responde com o mesmo SEQ e com CMD igual ao código do comando com o bit 7 ligado (`CMD OU 0x80`). O campo PAYLOAD carrega os dados de retorno, quando existirem. Para os comandos diretos, o ACK

indica que a operação foi executada; para RUN_ROUTINE, indica que a rotina foi validada e sua execução começou.

A.4.2 NACK

Quando o comando não pode ser executado, a jiga responde com o mesmo SEQ, CMD igual a `0xFF` e PAYLOAD com o código do comando recusado (`u8`) e o código do erro (`u8`), conforme o Quadro 11. No erro ROTINA_INVALIDA, acrescenta-se um terceiro byte com o índice do passo recusado (a partir de 0), ou `0xFF` quando o problema está no cabeçalho da rotina (tempo máximo ou número de passos fora da faixa) ou em bytes que sobram depois do último passo. Nenhuma ação é executada quando há NACK.

Quadro 11 – Códigos de erro do NACK.

Código	Nome	Significado
`0x01`	CMD_DESCONHECIDO	O código do comando não existe.
`0x02`	PARAMETRO_INVALIDO	Parâmetro fora da faixa permitida (por exemplo, canal inexistente ou valor do DAC acima de 4095).
`0x03`	NAO_CONFIGURADO	O periférico exige configuração prévia (I2C, SPI ou UART).
`0x04`	CRC_INVALIDO	Reservado; não é enviado. Um quadro com CRC inválido é descartado sem resposta, pois o SEQ e o CMD recebidos não são confiáveis.
`0x05`	TAMANHO_EXCEDIDO	Reservado; não é enviado. Um LEN maior que 240 também descarta o quadro sem resposta.
`0x06`	ERRO_PERIFERICO	O DUT não respondeu: falta de ACK no I2C ou *timeout* no periférico.
`0x07`	OCUPADO	Há uma rotina em execução, ou um comando anterior ainda está sendo executado; durante a rotina, apenas PING, ROUTINE_STATUS e ABORT são atendidos.
`0x08`	ROTINA_INVALIDA	A rotina contém um passo desconhecido, um parâmetro fora da faixa, está truncada ou produziria mais resultados do que a área disponível. O índice do passo é informado.
`0x09`	SEM_RESULTADO	READ_RESULT recebido sem que haja uma rotina finalizada.

Fonte: elaborado pelo autor.

A.5 COMANDOS DIRETOS

Os Quadros 12 e 13 listam os comandos diretos. Um traço (–) indica payload vazio. Nos comandos com dados de tamanho variável, os dados são sempre precedidos da quantidade, e o total do payload não pode ultrapassar 240 bytes.

Algumas regras complementam os quadros. Uma frequência de PWM que não pode ser gerada exatamente pelos divisores do temporizador é aproximada pela mais próxima, sem NACK; o prescaler escolhido é o menor possível, o que dá a melhor resolução da razão cíclica. PWM_STOP leva a razão cíclica a zero com o temporizador

Quadro 12 – Comandos diretos de sistema, DAC, GPIO, PWM e ADC.

Código	Comando	Payload da requisição	Payload da resposta (ACK)
`0x01`	PING	–	versão do protocolo (`u8`), versão do *firmware* maior, menor e correção (3 × `u8`)
`0x02`	RESET_ALL	–	–
`0x10`	DAC_SET	canal (`u8`, 1–2), valor (`u16`, 0–4095)	–
`0x20`	GPIO_SET	pino (`u8`, 1–4), nível (`u8`, 0 ou 1)	–
`0x21`	GPIO_READ	–	máscara das entradas GPIO5–8 (`u8`)
`0x30`	PWM_SET	canal (`u8`, 1–3), frequência em Hz (`u32`, 1–100 000), razão cíclica em centésimos de por cento (`u16`, 0–10000)	–
`0x31`	PWM_STOP	canal (`u8`, 1–3)	–
`0x40`	ADC_READ	canal (`u8`, 0–2)	canal (`u8`), valor bruto de 12 bits (`u16`, 0–4095)

Fonte: elaborado pelo autor.

em funcionamento, deixando a saída em nível baixo. Como mestre, o SPI usa a maior frequência que não ultrapassa a pedida (54 MHz divididos por 2 a 256). Os comandos de I2C, SPI e UART que dependem de configuração (escritas, leituras e transferências) recebem NACK NAO_CONFIGURADO se o periférico não tiver sido configurado, com exceção da carga e da leitura do mapa do escravo I2C, que funcionam sem I2C_CONFIG.

A.5.1 Emulação de dispositivos I2C e SPI

No papel de escravo, a jiga emula um sensor ou periférico, permitindo ao DUT, que atua como mestre, comunicar-se com ela como faria com o dispositivo real. No I2C, a jiga mantém um mapa de 256 registradores, carregado por I2C_SLAVE_LOAD e lido de volta por I2C_SLAVE_DUMP, o que permite tanto fornecer valores ao DUT quanto verificar o que ele escreveu. O mapa segue o padrão das memórias 24C02: o primeiro byte escrito pelo mestre é o ponteiro de registrador, e as leituras e escritas seguintes avançam o ponteiro, que volta de `0xFF` para `0x00`. No SPI, a jiga mantém um *buffer* de transmissão, carregado por SPI_SLAVE_LOAD, e um *buffer* de recepção, lido por SPI_SLAVE_READ; cada transação recomeça a transmissão do primeiro byte do *buffer*, completando com `0xFF` quando o mestre pede mais bytes do que foram carregados, e os bytes recebidos se acumulam até a leitura. Na UART, os bytes recebidos do DUT são guardados em um *buffer* de 256 bytes até o UART_READ. No papel de mestre, a jiga inicia as transações com um periférico do DUT por I2C_MASTER_XFER e SPI_MASTER_XFER, e o sinal de seleção de escravo do SPI é acionado no pino PD7.

Quadro 13 – Comandos diretos de I2C, SPI e UART.

Código	Comando	Payload da requisição	Payload da resposta (ACK)
`0x50`	I2C_CONFIG	papel (`u8`, 0 escravo, 1 mestre), endereço de 7 bits (`u8`, `0x08`–`0x77`, usado como escravo), velocidade (`u8`, 0 para 100 kHz, 1 para 400 kHz)	–
`0x51`	I2C_SLAVE_LOAD	*offset* (`u8`), quantidade (`u8`), dados (*n* bytes)	–
`0x52`	I2C_SLAVE_DUMP	*offset* (`u8`), quantidade (`u8`, até 240)	dados (*n* bytes)
`0x53`	I2C_MASTER_XFER	endereço (`u8`), bytes a escrever *nw* (`u8`), bytes a ler *nr* (`u8`, até 240), dados (*nw* bytes)	dados lidos (*nr* bytes)
`0x54`	I2C_SLAVE_MASK	*offset* (`u8`), quantidade (`u8`), máscaras (*n* bytes; bit 1 = o DUT pode escrever)	–
`0x56`	I2C_SLAVE_ADDR2	segundo endereço da escrava (`u8`, `0x08`–`0x77`, ou 0 para nenhum)	–
`0x60`	SPI_CONFIG	papel (`u8`, 0 escravo, 1 mestre), modo (`u8`, 0–3), frequência em Hz (`u32`, usada como mestre, a partir de 210 938 Hz)	–
`0x61`	SPI_SLAVE_LOAD	quantidade (`u8`, 0–239), dados (*n* bytes)	–
`0x62`	SPI_SLAVE_READ	–	quantidade (`u8`), dados recebidos (*n* bytes)
`0x63`	SPI_MASTER_XFER	quantidade (`u8`, 1–239), dados (*n* bytes)	dados recebidos (*n* bytes)
`0x64`	SPI_SLAVE_REG	modo registradores (`u8`, 0 ou 1), complemento do endereço (`u8`, `0x00` ou `0x80`)	–
`0x70`	UART_CONFIG	taxa em *baud* (`u32`, 1200–1 000 000), formato 8N1	–
`0x71`	UART_WRITE	quantidade (`u8`, 1–239), dados (*n* bytes)	– (enviado ao fim da transmissão)
`0x72`	UART_READ	quantidade máxima (`u8`, até 239)	quantidade (`u8`), dados (*n* bytes)

Fonte: elaborado pelo autor.

A.5.2 Emulação de sensores

Três comandos complementam a escrava para emular sensores comerciais. I2C_SLAVE_MASK define, para cada registrador do mapa, quais bits o DUT pode escrever: os demais mantêm o valor carregado pelo computador, que continua podendo gravar qualquer bit com I2C_SLAVE_LOAD. Todas as máscaras começam em `0xFF`. I2C_SLAVE_ADDR2 define um segundo endereço de escrava, aplicado no próximo I2C_CONFIG de escrava; os dois endereços dividem o mesmo mapa. SPI_SLAVE_REG coloca a escrava SPI, já configurada por SPI_CONFIG, no modo registradores: em cada transação, o primeiro byte é o endereço de 7 bits, completado

pelo segundo parâmetro (`0x80` para os registradores de `0x80` a `0xFF`), com o bit 7 em 1 para leitura e em 0 para escrita, e os bytes seguintes são lidos do mapa ou escritos nele (só nos bits graváveis), com auto-incremento. Entre o byte de endereço e o primeiro dado, o mestre precisa deixar uma pausa de alguns microssegundos (subseção 4.3.6).

A.6 COMANDOS DE ROTINA

O Quadro 14 lista os comandos que transferem, acompanham e interrompem uma rotina de teste.

Quadro 14 – Comandos de rotina.

Código	Comando	Payload da requisição	Payload da resposta (ACK)
`0x08`	RUN_ROUTINE	identificador (`u16`), tempo máximo em ms (`u32`), número de passos (`u8`), passos (subseção A.6.1)	–
`0x09`	ROUTINE_STATUS	–	estado (`u8`, Quadro 16), passos concluídos (`u8`), identificador (`u16`), código de erro (`u8`, 0 se não houver)
`0x0A`	READ_RESULT	*offset* (`u16`)	identificador (`u16`), tamanho total dos resultados (`u16`), até 236 bytes de resultados a partir do *offset*
`0x0B`	ABORT	–	–

Fonte: elaborado pelo autor.

A.6.1 Formato da Rotina

Os passos de uma rotina são escritos em sequência, sem separadores. Cada passo é formado por um código (`u8`) seguido dos seus parâmetros, e o tamanho de cada passo é deduzido do código e dos campos de quantidade. Os passos de estímulo, de configuração e de aquisição usam o mesmo código e a mesma codificação de parâmetros do comando direto correspondente, o que permite reaproveitar no *firmware* as mesmas funções de decodificação. Dois passos existem apenas dentro de uma rotina, porque dependem de tempo. O Quadro 15 resume os passos admitidos.

O passo ESPERA_GPIO aguarda até que a entrada indicada assuma o nível esperado, ou até o tempo limite. O instante registrado é o da transição detectada, e o registro traz também o tempo decorrido desde o início do passo; como o passo começa logo após o estímulo aplicado no passo anterior, esse tempo corresponde ao tempo de reação do DUT. Esgotar o tempo limite não interrompe a rotina: é um resultado, registrado como tal, a ser avaliado pelo computador.

Quadro 15 – Passos admitidos em uma rotina.

Códigos	Passos	Parâmetros	Registro gerado
`0x10`, `0x20`, `0x30`, `0x31`	DAC_SET, GPIO_SET, PWM_SET, PWM_STOP	iguais aos do comando direto	nenhum
`0x50`, `0x51`, `0x54`, `0x56`, `0x60`, `0x61`, `0x64`, `0x70`, `0x71`	configuração e carga de I2C, SPI e UART	iguais aos do comando direto	nenhum
`0x21`, `0x40`	GPIO_READ, ADC_READ	iguais aos do comando direto	o mesmo payload do ACK do comando direto
`0x52`, `0x53`, `0x62`, `0x63`, `0x72`	leituras e transferências de I2C, SPI e UART	iguais aos do comando direto	o mesmo payload do ACK do comando direto
`0x78`	ESPERA	duração em µs (`u32`)	nenhum
`0x79`	ESPERA_GPIO	entrada (`u8`, 5–8), nível esperado (`u8`), tempo limite em µs (`u32`)	resultado (`u8`: 0 se o nível ocorreu, 1 se o tempo esgotou) e tempo decorrido desde o início do passo, em µs (`u32`)
`0x7A`	ONDA	gerador, destino, registro, passo e período em µs, ciclos, modo, ruído, base, amplitude e de 2 a 32 pontos (subseção A.6.2)	nenhum

Fonte: elaborado pelo autor.

Se a entrada já estiver no nível esperado quando o passo começa, o registro é feito na hora, com tempo decorrido zero. O tempo máximo da rotina, informado em RUN_ROUTINE, vai de 1 ms a 1 h (3 600 000 ms): a base de tempo de 1 µs tem 32 bits e dá a volta em cerca de 71 minutos, de modo que uma hora garante que nenhum instante registrado se repita dentro de uma rotina.

Antes de iniciar a execução, a jiga valida a rotina inteira: códigos, parâmetros, integridade do encadeamento dos passos e tamanho máximo dos resultados que ela pode produzir, calculado a partir dos passos de aquisição. Se algum passo for inválido, a rotina é recusada com NACK ROTINA_INVALIDA e nada é executado; desse modo, uma rotina nunca fica executada pela metade por causa de um erro de codificação.

A.6.2 Passo ONDA

O passo ONDA começa uma onda em segundo plano e termina na hora: a rotina segue para o passo seguinte com a onda em andamento. Seus parâmetros, em ordem, são: gerador (`u8`, 0 a 5; uma onda nova substitui a do mesmo gerador ou do mesmo destino), destino (`u8`: 0 DAC, 1 razão do PWM, 2 registrador de 16 bits com sinal, 3 registrador de 16 bits sem sinal, 4 registrador de 20 bits no formato do

BME280), registro (`u8`: canal do DAC ou do PWM, ou o primeiro registrador no mapa), passo em µs (`u32`), período em µs (`u32`), ciclos (`u16`; 0 para até o fim da rotina), modo (`u8`: 0 interpolação linear, 1 degrau), ruído (`u16`, amplitude uniforme em valor bruto), base (`s32`), amplitude (`s32`), número de pontos (`u8`, 2 a 32) e os pontos (`u16` cada). Os pontos são igualmente espaçados no período, e o valor bruto escrito no destino é base $+$ amplitude $\cdot$ ponto$/65535$, saturado na faixa do destino. O passo mínimo é de 100 µs, exceto no DAC, que aceita de 1 a 99 µs desde que o período não passe de 1024 passos. Ao fim dos ciclos, o destino fica no último ponto.

A.6.3 Formato dos Resultados

Cada passo de aquisição gera um registro, gravado em uma área de resultados de 2 048 bytes, composto pelo índice do passo na rotina (`u8`, a partir de 0), pelo instante da medição em microssegundos contados a partir do início da rotina (`u32`), pela quantidade de bytes de dados (`u8`) e pelos dados. A jiga não avalia os registros: o veredito é emitido pelo computador, que compara as medições com os critérios do caso de teste. Como os resultados podem ultrapassar o tamanho de um quadro, o computador os lê em partes, com comandos READ_RESULT de *offset* crescente, até completar o tamanho total informado.

A.6.4 Estados de Execução

O Quadro 16 apresenta os estados informados por ROUTINE_STATUS. Os resultados da última rotina permanecem disponíveis até o próximo RUN_ROUTINE ou RESET_ALL, que os descarta; nos estados ABORTADA, TEMPO_ESGOTADO e ERRO, eles contêm os registros dos passos executados até a interrupção.

Quadro 16 – Estados de execução de uma rotina.

Valor	Estado	Significado
0	OCIOSA	Nenhuma rotina desde a inicialização ou desde o último RESET_ALL.
1	EM_EXECUCAO	A rotina está sendo executada.
2	CONCLUIDA	Todos os passos foram executados.
3	ABORTADA	A execução foi interrompida por ABORT.
4	TEMPO_ESGOTADO	A rotina ultrapassou o tempo máximo informado em RUN_ROUTINE e foi interrompida.
5	ERRO	Um passo falhou durante a execução (por exemplo, ERRO_PERIFERICO); o código de erro é informado.

Fonte: elaborado pelo autor.

A.6.5 Regras de Execução

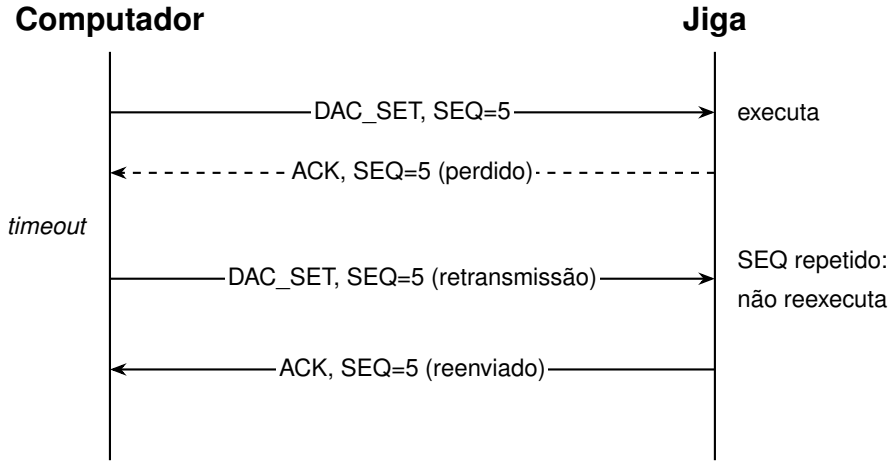
Enquanto uma rotina está em execução, a jiga atende apenas PING, ROUTINE_STATUS e ABORT, além das retransmissões (Apêndice A.7); qualquer outro

comando, inclusive um novo RUN_ROUTINE, recebe NACK OCUPADO. Ao receber ABORT, a jiga interrompe a rotina, inclusive durante uma espera, e responde ACK; o estado passa a ABORTADA. Ao término da rotina, em qualquer dos estados finais, a jiga retorna ao estado seguro (Apêndice A.8), de modo que cada rotina começa sem estímulos herdados da anterior.

A.7 SEQUENCIAMENTO E RETRANSMISSÃO

O computador espera a resposta de cada comando antes de enviar o próximo (modelo *stop-and-wait*). Se a resposta não chegar dentro do tempo limite, ou chegar com CRC inválido, o computador retransmite o comando **com o mesmo SEQ**, até um número máximo de tentativas, após o qual sinaliza falha de comunicação ao usuário. A Figura 20 ilustra o caso de perda da resposta.

Figura 20 – Retransmissão após perda da resposta.



Fonte: elaborado pelo autor.

A jiga guarda o SEQ e a resposta do último comando executado. Se receber um quadro com o mesmo SEQ, reenvia a resposta guardada **sem executar o comando novamente**. Isso torna a retransmissão segura: um comando cuja resposta se perdeu não é aplicado duas vezes (por exemplo, uma escrita I2C duplicada) e, no caso de RUN_ROUTINE, a rotina em andamento não é reiniciada. Uma retransmissão que chega enquanto o comando original ainda está em execução é ignorada: a resposta sai uma vez, quando ele termina.

O SEQ guardado vale por **sessão**. A sessão começa quando a aplicação abre a porta serial, o que o computador sinaliza ativando a linha DTR (requisição SET_CONTROL_LINE_STATE do USB-CDC); nesse momento a jiga esquece o último SEQ e a resposta guardada. Sem essa regra, uma aplicação reaberta que começasse pelo mesmo SEQ do último comando da sessão anterior receberia a resposta antiga, sem que o comando fosse executado. A resposta OCUPADO não é guardada, pois o

comando recusado não foi executado e pode ser reenviado com o mesmo SEQ depois. Quando dois comandos chegam no mesmo bloco de dados da USB, o OCUPADO do segundo pode chegar antes do ACK do primeiro; o computador associa cada resposta ao comando pelo SEQ, e não pela ordem de chegada.

Os valores de temporização são parâmetros configuráveis. Adotam-se um tempo limite de resposta de 100 ms, 3 tentativas por comando (o envio original e até duas retransmissões), um intervalo de 20 ms entre consultas ROUTINE_STATUS e um tempo limite de 20 ms entre bytes na jiga, após o qual um quadro incompleto é descartado.

A.8 ESTADO SEGURO

Na inicialização, ao receber RESET_ALL e ao final de cada rotina, a jiga primeiro para as ondas em andamento e depois leva todas as saídas a um estado seguro: DAC no código 0 (cerca de 50 mV, limite inferior do *buffer* de saída), saídas GPIO1 a GPIO4 em nível baixo, PWM com razão cíclica zero (saída em nível baixo) e barramentos I2C, SPI e UART desligados, com o mapa do escravo I2C zerado, todas as máscaras em `0xFF`, sem segundo endereço, a escrava SPI fora do modo registradores, os *buffers* do SPI e da UART vazios e o pino de seleção do SPI (PD7) em nível alto. Isso impede que estímulos de uma execução anterior permaneçam aplicados ao DUT.

A.9 EXEMPLOS DE QUADROS

Os Quadros 17 e 18 apresentam exemplos reais de quadros do protocolo, com o CRC calculado segundo a seção A.3, em hexadecimal.

Quadro 17 – Exemplos de quadros de comandos diretos (bytes em hexadecimal).

Mensagem	**Quadro**
PING, SEQ=1	`7E 00 01 01 8C EF`
ACK do PING (protocolo 1, *firmware* 0.1.0)	`7E 04 01 81 01 00 01 00 AC B8`
DAC_SET, SEQ=2, canal 1, valor 2048	`7E 03 02 10 01 00 08 EC 29`
ACK do DAC_SET, SEQ=2	`7E 00 02 90 47 29`
NACK, SEQ=2, comando `0x10`, erro `0x02`	`7E 02 02 FF 10 02 B5 54`
ADC_READ, SEQ=3, canal 0	`7E 01 03 40 00 E8 A6`
ACK do ADC_READ, SEQ=3, canal 0, valor 2047	`7E 03 03 C0 00 FF 07 9E EE`
PWM_SET, SEQ=4, canal 1, 1 kHz, 50%	`7E 07 04 30 01 E8 03 00 00 88 13 5A 47`

Fonte: elaborado pelo autor.

O Quadro 18 acompanha uma rotina de três passos, com identificador 7 e tempo máximo de 1 s: aplicar 2,0 V no DAC1 (código 2482), aguardar até 500 ms que

a entrada GPIO5 vá a nível alto e ler o canal A0. No exemplo, a transição do GPIO5 ocorreu 42,350 ms após o início da rotina, 42,338 ms após o início da espera (tempo de reação do DUT), e a leitura do ADC, 21 µs depois da transição.

Quadro 18 – Exemplos de quadros de uma rotina (bytes em hexadecimal).

Mensagem	**Quadro**
RUN_ROUTINE, SEQ=5	`7E 14 05 08 07 00 E8 03 00 00 03 10 01 B2 09 79 05 01 20 A1 07 00 40 00 FB CD`
ACK do RUN_ROUTINE, SEQ=5	`7E 00 05 88 E9 23`
ROUTINE_STATUS, SEQ=6	`7E 00 06 09 13 F7`
ACK do ROUTINE_STATUS, SEQ=6 (em execução, 1 passo concluído)	`7E 05 06 89 01 01 07 00 00 12 1D`
ACK do ROUTINE_STATUS, SEQ=7 (concluída, 3 passos)	`7E 05 07 89 02 03 07 00 00 C9 A6`
READ_RESULT, SEQ=8, *offset* 0	`7E 02 08 0A 00 00 8D 17`
ACK do READ_RESULT, SEQ=8 (2 registros, 20 bytes)	`7E 18 08 8A 07 00 14 00 01 6E A5 00 00 05 00 62 A5 00 00 02 83 A5 00 00 03 00 FF 07 51 36`
ABORT, SEQ=9	`7E 00 09 0B 6F C7`
NACK, SEQ=9, comando `0x08`, erro OCUPADO	`7E 02 09 FF 08 07 D5 90`

Fonte: elaborado pelo autor.

APÊNDICE B – MAPEAMENTO DE PINOS

Os Quadros 19 e 20 trazem a posição de cada sinal da jiga nos conectores da placa NUCLEO-F767ZI.

Quadro 19 – Mapeamento de pinos dos conectores CN7 e CN8.

Conector	**Posição**	**Sinal**	**Pino**	**Observação**
CN7	2D	SPI2_MOSI	PB15	SPI (parte)
CN7	9D	DAC1	PA4	Saída analógica
CN7	5E	DAC2	PA5	Saída analógica
CN7	6E	PWM3/1 (TIM3_CH1)	PA6	Saída PWM
CN8	4D, 5D, 6D, 7D	3V3, 5V, GND, GND	–	Alimentação
CN8	1E–4E	GPIO1–4 (saída)	PC8, PC9, PC10, PC11	Estímulo digital
CN8	5E–8E	GPIO5–8 (entrada)	PC12, PD2, PG2, PG3	Aquisição digital; tolerantes a 5 V

Fonte: elaborado pelo autor.

Quadro 20 – Mapeamento de pinos dos conectores CN9 e CN10.

Conector	**Posição**	**Sinal**	**Pino**	**Observação**
CN9	1D, 2D, 3D	A0, A1, A2 (ADC)	PA3, PC0, PC3	Aquisição analógica
CN9	10D, 11D	I2C2_SCL, I2C2_SDA	PF1, PF0	Barramento I2C
CN9	13D, 14D	UART4_RX, UART4_TX	PD0, PD1	Comunicação serial com o DUT
CN9	1E	SPI2_CS (GPIO)	PD7	Seleção de *chip* SPI
CN10	5D	SPI2_MISO	PC2	SPI (restante)
CN10	16D	SPI2_SCK	PB10	SPI (restante)
CN10	15D	PWM2/1 (TIM2_CH1)	PA0	Saída PWM
CN10	2E	PWM1/1 (TIM1_CH1)	PE9	Saída PWM

Fonte: elaborado pelo autor.

No projeto do STM32CubeMX, as entradas GPIO5 a GPIO8 aparecem com os rótulos `GPIO1_IN` a `GPIO4_IN`, que numeram as entradas entre si; a numeração do protocolo e deste texto é a destes quadros.

APÊNDICE C – CASOS DE TESTE

Este apêndice descreve os casos de teste resumidos na subseção 3.3.3. Os Quadros 21 a 23 apresentam, para cada caso, o nível de teste, o procedimento, o resultado esperado e os requisitos comprovados.

Quadro 21 – Casos de teste de unidade e de integração com comandos diretos.

ID	Nível	Procedimento	Resultado esperado	Req.
CT01	Unidade	Calcular o CRC-16 da sequência ASCII `123456789`.	`0x29B1`.	RNF03
CT02	Unidade	Codificar e decodificar os 17 quadros dos Quadros 17 e 18.	Bytes idênticos aos dos quadros; campos recuperados.	RF06
CT03	Unidade	Alterar um bit de um quadro válido.	Quadro descartado, sem resposta; o computador retransmite por tempo limite.	RNF03
CT04	Unidade	Inserir bytes espúrios antes do SOF e truncar um quadro.	Quadro incompleto descartado; ressincronização no SOF seguinte.	RNF03
CT05	Unidade	Transporte simulado descarta a primeira resposta.	Retransmissão com o mesmo SEQ; comando executado uma única vez.	RNF03
CT06	Unidade	Compilar a bateria da Código 1 e um caso que não cabe em um quadro.	Rotina com os bytes esperados; caso grande recusado na validação.	RF07, RF10
CT07	Unidade	Executar baterias contra uma jiga simulada com respostas conhecidas, incluindo rotinas concluídas, em erro e interrompidas.	Vereditos passou, falhou e inconclusivo conforme o previsto.	RF07, RF08
CT08	Integração	Enviar PING.	ACK com as versões do protocolo e do *firmware*.	RF06
CT09	Integração	Enviar um código inexistente e um parâmetro fora da faixa.	NACK com erros `0x01` e `0x02`; nenhuma saída alterada.	RF06
CT10	Integração	DAC1 e DAC2 em 11 pontos de 124 a 3971 (faixa útil): leitura pelo ADC em *loopback* (PA4→PA3, PA5→PC0) e medição com multímetro (DAC1 nos 11 pontos; DAC2 no ponto médio).	Erros dentro do Quadro 5, no ADC e no multímetro.	RF01, RF05
CT11	Integração	*Loopback* GPIO1–4→GPIO5–8 (PC8–PC11→PC12, PD2, PG2, PG3), nas 16 combinações.	Máscara lida igual à escrita.	RF02, RF05
CT12	Integração	PWM nos três canais em frequências e razões cíclicas definidas, medidos com osciloscópio.	Dentro do Quadro 5.	RF03
CT13	Integração	*Loopback* MOSI→MISO (PB15→PC2; jiga mestra, modos 0 a 3 a 1 MHz e 239 bytes a 6,75 MHz) e UART4 TX→RX (PD1→PD0; 9 600, 115 200 e 1 000 000 *baud*).	Dados recebidos iguais aos enviados.	RF04, RF05

Fonte: elaborado pelo autor.

Quadro 22 – Casos de teste das rotinas.

ID	Nível	Procedimento	Resultado esperado	Req.
CT14	Integração	Enviar uma rotina com um passo de código inexistente no meio.	NACK ROTINA_INVALIDA com o índice do passo; nenhum passo executado.	RF06, RF10
CT15	Integração	Durante uma rotina, enviar um comando direto e um novo RUN_ROUTINE; consultar o estado.	NACK OCUPADO para ambos; ROUTINE_STATUS atendido normalmente.	RF06, RNF03
CT16	Integração	Enviar ABORT durante uma espera longa.	Estado ABORTADA; saídas no estado seguro.	RF10, RNF05
CT17	Integração	Executar uma rotina cuja duração excede o tempo máximo informado.	Estado TEMPO_ESGOTADO; saídas no estado seguro.	RF10, RNF05
CT18	Integração	Retransmitir um RUN_ROUTINE com o mesmo SEQ durante a execução.	Resposta reenviada; rotina não reiniciada.	RNF03
CT19	Integração	(a) Sem ligações: leituras separadas por ESPERAs de 1, 10 e 100 ms; (b) *loopback* GPIO1→GPIO5 (PC8→PC12): tempo de reação do ESPERA_GPIO nas duas bordas; (c) osciloscópio no GPIO1: pulsos de 1, 10 e 100 ms gerados por ESPERAs.	Intervalos, reações e larguras de pulso dentro do Quadro 5.	RF10

Fonte: elaborado pelo autor.

Quadro 23 – Casos de teste de sistema, de aceitação, de desempenho e do gerador de ondas.

ID	**Nível**	**Procedimento**	**Resultado esperado**	**Req.**
CT20	Sistema	DUT de referência (ESP32-S3), comandado por texto pela UART da jiga, lê e escreve a jiga escrava por I2C e por SPI e responde como escravo à jiga mestra.	Dados lidos e escritos iguais nos dois sentidos e nos dois barramentos.	RF04
CT21	Sistema	Executar dez vezes a bateria de *loopback*, com os *jumpers* do CT10, do CT11 e do CT13.	Mesmos vereditos e medições dentro das tolerâncias em todas as execuções; relatório em cada uma.	RF08, RF09, RF10, RNF04
CT22	Aceitação	Executar a bateria *black-box* do comportamento “ventilador com histerese” no ESP32-S3, com o *firmware* correto e com quatro versões com defeitos injetados; conferir o tempo de reação com o osciloscópio.	Correto: todos os casos passam. Defeituosos: cada um reprovado nos casos que o defeito afeta. Reação dentro do Quadro 5.	RF08, RF10, RNF02
CT23	Desempenho	Enviar 1 000 comandos PING consecutivos, medindo o tempo de ida e volta.	Latência máxima inferior a 50 ms; média e percentil 95 registrados.	RNF07
CT24	Desempenho	Enviar 10 000 comandos consecutivos, alternando escrita e leitura.	Nenhum erro não recuperado; taxa de retransmissão registrada.	RNF03
CT25	Sistema	A jiga emula o MPU-6500 e o BME280 no I2C, com valores fixos e alterados durante o caso; o DUT os lê com *drivers* comuns e informa os valores pela UART.	Valores informados iguais aos simulados, dentro da tolerância.	RF11
CT26	Sistema	BME280 emulado no SPI e MPU-6500 no I2C, ao mesmo tempo, lidos pelo DUT, inclusive durante uma rampa de temperatura.	Valores informados iguais aos esperados.	RF11, RF12
CT27	Integração	Ondas num registrador do sensor emulado, lido em instantes conhecidos da rotina; senoide no DAC por DMA e triangular na razão do PWM, medidas no osciloscópio.	Valores e formas dentro do Quadro 5.	RF12

Fonte: elaborado pelo autor.

APÊNDICE D – BATERIA DE TESTES E RELATÓRIO DE EXECUÇÃO

O Quadro 24 resume os itens admitidos em uma bateria de testes (subseção 4.5.3), com os campos de cada um e o passo da rotina que ele gera.

Quadro 24 – Itens de uma bateria de testes.

Item	**Campos**	**Passo gerado e critério**
Estímulos		
`dac`	canal, tensão em V (ou código)	DAC_SET
`gpio_out`	pino 1–4, nível	GPIO_SET
`pwm`, `pwm_stop`	canal, frequência em Hz, razão em %	PWM_SET, PWM_STOP
`espera_ms`, `espera_us`	duração	ESPERA
`i2c_config`, `i2c_load`	papel, endereço, velocidade; *offset* e dados	I2C_CONFIG, I2C_SLAVE_LOAD
`spi_config`, `spi_load`	papel, modo, frequência; dados	SPI_CONFIG, SPI_SLAVE_LOAD
`uart_config`, `uart_write`	taxa; texto ou bytes	UART_CONFIG, UART_WRITE
`mpu6500`, `bme280`	aceleração, rotação e temperatura; temperatura, pressão e umidade (declarados na chave `sensores` da bateria)	I2C_SLAVE_LOAD dos registradores de dados
`onda`	alvo, forma (rampa, senoide, triangular, quadrada, dente de serra, exponencial ou pontos), período ou duração, ciclos, passo e ruído	ONDA (uma ou mais)
Verificações		
`gpio_in`	pino 5–8, nível esperado	GPIO_READ; igualdade
`adc`	canal, tensão esperada, tolerância (padrão de 33 mV)	ADC_READ; dentro da tolerância
`reacao_gpio`	pino 5–8, nível, tempo máximo em ms	ESPERA_GPIO; transição antes do limite
`uart_read`	texto ou bytes esperados	UART_READ; igualdade
`i2c_dump`, `i2c_xfer`	*offset* ou endereço; bytes esperados	I2C_SLAVE_DUMP, I2C_MASTER_XFER; igualdade
`spi_read`, `spi_xfer`	bytes esperados	SPI_SLAVE_READ, SPI_MASTER_XFER; igualdade
`uart_valores`	comando de texto ao DUT, números esperados e tolerância	UART_WRITE, ESPERA e UART_READ; cada número dentro da tolerância

Fonte: elaborado pelo autor.

O Código 2 é a bateria do teste de aceitação (CT22), executada sem alterações sobre o *firmware* correto e sobre as versões com defeitos injetados. Ela usa apenas o DAC1 e a entrada GPIO5 da jiga, sem nenhum comando ao DUT. O Código 3 é um trecho do relatório CSV de uma execução no ESP32-S3.

Código 2 – Bateria do CT22 (ventilador com histerese).

```
bateria: ct22-ventilador
dut: ESP32-S3 (firmware ventilador)
casos:
  - nome: desligado em 1,0 V
    estimulos:
      - dac: {canal: 1, tensao: 1.0}
    espera_ms: 50
    verificar:
      - gpio_in: {pino: 5, esperado: 0}
  - nome: liga em 2,0 V em até 20 ms
    estimulos:
      - dac: {canal: 1, tensao: 1.0}
      - espera_ms: 50
      - dac: {canal: 1, tensao: 2.0}
    verificar:
      - reacao_gpio: {pino: 5, nivel: 1, max_ms: 20}
  - nome: histerese, continua ligado em 1,58 V
    estimulos:
      - dac: {canal: 1, tensao: 2.0}
      - espera_ms: 50
      - dac: {canal: 1, tensao: 1.58}
    espera_ms: 50
    verificar:
      - gpio_in: {pino: 5, esperado: 1}
  - nome: desliga em 1,2 V em até 20 ms
    estimulos:
      - dac: {canal: 1, tensao: 2.0}
      - espera_ms: 50
      - dac: {canal: 1, tensao: 1.2}
    verificar:
      - reacao_gpio: {pino: 5, nivel: 0, max_ms: 20}
  - nome: histerese, não liga em 1,72 V
    estimulos:
      - dac: {canal: 1, tensao: 1.0}
      - espera_ms: 50
      - dac: {canal: 1, tensao: 1.72}
    espera_ms: 50
    verificar:
      - gpio_in: {pino: 5, esperado: 0}
```

Fonte: elaborado pelo autor.

Código 3 – Trecho do relatório CSV de uma execução do CT22 no ESP32-S3.

```
caso;veredito;verificacao;esperado;obtido;instante_us
1;passou;GPIO5 = 0;0;0;50025
2;passou;GPIO5 vai a 1 em ate 20 ms;<= 20 ms;0.151 ms;50188
3;passou;GPIO5 = 1;1;1;100050
4;passou;GPIO5 vai a 0 em ate 20 ms;<= 20 ms;0.586 ms;50622
5;passou;GPIO5 = 0;0;0;100051
```

Fonte: elaborado pelo autor. Execução de 4 nov. 2026; colunas selecionadas do relatório, que traz também a bateria, o DUT, a versão do *firmware* e os estímulos de cada caso.